



UNIVERSITÀ DI PISA

Model Weight Learning as a novel paradigm in Computer Vision

Supervisors

Prof. Mario G.C.A. Cimino

Dr. Marco Parola

Candidate

Luca Di Giacomo

Pisa University

Department of Engineering - Artificial Intelligence and Data Engineering

Academic Year 2024/2025

Abstract

This thesis investigates model weight learning as a data modality to support different downstream tasks in the computer vision context. By treating model weights as data, we explore their potential to adapt knowledge for specific tasks. We first apply this perspective to the zero-shot learning problem, showing how model weights can be used to produce new weights for an image classifier, enabling the classification of new/unseen classes. Next, we explore the problem of machine unlearning, where weight-based strategies allow for efficient and targeted forgetting of specific classes without retraining from scratch. Finally, we draw a future direction for a general framework to jointly address both zero-shot learning and unlearning.

Acknowledgements

Before beginning the thesis analysis, I would like to express my deepest gratitude to all those who have supported me throughout this journey. In particular, I wish to thank:

- *Prof. Mario G.C.A. Cimino, who has always been extraordinarily kind and available for any request or doubt, not only as my supervisor, but above all, during my entire journey in the university.*
- *Dr. Marco Parola, as my second supervisor, who has always been approachable and friendly, and whose guidance inspired my curiosity and made this thesis both enjoyable and highly instructive.*
- *The whole University staff, including the professors and their associates, for their continuous dedication, and for creating a stimulating and welcoming environment that made this academic journey both enriching and enjoyable.*
- *My family, whose constant support and love has guided me in every decision and through all the challenges along the way.*
- *Finally, a special thanks goes to the colleagues and friends I met here at the University, with whom I had the pleasure and honor of sharing this journey. Their presence, both during study hours and beyond, has always made me feel appreciated and encouraged.*

Contents

1	Introduction	1
1.1	Limitation of the classical approach	1
1.1.1	Privacy and Data Memorization	2
1.1.2	Resource Constraints and Stifled Innovation	2
1.1.3	Lack of Explainability and Trust	2
1.2	Goals and Leading Idea	3
2	Experiments Setup and Preliminary Observations	4
2.1	Introduction to the Experimental Design	4
2.1.1	Baseline Framework Design	4
2.1.2	Splits Design	6
2.2	Datasets Selection and Preprocessing	6
2.2.1	Animals With Attributes	8
2.2.2	Annotated Pascal-Yahoo	8
2.2.3	Caltech-UCSD Birds-200-2011	9
2.2.4	SUN-Attributes	9
2.3	ResNet Choice as Base Classifier	10
2.4	Evaluation Metrics and Preliminary Results on Origins	11
2.4.1	Metrics Choice and Description	11
2.4.2	Membership Inference Attack Metric	12
2.4.3	Origins Training Results	13
3	Literature Analysis for Model Adaptability Tasks	16
3.1	Introduction to Baselines	16
3.2	A Unified Perspective: Adaptability in Weight-Space	17
3.3	Hypernetworks	18
3.3.1	DAEGNN [12]	18
3.3.2	DiHyFo [26]	19
3.4	Zero-Shot Learning (ZSL)	20
3.4.1	Formal Definition	20
3.4.2	Evaluation Metrics	21
3.5	Literature Methods	22
3.5.1	Transformers and Attention Heads	23
3.5.2	Adaptation to Computer Vision: Visual Transformer ViT [9]	26
3.6	Weight Learning in Image free ZSL	27
3.6.1	ConSE [21]	28
3.6.2	COSTA [18]	29
3.6.3	Analysis of I-ZSL Baselines Results	30
3.7	ICIS [5]	31
3.7.1	Architecture	31
3.7.2	Data Preparation	33
3.7.3	Training Process	34
3.7.4	Key Limitations	35
3.8	Machine Unlearning (MU)	36
3.8.1	Problem Statement	37
3.8.2	Evaluation Baseline	38
3.9	Literature Methods	41
3.9.1	Retraining	42
3.9.2	Fine-tuning	43
3.9.3	NegGrad+ [17]	46
3.9.4	Bad Teacher [6]	49
3.9.5	SCRUB [17]	52

3.9.6	Analysis of MU Baselines Results	55
3.10	HyCUS: ICIS Adaptation to the Unlearning Task	56
3.10.1	Applied Adaptations	57
3.10.2	Evaluations and Core Considerations	58
3.10.3	Need For A Deeper Analysis	60
3.10.4	Conclusive Analysis	62
4	The Weight-as-Data Framework	63
4.1	Introduction to the Weight Space	63
4.1.1	Weight Space Properties and Symmetrical Redundancies	63
4.1.2	Geometry of the Loss Landscape	64
4.1.3	Standard Deep Learning in Weight Space	64
4.2	SANE Framework [27]	65
4.2.1	Architecture	65
4.2.2	Data Preprocessing	67
4.2.3	Training Process	68
4.3	Adaptation to Generative Tasks	70
4.3.1	Preprocessing and Training Process	70
4.3.2	Single Origin Experiments	71
4.3.3	Augmented Single Origin Experiments	76
4.3.4	Multi-Origin Experiments	81
4.4	Visualization of the Hyper-Representation Space	84
4.4.1	Experimental Methodology	85
4.4.2	Key Observations	88
4.4.3	Differential Analysis of the Residual Blocks	90
4.4.4	Patterns of Separation Across Residual Blocks	92
4.5	Framework Simplifications	93
4.6	Results Summary and Considerations	94
5	Conclusions and Future Work	95
5.1	Core Contributions and Applications	97
5.1.1	Cross-Domain Learning	98
5.1.2	Explainable AI - XAI	99
5.1.3	Federated Learning	100
5.1.4	Continuous Learning	101
5.2	Future Directions and Final Thoughts	102

1 Introduction

Deep learning has rapidly evolved from a niche academic field to a cornerstone of modern artificial intelligence. While the foundational principles of neural networks and backpropagation have existed for decades, their exponential growth was ignited by the convergence of three key factors: (i.) the availability of massive datasets, (ii.) significant increases in computational power (especially GPUs and TPUs), and (iii.) and pivotal algorithmic breakthroughs.

A prime example of these breakthroughs is the advent of Transformer models. With their unprecedented performance, Transformers have influenced nearly every sector, driving research, shaping strategic decisions, and replacing previous state-of-the-art methods in fields such as natural language processing, computer vision, and generative AI.

However, the dominant paradigm in modern deep learning, which relies on large-scale architectures and vast datasets, presents significant limitations that cause this approach to increasingly pose barriers for researchers without access to immense computational resources and storage space.

In this thesis, these challenges will be addressed by questioning the traditional deep learning approach. Using the classification of vision tasks as research subject, novel solutions will be explored, that leverage generative autoencoders to augment the current paradigm. A specific goal of this work will be to overcome some of its inherent limitations, proposing a more efficient and accessible framework that could support and open new research avenues across various fields.

1.1 Limitation of the classical approach

In the traditional machine learning paradigm, models are trained over multiple epochs using different learning strategies to generalize concepts from a dataset, allowing them to be applied to new, unseen data. This process involves tuning a model’s parameters by minimizing a loss function, which guides learning by adjusting the model in its high-dimensional weight space along the steepest descent direction of the loss. A successful process results in a model that can generalize well, whose final state can be represented as a single point in this complex vector space.

This training strategy is highly effective for tuning parameters in a supervised and targeted way. Still, it is not without its limitations, which have become increasingly significant with the rise of modern, large-scale models.

In this section a brief introduction to these limitations will be detailed, together with motivations that drive the need of a **unified framework**, which directly acts on the trained model’s **weight space** to overcome them. This approach would also effectively **decouple model adaptability from the need for new data**, addressing a key challenge in modern deep learning.

1.1.1 Privacy and Data Memorization

The supervised training process requires large datasets and, in a sense, forces the model to memorize its training data to synthesize mappings and features. This memorization can be exploited by attackers using techniques like **Membership Inference Attacks (MIAs)**, which can infer whether a specific data point was part of the original training set. While once it was considered a minor concern, these data leaks have now become a major issue, especially for large transformer-based models, leading to new regulatory frameworks like the GDPR and the urgent need for strategies to "unlearn" data from a model's "memory".

Although the training process must synthesize knowledge to achieve optimal performance, it can also result in an undesirable level of data memorization and overfitting. However, further steps can be applied after training, in a **post-hoc manner**, to find a nearby states within the model weight space that mitigates these issues.

1.1.2 Resource Constraints and Stifled Innovation

The increasing complexity of modern models and their growing demand for massive training datasets have led to a new paradigm: **pre-training**. In this approach, a model is first pre-trained for general learning on a vast dataset and then fine-tuned for specific, downstream tasks. While highly effective, this method requires immense amounts of data and computational resources, putting it out of reach for smaller organizations and individual researchers. Moreover, once a model is pre-trained, it exists in a fixed state, and adapting it to new information or tasks requires another costly and time-consuming retraining process. This creates a significant competitive and innovative disadvantage, limiting cutting-edge research to a few large entities that can afford the required resources.

Direct action on the weight space could address these limitations by exploiting a carefully **driven generative approach** that would remove the need for continuous, resource-intensive retraining, and instead adapt the pre-trained model to new downstream tasks with a significantly lower computational cost.

1.1.3 Lack of Explainability and Trust

Another major issue with the current paradigm is the inverse relationship between model parameters and explainability. In fact, as the number of parameters increases, it becomes exponentially harder to understand how or why a model learned a task. It is often impossible to tell if a model has truly learned a general concept or if it has simply found a superficial "shortcut" that works on the training data but lacks a true, robust understanding. This makes it difficult to trust the model's decisions, ensure its fairness, and effectively debug its failures.

By leveraging **generative autoencoders**, it is possible to develop a **neuron-centric framework** designed to simplify networks. This approach effectively reduces redundancy within network layers, and produces a **latent space where the parameters structure can be analyzed**, facilitating the explainability of specific downstream tasks.

1.2 Goals and Leading Idea

This thesis undertakes a comprehensive analysis and expansion of a foundational framework for computer vision, initially proposed by Christensen et al. at the International Conference on Computer Vision (ICCV) 2023 [5]. The original framework, denominated ICIS (Image-free Classifier Injection with Semantics), was developed to address the challenges of Zero-Shot Learning (ZSL): a downstream task that enables a model to classify unseen classes without prior exposure. As illustrated in section 3.7, this framework will serve as an initial exploration of the weight-learning paradigm, posing the foundations and the main inspiration of this thesis work.

Following a thorough analysis of Christensen et al.’s original contribution, the thesis will transition to a critical re-evaluation of its applicability to the distinct downstream task of Machine Unlearning (MU): a thriving field focused on the deliberate removal of specific data’s influence from a trained model’s parameters. section 3.10 will meticulously explore a revised version of the ICIS framework, adapted for MU, with a specific focus on identifying its inherent limitations and potential weaknesses. This critical assessment will highlight the conceptual and practical gaps occurring when trying to adapt weight-learning to a different, more complex problem.

Building upon these insights, this thesis work will move to the development of a novel generative framework, based on Schürholt and his associates’ work [27], that could operate on large scales to simultaneously address multiple downstream tasks without any need for additional data, and keeping the post-hoc application hypothesis.

In conclusion, it will critically explore and analyze the potential and core limitations of the novel framework to find exciting and useful possible outcomes, spanning multiple fields of research in Artificial Intelligence, such as Federated Learning, and genetic exploration of the weight space.

2 Experiments Setup and Preliminary Observations

2.1 Introduction to the Experimental Design

To start exploring the discussed downstream tasks of Machine Unlearning (MU) and Zero-Shot Learning (ZSL), the first crucial step has been to obtain working versions of a base classifier over the analyzed datasets. The primary goal of those versions, which will be referred to as **origins** during the course of this thesis, is to obtain the initial hypothesis of a Deep Neural Network that is able to solve a classification task over the Computer Vision datasets required.

2.1.1 Baseline Framework Design

Given the inherent complexity and the variety of methods involved, a framework based on **hydra**¹ was established to handle multi-runs and configuration management within a single package. The overall development was designed to be as simple as possible to minimize potential bugs arising from the complexity of the tasks.

For this reason, the framework is divided into three completely independent sub-tasks:

1. **Origin Training:** In this stage, a base classifier is selected along with a set of pre-trained weights and is then fine-tuned on a specified dataset. Hydra’s multi-run capability allows for fine-tuning on multiple datasets simultaneously, even in a multiprocessing environment.
2. **Machine Unlearning Methods:** A dispatcher is used to select and run a specific method over the chosen dataset. The selection is based on a match between the method name and its implementation file within the machine unlearning sub-folder. The application of these methods will although require the presence of the relative origin, trained on the selected dataset.
3. **Zero-Shot Learning Methods:** This task uses its own dispatcher, which operates identically to the one for Machine Unlearning, facilitating the correct selection and execution of methods. Also the application of these methods will require a trained origin to be applied.

With this setup, it has been possible to focus on the implementation of individual methods, all of which are presented in a similar configuration to facilitate dispatching, as shown in fig. 1. Each method’s Run is divided into 3 steps:

- A **preparation step** where Hydra configuration is used to selectively load the required set of data and store the necessary information into a method-specific state.
- The **core execution of the method** that takes the state instance built in the previous step, and executes the core algorithm of the method.
- The **evaluation** of the method, which consists in instantiating a predefined set of metrics, and its application to the required data, manageable by the hydra configuration files.

¹<https://hydra.cc/docs/intro/>

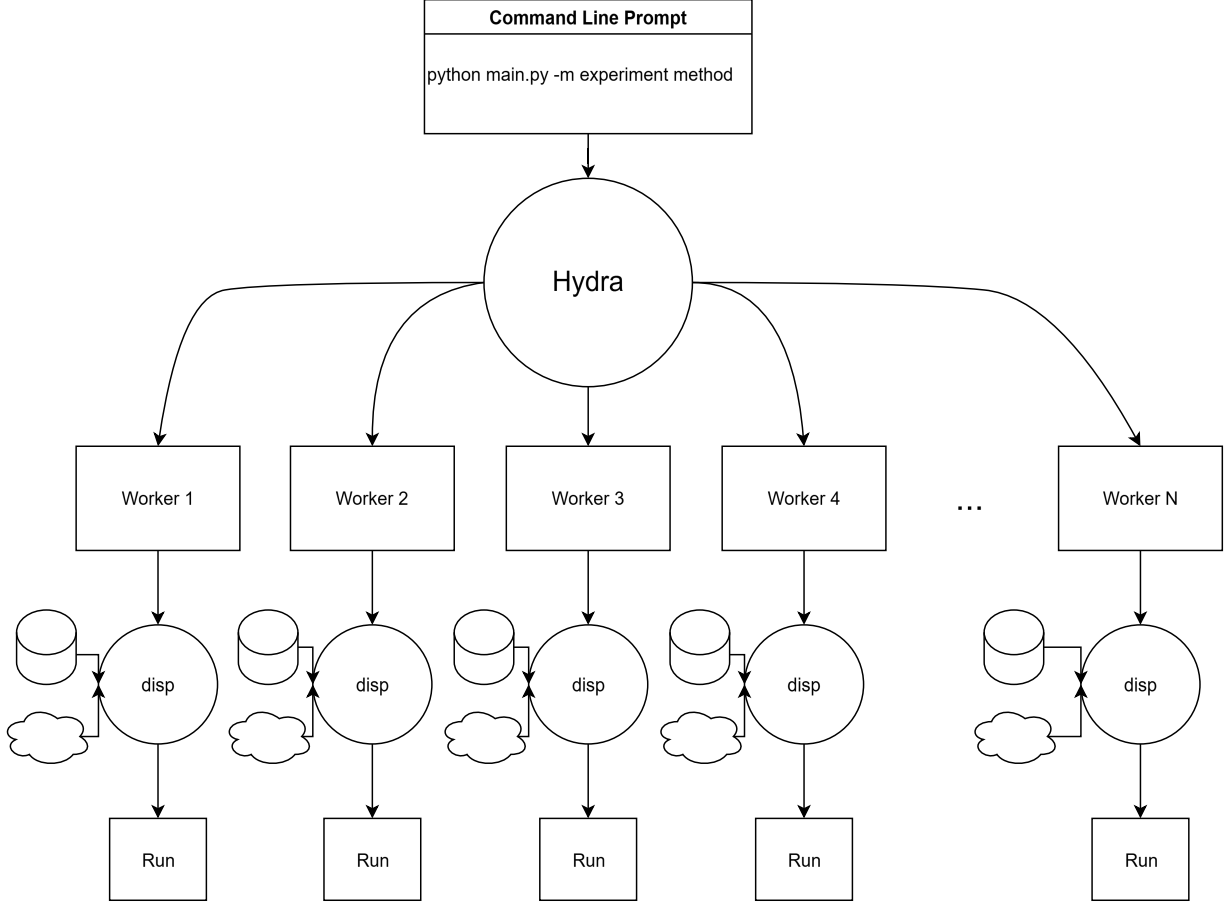


Figure 1: Illustration of the baseline development process using **Hydra**. The command-line prompt `python main.py -m experiment method` initiates a multirun process where **Hydra** distributes the workload to individual workers, denoted as W_1 to W_N . For each worker, a dispatcher is called to load data, execute the specified method, and store the results in a dedicated location within the file system. The **experiment** parameter defines the category of the task (e.g., origin training, machine unlearning), while the **method** parameter specifies the specific algorithm or technique to be applied.

2.1.2 Splits Design

With the framework in place, the choice for data preparation was to develop basic scripts independent of the framework which could be used to download, preprocess, and store well-structured data along with their splits. The latter was the most difficult part due to the necessity of having training, testing, and validation sets for all possible categories and combinations.

To keep it as simple as possible, a folder containing all the *.png* image files was created once the data was downloaded and prepared following the work of Xian et al.[30] (2019). The splits were then divided into two types:

- **Image-splits:** These contain the training, testing, and validation images for all existing labels in a dataset.
- **Class-splits:** These contain the various forget-sets and unseen-sets that are filtered out in each individual method.

This allows for the use of the full dataset as well as different portions of it, making sure that each of the subdivisions contains at least one image per class.

2.2 Datasets Selection and Preprocessing

Datasets used for the experiments were chosen among the ZSL Benchmark datasets as those comes with semantical information about each label that will be necessary in the next steps of the work. The choice was mainly due to the amount of details provided with those datasets, and their vast usage in the Zero-Shot Learning context. Secondly, because of the intent to adapt [5] work to both the tasks, so to take advantage of the already existing analysis and having a more analytical and deep approach about Machine Unlearning subtask which is still unexplored on those datasets and for this specific goal.

Additionally, we aimed to adapt the methodology of Christensen and his colleagues to the unlearning task. Utilizing the same datasets as their work allows us to directly leverage their existing analysis of ZSL, providing a robust foundation for our own experiments. More importantly, this approach enables a deeper, more analytical exploration of the Machine Unlearning (MU) problem within these specific datasets. This is a crucial contribution, as the vast majority of existing MU research is confined to standard "playground" datasets like CIFAR-10 and CIFAR-100, leaving the application of unlearning techniques to more semantically-rich datasets largely unexplored.

A preliminary analysis was conducted for each dataset to optimally organize the necessary splits. To ensure our experimental setup is as fair and consistent as possible, we first established a baseline by training a model on a subset of the total labels, which we refer to as the **seen data** ($D_s := \{(x_s, y_s)\} \subset D$), where D is the full dataset of image-label pairs (x, y) .

A crucial requirement for this setup is that the excluded set of labels, from now on referred to as the **unseen data** ($D_u := \{(x_u, y_u)\} \subset D$), was effectively never seen during the model’s initial pre-training. This design is fundamental to our approach, as it allows us to transfer the knowledge from the original model’s training to the new datasets without the need for a complete retraining from scratch, which would be computationally prohibitive and time-consuming. Furthermore, an essential requirement is that the seen and unseen data are mutually exclusive: $D_s \cap D_u = \emptyset$ as violating this condition would lead to inflated Zero-Shot Learning (ZSL) metrics, as some of the unseen labels would be inadvertently included in the training data, compromising the integrity of the experiment.

To effectively implement this, the proposed splits from the work of Xian et al. [30] were utilized. A key advantage of these splits is that the seen and unseen classes were deliberately selected to have no overlap with the ImageNet1K dataset, which served as the original source for our pre-trained classifier. Critically, their methodology also includes additional subsets of data taken from the seen classes D_s , originally used for hyperparameter tuning.

While these subsets were not directly used for their original purpose in this thesis, we repurposed them to serve as the forget-sets for our Machine Unlearning experiments.

This was a deliberate choice to ensure the integrity of the re-training from scratch baseline. By using these specific subsets, we can be confident that the data to be forgotten was indeed not included in the re-training from scratch baseline, which establishes a clean and valid benchmark for measuring the true efficacy of the unlearning process. To evaluate each method across multiple contexts, these subsets of labels were merged into a single pool F of possible choices for the **forget set** $D_f := \{(x_f, y_f) \text{ s.t. } y_f \in F\}$.

To ensure a comprehensive analysis, three distinct scenarios were randomly sampled from this pool, with the number of labels in each scenario being proportional to the total number of available labels in D_s :

- **Low-Forget Scenario:** This scenario involves a small number of labels to be forgotten. Its primary purpose is to test the fundamental unlearning capability of each method.
- **High-Forget Scenario:** This scenario uses a large number of labels to be forgotten. Its goal is to test the limits and scalability of each method under demanding conditions.
- **Average-Forget Scenario:** This scenario represents an intermediate number of labels to be forgotten. It is primarily used to analyze the method’s performance linearity, providing insight into its robustness and stability as the number of classes to be unlearned increases.

Following, a brief description of the utilized datasets and the preprocessing required to effectively use them.

2.2.1 Animals With Attributes

The AWA2 (Animals with Attributes 2) dataset comprises 37,322 images across 50 animal species, each species is annotated with 85 semantic attributes. Following the splits proposed in Xian et al. [30], 40 classes were designated as seen data D_s for the origin model’s transfer-learning, with the remaining 10 classes forming the unseen data D_u .

Preprocessing for this dataset was minimal, primarily involving resizing and normalization to match the input requirements of the pre-trained classifier, which was originally trained on the ImageNet1K dataset. Based on the sub-splits provided by [30] the following forget scenarios were established, defined by the number of classes to be unlearned:

- **Low-forget scenario:** $\#D_{f-low} = 5$
- **Average-forget scenario:** $\#D_{f-avg} = 8$
- **High-forget scenario:** $\#D_{f-high} = 12$

This dataset provides a relatively straightforward scenario for unlearning tasks, making it a suitable starting point for evaluating the baseline methods. Its simplicity aligns with the common settings often used in the broader Machine Unlearning literature, thereby providing a clear initial reference for our results.

2.2.2 Annotated Pascal-Yahoo

The aPascal & aYahoo (aPY) dataset, a combination of images from Pascal VOC and Yahoo, was also used in our experiments. It consists of 15,339 images spanning 32 classes, each with 64 semantic attributes. Following the splits commonly used in the literature, 20 of these classes are designated as seen data D_s and the remaining 12 as unseen data D_u .

Given the more diverse and hybrid nature of its classes, the aPY dataset provides a more challenging environment for evaluating the generalizability and robustness of the unlearning methods. This dataset spans a wide range of image content, from animal species to human-made objects, often within complex scenes. The images frequently contain multiple objects, leading to the necessity of an intrusive preprocessing to isolate the single components and effectively pair them with the respective label. This inherent complexity makes the classification task more difficult and, consequently, provides a more rigorous test of a method’s ability to effectively unlearn knowledge while retaining performance on challenging, multi-object detection images.

To effectively prepare the dataset for classification, the procedure described in [30] was followed. For each image, the object corresponding to the provided label was isolated by selecting its bounding box. The image was then cropped, resized, and normalized in a manner consistent with the AWA2 dataset, before being stored along with its corresponding label.

Finally, the scenarios have been extracted from the proposed sub-splits following the same logic of other datasets:

- **Low-forget scenario:** $\#D_{f-low} = 3$
- **Average-forget scenario:** $\#D_{f-avg} = 5$
- **High-forget scenario:** $\#D_{f-high} = 7$

While more complex in terms of data variety and visual context, this dataset contains a limited number of classes. Although it provides good insight into the robustness of methods across different data distributions, it does not offer a comprehensive analysis of how these methods behave with a significantly larger number of classes to be unlearned.

2.2.3 Caltech-UCSD Birds-200-2011

The CUB-200-2011 (Caltech-UCSD Birds) dataset was selected as a fine-grained classification test ground. This dataset contains 11,788 images across 200 bird species, each annotated with 312 binary attributes. Its fine-grained nature presents a distinct challenge compared to the broad categories of AWA2 and the complex scenes of aPY, requiring methods to distinguish between highly similar classes.

Due to its larger number of classes, CUB-200-2011 is particularly suited for evaluating the scalability of the methods. The standard splits proposed in the ZSL literature were followed, with 150 classes designated as seen data D_s and the remaining 50 as unseen data D_u , then the following forget scenarios were established using [30] proposed sub-splits:

- **Low-forget scenario:** $\#D_{f-low} = 10$
- **Average-forget scenario:** $\#D_{f-avg} = 20$
- **High-forget scenario:** $\#D_{f-high} = 30$

It is important to consider that while this dataset consists of animal images, a data distribution similar to ImageNet1K, it provides a limited number of per-class images. This scarcity makes the transfer-learning process more challenging compared to the previous datasets. Nevertheless, this dataset remains highly valuable as it provides significant insights into how the methods generalize and scale to a larger number of labels.

2.2.4 SUN-Attributes

The SUN-Attribute dataset was considered to evaluate the methods' performance on a large-scale scene recognition task. This dataset, which comprises 14,340 images across 717 scene categories, was deemed a potential candidate for assessing scalability and generalization on a completely different data distribution.

However, its image distribution was found to exhibit a significant domain shift from the object-centric categories of the ImageNet1K dataset used for pre-training. This factor, combined with the very limited number of images available per class, made effective transfer learning infeasible. Consequently, this dataset was excluded from the final analysis, and no further sub-splits were generated. The results of this initial analysis can be found at section 2.4.

This result, while negative, confirms the key limitations regarding the adaptability of modern AI models, as discussed in Chapter 1. It specifically underscores the high data requirements for successful domain adaptation, which can be a significant barrier in resource-constrained research environments. This finding, therefore, highlights the potential of exploring more data-efficient and data-agnostic approaches for these tasks.

2.3 ResNet Choice as Base Classifier

The Residual Network (ResNet) architecture was selected as the base classifier for its proven effectiveness in diverse computer vision tasks, particularly in image classification. The foundational paper, "Deep Residual Learning for Image Recognition" by He et al. [14], introduced the innovative concept of residual connections (also known as skip connections). This design addresses the degradation problem: a phenomenon where the accuracy of very deep networks saturates and then rapidly degrades, contrary to what might be expected as more layers are added.

The ResNet18 model was chosen specifically for its optimal balance between computational efficiency and performance. It consists of 18 layers, organized as follows (fig. 2):

- An initial convolutional layer followed by batch normalization and a ReLU activation function.
- Four subsequent "stages", each composed of two residual blocks. The first block in each of the last three stages includes a downsampling layer to reduce the spatial dimensions and an additional 1x1 convolution in the shortcut connection to align dimensions.
- A final average pooling layer and a fully connected (FC) layer for classification. The model outputs a 512-dimensional feature vector, which is then fed into a linear classifier that is completely replaced to match each dataset's D_s .

To accelerate the training process and enhance generalization, the ResNet18 model was initialized with weights pre-trained on the ImageNet-1K dataset [7]. ImageNet-1K is a massive dataset comprising over 1.2 million high-resolution images across 1,000 object classes. The use of these pre-trained weights allows the model to leverage a rich hierarchy of features learned from a large, diverse dataset, from low-level features (e.g., edges and textures) to more complex, high-level ones. This approach has been particularly effective given that the target datasets were all relatively small, as it helped preventing overfitting and significantly reduced the required training time.

For the specific task, the final fully connected classifier was replaced with a new, randomly initialized FC layer, appositely configured to match the number of labels in the respective target dataset's seen classes D_s . This fine-tuning strategy allows the model to adapt the pre-learned features for the specific classification task while retaining the powerful feature extraction capabilities of the ResNet backbone.

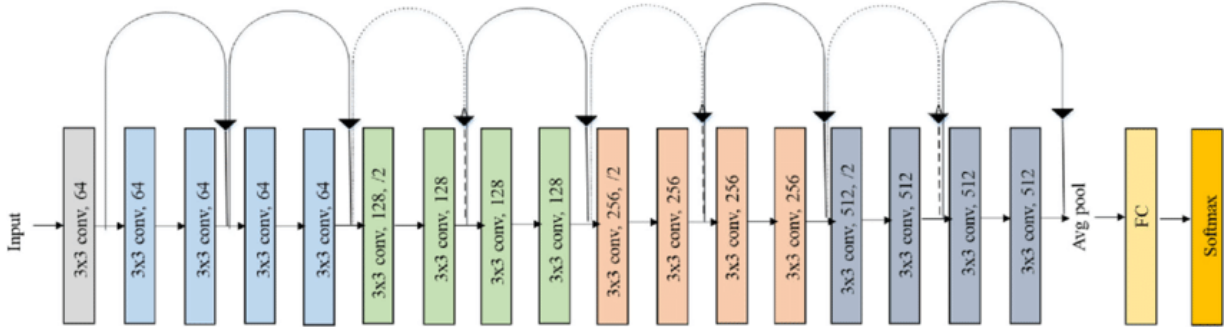


Figure 2: This figure depicts the complete ResNet18 network. The residual connections, shown as curved arrows, are the defining feature, allowing the model to bypass layers and mitigate the vanishing gradient problem. The architecture is organized into four distinct stages (Layers 1-4, indicated by different colors). The average pooling layer at the end converts the multi-dimensional feature maps into a fixed-size vector before the final classification.

2.4 Evaluation Metrics and Preliminary Results on Origins

Given the aforementioned setup, the finetuning was run for a higher amount of epochs (a maximum of 150 in each dataset) to ensure that the process, while not a proper retraining, could perform as similarly as possible to it. This approach allowed for a balanced tradeoff between performance and time efficiency. This time saving was also significantly enhanced by the similarity of some of the used datasets to the original pretraining data of the chosen classifier, which resulted in the finetuning process being much faster than a full retraining of a ResNet18 on any of the datasets. While this last characteristic is great for time-saving purposes, it makes the timing metric not indicative of how much each of the analyzed methods could gain compared to training from scratch on a completely new dataset.

2.4.1 Metrics Choice and Description

To test the origins' performance on each dataset, we chose to include all the core classification metrics:

- Accuracy, which is the ratio of correctly classified instances to the total number of instances. It was included to ensure the model's predictions were not random.
- Precision and Recall, which were used to support the accuracy metric, as the selected datasets were not balanced. This was necessary to ensure the finetuning process did not lead to overly biased models.
- F1-Score, which is the harmonic mean of Precision and Recall, was also included as a single metric that summarizes both.

2.4.2 Membership Inference Attack Metric

In addition to the classification metrics, an evaluation of the finally trained models on the vulnerability to Membership Inference Attack (MIA) was conducted.

Membership inference is a type of privacy attack where an adversary attempts to determine if a specific data record was included in a machine learning model’s training dataset. The attack leverages the phenomenon of overfitting, where a model becomes highly specialized to the data it was trained on. This specialization causes the model to behave differently on training data versus new, unseen data. Generally, a model that has overfit will be more confident in its predictions, producing lower losses on training data compared to data it has never encountered. A membership inference attack exploits this behavior by training a second, binary classifier able to distinguish between training and non-training data.

While an attacker cannot directly know which data records were used to train the target model, a common strategy for establishing a ground truth to train the adversary’s classifier is to leverage shadow models, as described by Kurmanji et al. in [17]. These auxiliary classifiers have the same architecture as the primary system and are trained on distinct datasets that mirror the data distribution of the original. By analyzing the behavior of these surrogates on both their training data and unseen data, the attack classifier learns to recognize the subtle differences that signal a data point’s inclusion in a training set. This approach allows the attacker to infer membership on the target without requiring direct access to its specific training data.

For the specific academic objectives of this thesis, the use of shadow models to simulate an adversary is not required, as a direct access to the ground truth regarding which data points belong to the training set is already known at training time. Instead, the application of the Membership Inference Attack (MIA) metric is intended to analyze each method’s ability to mitigate this vulnerability. Consequently, the attacker model for the purpose of the MIA metric is a simple logistic regression classifier, which is trained directly on the origin’s training and test sets. This approach allows for a direct and controlled assessment of the model’s vulnerability, comparing its state before and after the unlearning method is applied. This direct comparison provides a clear measure of the method’s effectiveness in removing the vulnerability, as described in [17].

In addition, a second attacker model is trained using a similar setup. This model is designed to recognize feature distributions rather than loss values. The distinguishing characteristic used to separate the training data from the unseen data are the features produced immediately before being passed to the classification head. This approach will be valuable for a later analysis to differentiate between methods that function primarily as output suppression and those that perform proper unlearning. Throughout this thesis, the classical MIA (Membership Inference Attack) metric will be referred to as loss-MIA (lMIA), while this new metric will be called feature-MIA (fMIA).

2.4.3 Origins Training Results

Given the training setup, which used an early stopping condition with a patience of 10 epochs based on the cross-entropy loss, the table below summarizes the results obtained for the different origins. This training served as a foundational step, providing classification models that will be used as starting points for further analysis, as it is a base requirement for any subsequent experimentation.

As anticipated, the pre-trained ResNet18 model demonstrates strong generalization capabilities on the AwA2 and aPY datasets. This is likely because these datasets, which feature classes like animals and people, are semantically similar to the original ImageNet-1K dataset the model was pre-trained on. While the time taken for this process isn't a perfect metric for a full retraining, it serves as a useful benchmark for the time-efficiency of the current adaptation methodology. Furthermore, the CUB and SUN datasets proved to be the most challenging for transferring the original learned weights. While the results for the CUB dataset are fairly acceptable, the performance on the SUN dataset is insufficient for further analysis. Still, these results highlight the limitations of the classic adaptation approach and the exponential increase in computational time required for more complex datasets.

Dataset	Accuracy	Precision	Recall	F1	lMIA	fMIA	time
AwA2	92.6	90.9	89.5	90.2	50.4	61.2	46m 00s
aPY	89.9	91.0	90.2	90.6	55.5	68.3	12m 02s
CUB	72.4	74.2	72.5	73.4	72.3	83.2	35m 12s
SUN	33.7	36.9	33.8	35.2	-	-	1h 04m 20s

Table 1: Finetuning Results of a Pretrained ResNet18 on ZSL Benchmark Datasets. The table presents the model's performance on ZSL datasets, showing standard metrics such as Accuracy, Precision, Recall, and F1-Score. In addition, it includes two metrics for evaluating membership inference risk: Loss-MIA, which measures a logistic regressor's ability to distinguish between train and test samples based on the loss values produced by the ResNet18 in its final state of finetuning, and Features-MIA, which measures the same ability but using the features produced by the same model in the last layer before the fully connected classifier.

A crucial consideration regarding these results is the number of items iterated through. Each dataset was partitioned into training, validation, and testing sets with a respective ratio of 65%, 20%, and 15% of the total dataset size. This resulted in the following training sample counts and time-to-completion metrics:

- 24,310 training samples were iterated for 17 epochs in 46 minutes for AwA2, obtaining the optimal result after 7 epochs.
- 10,001 training samples, iterated for 30 epochs in 12 minutes and 2 seconds for aPY, obtaining the optimal result after 20 epochs.

- 7,849 training samples for CUB, iterated for 68 epochs (optimal metrics after 58 epochs) in 35 minutes and 12 seconds, demonstrating the significantly longer time required for effective learning through the classic adaptation approach.
- 10,038 training samples for SUN, iterated for 81 epochs in 1 hour, 4 minutes, and 2 seconds, with training being stuck after 71 epochs. This result highlights the increased difficulty for the classic approach to adapt when the data distribution is substantially different and the number of classes to be learned is higher.

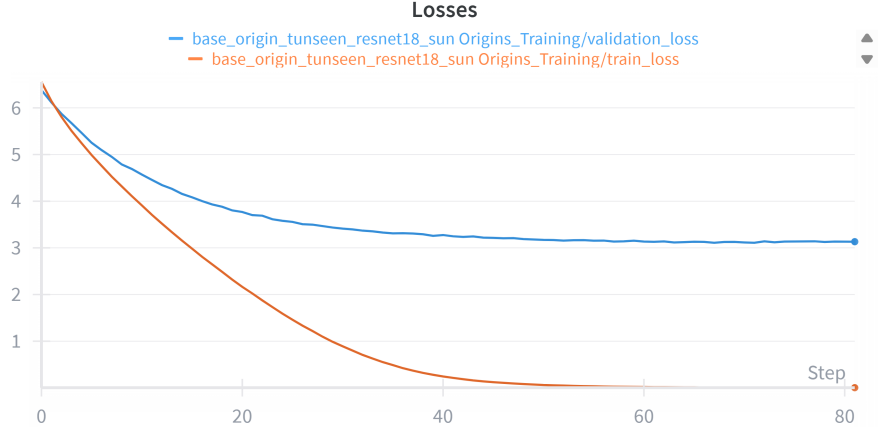
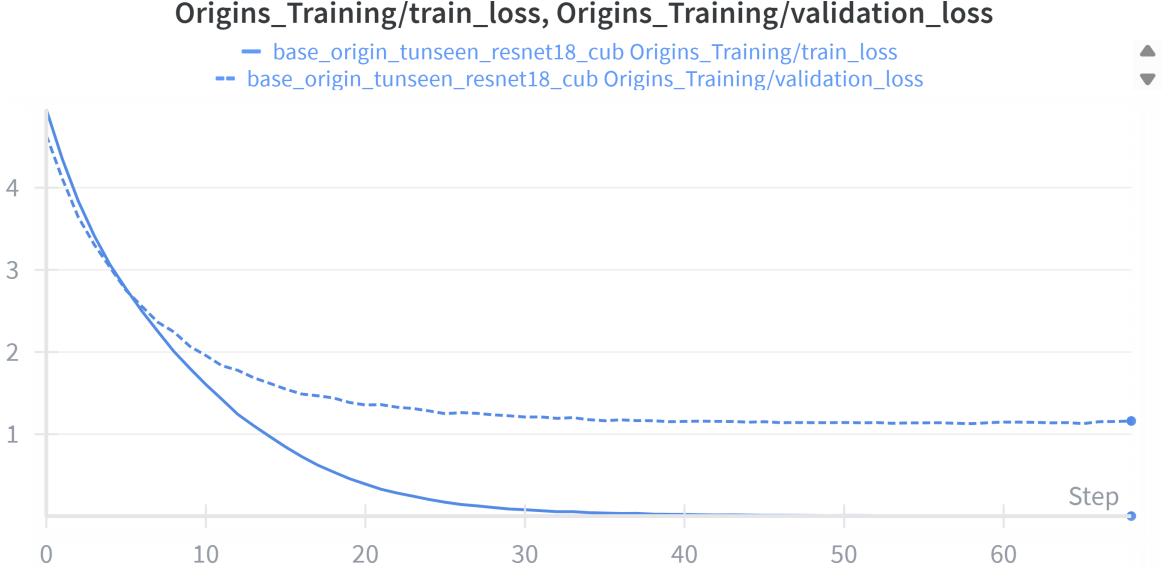
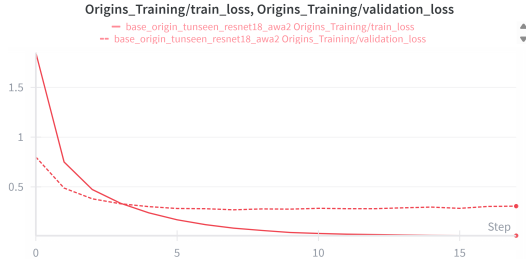


Figure 3: Training and validation loss curves illustrating poor generalization and the inability to effectively transfer learning from pre-trained weights to the SUN dataset.

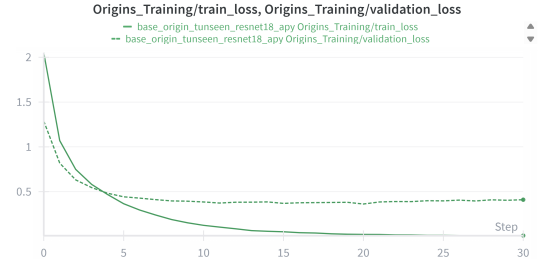
Regarding the Membership Inference Attack (MIA) metrics, all datasets exhibit a slight vulnerability in their feature behavior. However, only the CUB dataset demonstrates evident vulnerability to both types of attacks. This is primarily attributed to the difficulty of transferring learning to a significantly different data distribution, which necessitated a greater number of epochs. The increased training time led to poorer generalization and, consequently, more overfitting, as illustrated in fig. 4.



(a) CUB



(b) AwA2



(c) aPY

Figure 4: Loss curves for training and validation on the CUB, AwA2, and aPY datasets. The visual representation highlights a strict correlation: as the gap between the training and validation loss widens, the model’s vulnerability to Membership Inference Attacks (MIA) increases.

3 Literature Analysis for Model Adaptability Tasks

3.1 Introduction to Baselines

This section provides a formal introduction to the downstream tasks of Zero-Shot Learning (ZSL) and Machine Unlearning (MU). Building upon the foundational concepts introduced in earlier chapters, the established literature and methods for both tasks will be analyzed and compared to the relative weight-learning approach considered for the task. The existing work of Christensen et al. for ZSL is primarily focused on, with a detailed analysis of their methodology and results being presented, and then, it will be demonstrated how a similar approach can be adapted and applied to the Machine Unlearning task. By using these two methods and the relative weaknesses and strengths, the leading hypothesis of unifying the approaches into a single weight-learning framework will then be explored in the next chapter.

Zero-Shot Learning addresses the challenge of classifying instances from classes that were not present during the origin’s training. Methods that allow for zero-shot learning either apply modifications directly to the origin model itself, enabling it to recognize labels in D_u , or mathematical computations that are able to transfer origin’s knowledge to this new set of labels.

This task can be intended as the ability to recognize an entirely new set of labels only (Classical Zero-Shot Learning), or the ability to recognize new, unseen labels as well as old, seen ones (Generalized Zero-Shot Learning).

After the advent of Visual Transformers [9], two main categories of methods for ZSL can be recognized: ViT-based and non-ViT-based.

Conversely, the Machine Unlearning task is the ability to remove knowledge about samples from a trained network, in a way that the model to which the method was applied: the **unlearned model**, will behave as similarly as possible to as if it never saw those samples before. While in first instance it can be considered a direct mirror of the ZSL task, it has to be said that MU can have multiple interpretations:

- **Class Unlearning**, that is the one this thesis will focus on, which tries to remove knowledge about an entire set of labels, defined as **forget-set** D_f . After applying the method to the origin network, the latter should behave as if it never saw those labels during its training.
- **Sample Unlearning**, which consists in removing specific samples’ influence from the training of the origin, for example, bad data samples that degrade the model’s performance.
- **Concept/Feature Unlearning**, where an entire set of features or concepts (e.g. nudity, violence, etc...) can be removed from a model.

3.2 A Unified Perspective: Adaptability in Weight-Space

When training a model, its weights are fitted to a set of data, carefully adjusting the values to minimize an objective function summarizing the final task. In the case of classification, by using the cross-entropy loss defined as:

$$l_n = -w_{y_n} \log \frac{\exp(x_{n,y_n})}{\sum_{c=1}^C \exp(x_{n,c})} \quad (1)$$

$$l(x, y) = \sum_{n=1}^N \frac{1}{\sum_{n=1}^N w_{y_n}} l_n \quad (2)$$

Figure 5: *Equation of Cross Entropy Loss implemented in PyTorch with default 'mean' reduction; w_{y_n} is an optional weight assignable to single classes c_i , C is the total number of classes, and N is the total number of elements in the batch to be computed.*

While equation (2) simply calculates the mean between the l_n values, which can be optionally weighted by the assigned w_{y_n} , equation (1) drives the gradients that will modify the single weights of the model. This correction can be translated into a move across the model's weight-space.

In each training step, a batch of images is processed, then the model produces a probability distribution for each image, which is compared to a one-hot encoded vector representing the true class. When the cross-entropy loss is minimized, gradients are computed to guide the model along the steepest descent direction in the loss landscape. This movement in the weight space causes the model's output probability distribution to align increasingly with the one-hot encoded true class distribution.

As this learning process gets distributed across the network's many neurons, the interconnected weights and biases collectively learn to map inputs to outputs. During training, the iterative updates of these connections cause the model to become highly specialized to the training data. With a theoretically infinite number of training steps, the network would essentially memorize the training set, including any noise or random fluctuations within the data. This state of a perfect fit for the training data is known as overfitting, and it severely impairs the model's ability to generalize to new, unseen examples.

When a model trained on one dataset needs to be adapted to a new one, the process involves repositioning it in the weight space to a state that would otherwise be reached through training from scratch. This process can be understood as simultaneously unlearning the old information while learning the new data, and is known in MU literature as **golden approach** as it leads to perfect unlearning but is computationally expensive.

Such states, as well as overfitted states, already exist in the weight space of the model and can hypothetically be reached even without having any knowledge about new datasets, for example, by using a mathematical framework to find the geodesics connecting them, as proposed by Raghvan and his collaborators in [25]

3.3 Hypernetworks

While performing complete random walks across the weight space could be an impossible task, especially with the increase in parameter size, this perspective opens up a unified way of approaching models’ adaptability tasks, such as new data learning and unlearning.

A generative approach, where a model parameters are used as a summary of the training set to generate new ones that could solve a specific task, would decouple the adaptability tasks from any requirement of new data. These types of networks are known in the literature as hypernetworks and have already been explored in both the separated tasks of ZSL (DAEGNN [12]), and MU (DiHyFo [26]).

3.3.1 DAEGNN [12]

This work addresses a fundamental challenge in modern deep learning: few-shot object recognition. Unlike zero-shot learning, which has no examples of the new class, few-shot learning assumes the availability of a few examples for a model to learn new, previously unseen labels.

The authors propose a novel meta-learning framework designed to overcome this data scarcity by learning how to generate discriminative classification weights. The central innovation is a *meta-model* that ingeniously combines two core components: a **Denoising Autoencoder (DAE)** and a **Graph Neural Network (GNN)**.

In the proposed architecture both the components have distinct roles: the DAE component serves a crucial role in regularization, taking an initial set of classification weights, which have been corrupted with Gaussian noise, and learns to reconstruct more accurate, ”target-discriminative” weights. This process forces the model to learn a robust mapping from low-quality, noisy inputs to high-quality outputs.

The GNN, on the other hand, has the role to generate weights for the unseen data classes by organizing the known labels’ weights in a graph. The GNN exploits the graph’s structure, which represents relationships and similarities between classes, to deduce and generate the weights for new labels based on their proximity in the graph neighborhood.

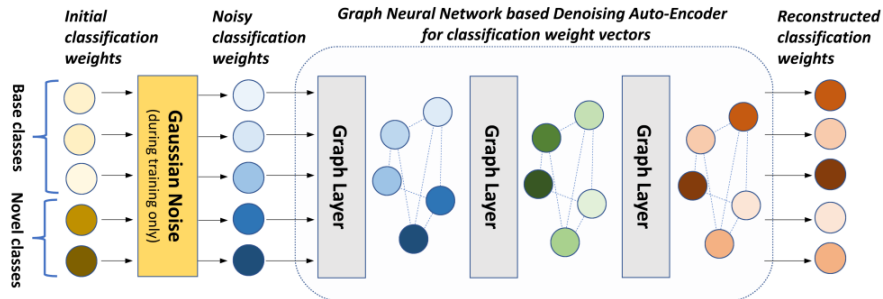


Figure 6: Conceptual representation of DAEGNN, taken from the ”Generating Classification Weights with GNN Denoising Autoencoders for Few-Shot Learning” paper [12]

3.3.2 DiHyFo [26]

In this work the authors introduce a novel framework for machine unlearning (MU) named HyperForget. The central innovation of this work is the development of a single, pre-trained generative model, a Diffusion HyperForget Network (DiHyFo), which possesses the unique ability to produce unlearned models for a variety of different forgetting tasks from a single training run. This approach offers a significant departure from the traditional "gold standard" of unlearning, which necessitates computationally and resource-intensive retraining of the model from scratch for each individual data removal request as it can be applied multiple times using the same trained version.

The DiHyFo architecture is trained on a dataset of checkpoints from a pre-trained target model. During this training, the hypernetwork is fed with data representing current and future parameters and their associated class losses. It learns to generate the future parameters based on the time-step, the current losses, and the desired loss differences. This training process effectively teaches the DiHyFo to navigate the parameter space and achieve specific performance goals. Particularly, it can reduce the loss on the retain set, while simultaneously increase the loss on the forget set, that could be considered a valid Zero-Shot Unlearning method.

To condition the generative process in a way that it is possible to selectively forget classes, a Diffusion Transformer (DiT [23]) is used to generate the future parameters. During the forwarding (diffusion) process, noise is progressively added to the target model parameters so that the DiT can learn to reverse this process, predicting the noise added at each step, and thus effectively generating the desired future parameters. This conditioning vector added in the DiT, effectively enable the model to apply specific loss changes by acting on the origin's weight-space.

Although very powerful, a significant drawback of this method is the prerequisite of multiple checkpoints from the same target network. The absence of these checkpoints would necessitate a full retraining procedure to acquire the information essential for this framework to work.

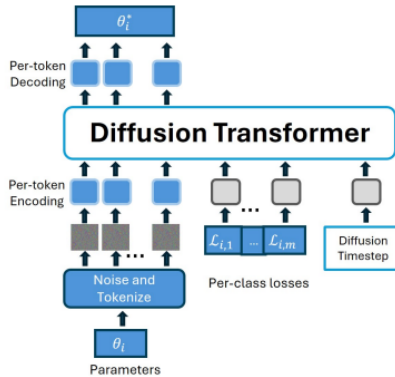


Figure 7: DiHyFo base architecture taken from Rangel et al.[26] paper. Diffusion Transformer, follows the work described in [23]

3.4 Zero-Shot Learning (ZSL)

Zero-Shot Learning (ZSL) is a machine learning paradigm that enables a model to classify or make predictions for unseen classes that it has not encountered during its training phase. The "zero-shot" in its name means that the model can accomplish this task without seeing any labeled examples of the new classes. There are easier and often more realistic scenarios where the model is taught to classify new classes via one or a few examples; those methods are referred to as "one-shot" or "few-shot", depending on the amount of available samples.

The way it is achieved is by leveraging auxiliary information, such as textual descriptions, attributes, or a semantic embedding space, to transfer knowledge from seen classes (classes with labeled training data) to unseen ones.

3.4.1 Formal Definition

Given a set of seen classes Y_{seen} and a set of unseen classes Y_{unseen} , such that $Y_{seen} \cap Y_{unseen} = \emptyset$, a ZSL model aims to learn a function $f : X \rightarrow Y_{unseen}$ that can predict the correct class label for a data instance $x \in X$ even though no training examples from Y_{unseen} are provided.

By exploiting, in fact, side information that could be some semantically relevant description of the classes A (*Zebra: "a horse-like animal with black and white stripes."*), it has been shown that it is possible to learn a mapping function $\phi : X \rightarrow A$ and a classification function $\varphi : A \rightarrow Y_{unseen}$. With the combined usage of those two functions it is possible to project a model's prediction in the semantic space and use this information to predict the unseen label, or alternatively, to project the image data $x \in X$ s.t. : $y_x \in Y_{unseen}$ into the feature space by using the semantic relationship relative to the features.

As already mentioned in the preamble of this chapter, there are two primary setups for the ZSL task:

- **Conventional Zero-Shot Learning (CZSL):** The model is evaluated on a test set that only contains samples from unseen classes. This is a simplified scenario used for research but is often not reflective of real-world applications.
- **Generalized Zero-Shot Learning (GZSL):** This is a more realistic and challenging task. The test set contains samples from both seen and unseen classes, and the model must correctly classify an instance into its corresponding class, regardless of whether it's from a seen or unseen category. This setting is prone to a bias where the model may favor predicting seen classes due to its training data.

3.4.2 Evaluation Metrics

To properly assess the quality of Zero-Shot Learning (ZSL) methods, it is essential to evaluate their performance under two distinct settings: Conventional Zero-Shot Learning (CZSL) and Generalized Zero-Shot Learning (GZSL).

For CZSL, the model’s performance is measured by its classification accuracy on a test set composed exclusively of unseen classes Y_{unseen} . The metric used is top-1 accuracy on this unseen set, which measures the model’s ability to correctly classify a new sample $x \in D_u$ into one of the previously unseen categories $y_x \in Y_{unseen}$.

In contrast, Generalized Zero-Shot Learning (GZSL) requires a more comprehensive evaluation. It is crucial to separately measure the model’s accuracy on both the seen classes Y_{seen} and the unseen classes Y_{unseen} in order to recognize which methods are mostly basing their accuracy metric on the mere prediction of known samples. In this perspective, the separate measures allow to control if a specific method is too much biased toward seen classes by looking at the gap between the seen accuracy (S) and the unseen accuracy (U)

The overall performance for GZSL is thereby typically reported using the harmonic mean of these two accuracies (H). This is a critical metric because it provides a balanced assessment, penalizing models that perform exceptionally well on one set while performing poorly on the other. For example, a model that would perfectly be able to recognize the seen set, but completely misses the unseen set producing $U = 0$, would result in an harmonic mean conventionally set to 0 as well.

These three metrics (seen accuracy S , unseen accuracy U , and their harmonic mean H) are the standard for benchmarking in the ZSL literature. They provide a thorough evaluation of a method’s quality by assessing both its ability to recognize novel, unseen categories and its capacity to retain performance on previously seen classes. This approach also offers valuable insights into the training bias, revealing the model’s natural tendency to favor predictions for seen classes over unseen classes.

3.5 Literature Methods

The typical approach for ZSL is to use training images x to learn the ϕ mapping function between the feature extracted by the trained model’s backbone and the semantic description of the label associated with them $y_x \in Y_{seen}$.

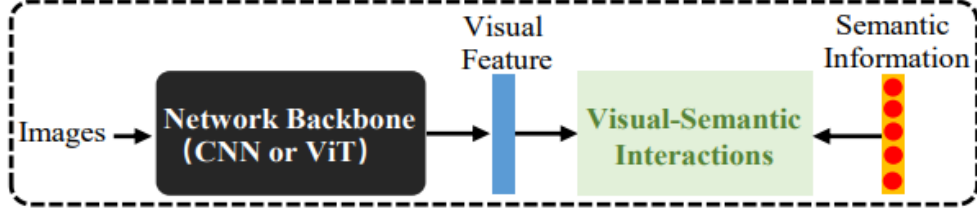


Figure 8: Chart² showing the classical approach to learn the ϕ mapping function between the sample x and the semantic associated to its label $y_x \in Y_{seen}$

With the advent of transformers, applications to computer vision were also studied, introducing the Visual Transformer (ViT [9]) that led to many new applications also in the field of ZSL, and in particular ZSLViT [4], which is one of the most recent and well-performing methods in the literature, as proved in the table provided by them in the experimentation chapter:

	Methods	Venue	CUB				SUN				AWA2			
			CZSL	GZSL			CZSL	GZSL			CZSL	GZSL		
			acc	U	S	H	acc	U	S	H	acc	U	S	H
#	AREN	CVPR’19	71.8	38.9	78.7	52.1	60.6	19.0	38.8	25.5	67.9	15.6	92.9	26.7
	f-VAEGAN	CVPR’19	61.0	48.4	60.1	53.6	64.7	45.1	38.0	41.3	71.1	57.6	70.6	63.5
	TF-VAEGAN	ECCV’20	64.9	52.8	64.7	58.1	66.0	45.6	40.7	43.0	72.2	59.8	75.1	66.6
	LsrGAN	ECCV’20	–	48.1	59.1	53.0	–	44.8	37.7	40.9	–	54.6	74.6	63.0
	DAZLE	CVPR’20	66.0	56.7	59.6	58.1	59.4	52.3	24.3	33.2	67.9	60.3	75.7	67.1
	APN	NeurIPS’20	72.0	65.3	69.3	67.2	61.6	41.9	34.0	37.6	68.4	57.1	72.4	63.9
	Composer	NeurIPS’20	69.4	56.4	63.8	59.9	62.6	55.1	22.0	31.4	71.5	62.1	77.3	68.8
	FREE	ICCV’21	–	55.7	59.9	57.7	–	47.4	37.2	41.7	–	60.4	75.4	67.1
	GCM-CF	CVPR’21	–	61.0	59.7	60.3	–	47.9	37.8	42.2	–	60.4	75.1	67.0
	HSVA	NeurIPS’21	62.8	52.7	58.3	55.3	63.8	48.6	39.0	43.3	–	59.3	76.6	66.8
	MSDN	CVPR’22	76.1	68.7	67.5	68.1	65.8	52.2	34.2	41.3	70.1	62.0	74.5	67.7
	GEM-ZSL	CVPR’22	77.8	64.8	77.1	70.4	62.8	38.1	35.7	36.9	67.3	64.8	77.5	70.6
	SE-GZSL	AAAI’22	–	53.1	60.3	56.4	–	45.8	40.7	43.1	–	59.9	80.7	68.8
	TransZero	AAAI’22	76.8	69.3	68.3	68.8	65.6	52.6	33.4	40.8	70.1	61.3	82.3	70.2
	VS-Boost	IJCAI’23	–	68.0	68.7	68.4	–	49.2	37.4	42.5	–	–	–	–
	ICIS	ICCV’23	60.6	45.8	73.7	56.5	51.8	45.2	25.6	32.7	64.6	35.6	93.3	51.6
b	CLIP	ICML’21	–	55.2	54.8	55.0	–	–	–	–	–	–	–	–
	CoOp	IJCV’22	–	49.2	63.8	55.6	–	–	–	–	–	–	–	–
	I2DFormer-Wiki	NeurIPS’22	45.4	35.3	57.6	43.8	–	–	–	–	76.4	66.8	76.8	71.5
	CoOp+SHIP ¹	ICCV’23	–	55.3	58.9	57.1	–	–	–	–	–	–	–	–
	I2MVFormer-Wiki	CVPR’23	42.1	32.4	63.1	42.8	–	–	–	–	73.6	66.6	82.9	73.8
	DUET	AAAI’23	72.3	62.9	72.8	67.5	64.4	45.7	45.8	45.8	69.9	63.7	84.7	72.7
	ZSLViT	–	78.9	69.4	78.2	73.6	68.3	45.9	48.4	47.3	70.7	66.1	84.6	74.2

Figure 9: Table² of results provided by [4], separating methods in # which are the non-ViT based methods, and b that are the ViT-based methods

²taken from the work conducted by Chen et al.[4]

3.5.1 Transformers and Attention Heads

While not strictly correlated with this thesis work, introducing transformers and their operational modality is a crucial step for understanding their application in computer vision tasks and, specifically, the weight-learning transformer presented in Chapter 4 based the framework proposed in [27].

As reported in the paper: **Attention is All You Need** [29], the attention system is what drives modern Large Language Models (LLMs) by exploiting self-attention to learn inter-elements correlation. The first step for a transformer to function is to extract a sequence of tokens from the data through the tokenization process.

Tokenization is a fundamental process in natural language processing (NLP) that breaks down a piece of text into smaller, meaningful units called tokens. The resulting tokens can be words, subwords, or extra artificial tokens like the [CLS] or the [SEP] ones used for side tasks like indicating respectively the summarization of the current sequence, and the end of it. This process is mainly split in 4 sub-steps:

1. **Text Preprocessing:** Raw text is often cleaned before tokenization. This involves normalization, which includes converting the text to lowercase and handling special characters, and the handling of punctuation and stopwords (common connective words that do not provide significant information about the context).
2. **Splitting/Tokenization:** The preprocessed text is split into tokens based on the chosen tokenization method. By modifying the tokenization method it is possible to modify the vocabulary and hence the IDs associated with the tokens.
3. **Vocabulary Creation:** A vocabulary is built from all the unique tokens found in the text corpus. Each token is then assigned a unique integer ID.
4. **Token to ID Conversion:** Finally, the tokens are converted into their corresponding integer IDs. These numerical representations are what machine learning models, like large language models (LLMs), can process and understand.

Before the advent of transformers, word embeddings were primarily static. This meant that a single, fixed vector was assigned to each word, regardless of the context in which it appeared. This approach, exemplified by models like Word2Vec [19] and GloVe [3], was purely statistical and, as a result, failed to capture the different meanings and nuances a word can have in various contexts.

Recurrent Neural Networks (RNNs) were the first method developed to address this limitation. They process sequences of data by maintaining an internal state, which theoretically allows them to capture dependencies between words. However, a major drawback of early RNN architectures was their inability to handle long-range dependencies effectively, meaning they struggled to remember information from the beginning of a long text.

The main weakness of Recurrent Neural Networks (RNNs) in the context of Large Language Models (LLMs) is their difficulty in capturing long-range dependencies: while an RNN's sequential nature allows it to correlate a word with its immediate predecessors or successors, it struggles to maintain information from much earlier words in the sequence.

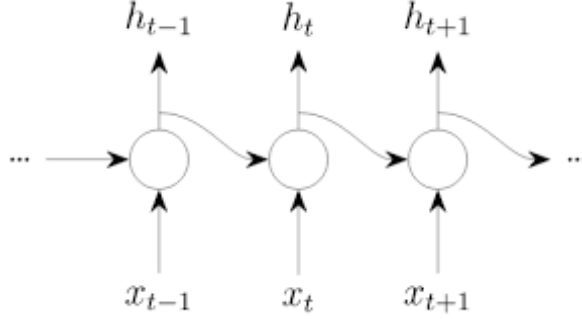


Figure 10: Recurrent Neural Networks computation of a sequence: weights are learned and shared among all the states (represented with a circle in the image). The output at time step t , denoted as h_t , is generated from the current input x_t and the previous state's output, h_{t-1} . This new output, h_t , then serves as the input for the next time step, $t + 1$, to produce h_{t+1} and so forth.

This is a significant limitation for natural language, where the meaning of a word can be influenced by other words that are located much farther away in the text.

The way transformers fix those issues is by adding a positional embedding to the token representing a word in a sequence, and computing the attention between all the words in the sequence (self-attention) and between different sequences (cross-attention). In those attention systems, the enhanced embedding is separated into three different representations: Key, Query, and Value.

Those three representations are conceptually related to the information retrieval task, where the key is the identifier or index of a piece of information, the query represents the input used to find relevant information, and the value is the information itself.

For this reason, these elements are combined using the scaled dot-product attention operation, which works as follows:

1. A dot product is computed between the query Q and all keys K to determine their relevance, carefully masking with a score of $-\infty$ the future tokens with respect to the actual key in the sequence to maintain causality of information.
2. The resulting scores are scaled by the square root of the dimensionality of the keys, d_k , to prevent the dot products from becoming too large.
3. A softmax function is applied to these scaled scores, converting them into a set of weights that represent the probability of relevance for each key-value pair.
4. Finally, these weights are used to compute a weighted sum of the values (V), producing the final output.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (3)$$

The purpose of the $\frac{1}{\sqrt{d_k}}$ normalization factor is to prevent the arguments of the softmax function from becoming excessively large. If the dot products were very large, the softmax

function would produce a probability distribution where one probability is very close to 1 and all others are close to 0. This could lead to vanishing gradients during training and could effectively reduce the attention mechanism to selecting only a single item, falling back to the RNN problem.

To better capture the relationships between different words, the self-attention mechanism is extended with a technique called multi-headed attention. This method allows the model to attend to different parts of the input sequence simultaneously and from multiple perspectives.

This approach is beneficial because each attention head can learn to focus on different types of relationships. For example, one head might learn to capture syntactic relationships (e.g., subject-verb agreement), while another might focus on semantic relationships (e.g., dependencies between related nouns). By combining these diverse perspectives, the model's overall ability to understand the context and capture complex dependencies is significantly enhanced. MLP blocks are then used to further refine the produced embeddings.

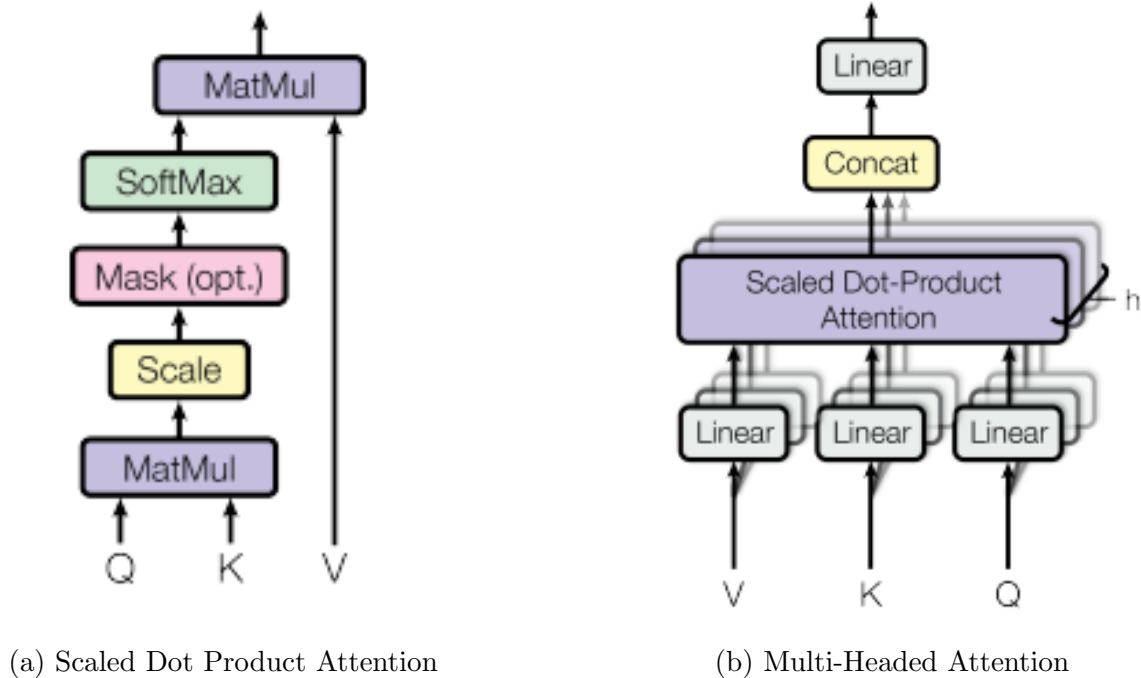


Figure 11: Components of the Transformer architecture.

While it is enough for an encoder-only architecture to only have a self-attention system, Transformers also allow for encoder-decoder architectures where decoders use Keys and Values generated from the encoder to find the best fitting generated Queries. This approach is called cross-attention and exploits the same scaled-dot-product-attention mechanism, combining the decoder's Q matrix with the encoder's K and V matrices.

3.5.2 Adaptation to Computer Vision: Visual Transformer ViT [9]

In classical transformer architectures, processing a sequence of tokens in textual data is straightforward; however, extending this paradigm to visual data is considerably more challenging. Dosovitskiy et al. [9] proposed an effective strategy to adapt transformers to Computer Vision tasks.

Their method introduces a preprocessing stage in which an image is partitioned into fixed-size patches (16×16 pixels in their implementation). Each patch is then flattened and projected into a linear embedding (noting that images are inherently represented numerically, with pixel values typically in the range $[0, 1]$ or $[0, 255]$). These embeddings serve as the input tokens for the transformer, after which the network proceeds with the standard computational pipeline described in section 3.5.1.

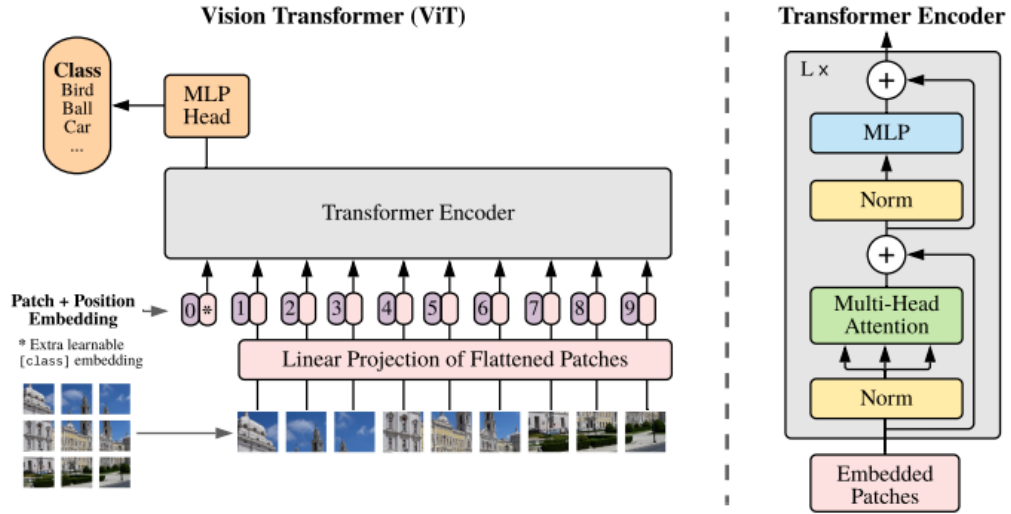


Figure 12: Adaptation of the transformer architecture to Computer Vision, inspired by Vaswani et al. [29], originally published in 2017.

Although the Vision Transformer (ViT) can be directly applied to the Zero-Shot Learning (ZSL) task by utilizing the output of the MLP head for classification, as demonstrated in CLIP [24] for online ZSL classification, ZSLViT [4] proposes a reformulation of the transformer. This approach leverages both the tokenized image representation and the semantic description associated with each label, thereby enabling the extraction of semantically informed visual features.

This formulation enables the transformer to extract features that are semantically aligned with the label space, rather than relying solely on backbone representations derived from a conventional supervised classifier.

A key challenge in integrating such methods is the *Image-Free* constraint, which defines one of the main scopes of this thesis. The objective of the present work is to mitigate the reliance on large-scale datasets by investigating how to exploit the representational knowledge already encoded within the network itself.

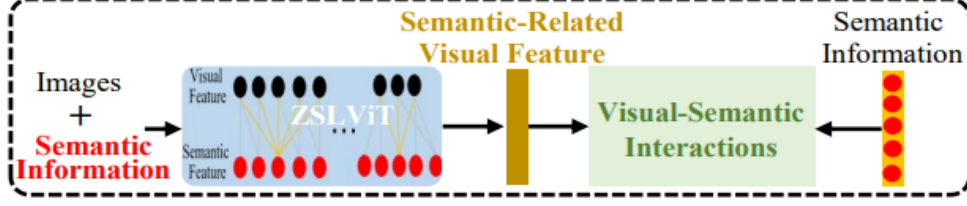


Figure 13: Schematic² of the ZSLViT approach, which learns the mapping function ϕ between a sample x and the semantics associated with its label $y_x \in Y_{seen}$.

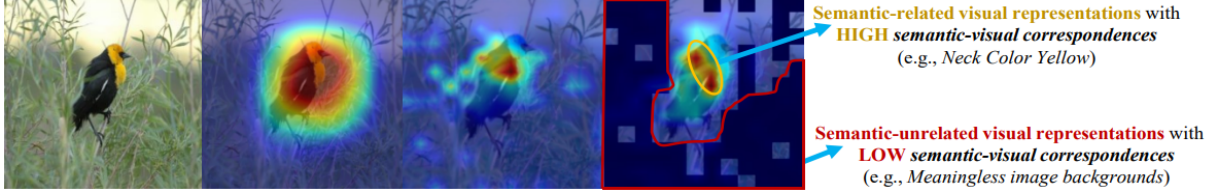


Figure 14: Visualization² of feature heatmaps extracted by a CNN, ViT, and ZSLViT (from left to right).

3.6 Weight Learning in Image free ZSL

In their work, the authors of [5] introduce Image-Free ZSL, a novel family of zero-shot learning (ZSL) algorithms in which existing methods are adapted to operate without the need for any reference images, by leveraging the information encoded within a trained model’s weight-space. With this approach, it is possible to extract valuable insights about seen classes that can thereby be used to transfer the model’s knowledge to the unseen ones, via the semantic space A

In addition, the image-free scenario makes most of the traditional ViT-based methods completely unavailable, as these models are fundamentally designed to process visual inputs. Therefore, the absence of any image prevents these models from performing their primary function of extracting visual features and representations.

Contrarily, Image-free Classifier Injection with Semantics (ICIS) doesn’t require any preliminary image, as it learns to map semantic information directly to the set of weights in a trained model for then injecting the new weights into the classification head. Depending on how the weights are injected in the classification head, ICIS is able to generate a classifier able to recognize only the new labels (CZSL) or both the label sets (GZSL).

Although the authors of [5] were the first to introduce Image-Free Zero-Shot Learning (I-ZSL), their work demonstrates that many existing methods suffer substantial performance degradation when transferred to the image-free scenario. In fact, they took a variety of approaches into account, ranging from completely online methods like ConSE [21] to hypernetwork-based ones such as wDAEGNN [12] and adapted them to the new limitation of I-ZSL. In their comprehensive results table, it is shown that their proposed method, ICIS, significantly outperforms these adapted methods in the image-free scenario.

Image-free Zero-Shot Learning	Zero-Shot Accuracy			Generalised Zero-Shot Accuracy								
	CUB	AWA2	SUN	CUB			AWA2			SUN		
	Acc	Acc	Acc	u	s	H	u	s	H	u	s	H
	Acc	Acc	Acc	u	s	H	u	s	H	u	s	H
ConSE	41.9	44.0	44.4	0.5	88.0	0.9	3.0	96.1	5.7	0.1	47.9	0.1
COSTA	31.9	40.9	19.9	0.0	87.6	0.0	0.0	96.1	0.0	0.0	50.1	0.0
Sub. Reg.	37.6	37.5	48.3	0.0	87.6	0.0	0.0	96.1	0.0	0.0	50.1	0.0
wDAE	38.2	37.0	49.9	0.0	87.3	0.0	0.1	96.0	0.3	0.0	49.3	0.0
WAvg	2.0	20.1	1.4	1.9	52.3	3.7	5.5	92.4	10.4	0.0	50.1	0.0
SMO	45.1	55.4	42.7	39.2	52.3	44.8	31.8	92.4	47.3	42.5	1.6	3.1
ICIS	60.6	64.6	51.8	45.8	73.7	56.5	35.6	93.3	51.6	45.2	25.6	32.7

Figure 15: Table of results taken from [5], showing methods performances after being adapted to the image-free context

To better understand the mechanisms that enable knowledge transfer without model re-training, the subsequent sections will detail the adaptations of some of the existing methods. This analysis serves as a foundation for the subsequent introduction of the ICIS architecture and its conceptual framework.

3.6.1 ConSE [21]

Convex Combination of Semantic Embeddings (ConSE) is a framework that allows for online knowledge transfer by the use of simple mathematical concepts, together with the knowledge stored within the trained model.

To achieve that, this framework takes a new sample x and lets the trained model produce the probabilities associated with it. For simplicity, let's denote the probability assigned by the trained model θ to a label $y_s \in Y_{seen}$ as $p_\theta(y_s|x)$, such that $\sum_{y_s \in Y_{seen}} p_\theta(y_s|x) = 1$. For each input sample, the model assigns a probability to every label in the seen set, Y_{seen} . This assignment occurs irrespective of whether the sample's ground-truth label is a member of the seen set or not.

Given the probability distribution p_θ produced by the model trained on D_s for a specific sample x , the top- k most likely predictions can be retrieved. The top-1 prediction, according to model θ , is denoted as $y_\theta(x, 1)$, which is formally defined as:

$$y_\theta(x, 1) = \arg \max_{y \in Y_{seen}} p_\theta(y|x) \quad (4)$$

Analogously, the t^{th} largest prediction is denoted as $y_\theta(x, t)$. From the top- T predictions, ConSE computes a new semantic embedding $conse(x)$ as a convex combination using the following formula:

$$conse(x) = \frac{1}{Z} \sum_{t=1}^T p_\theta(y_\theta(x, t)|x) \cdot s(y_\theta(x, t)) \quad (5)$$

Here, $s(y_\theta(x, t))$ is the semantic embedding for label $y_\theta(x, t)$, and $Z = \sum_{t=1}^T p_\theta(y_\theta(x, t)|x)$ is a normalization factor. The final prediction is made by finding the unseen label $y' \in Y_{unseen}$ with the highest cosine similarity to the $conse(x)$ embedding computed using eq. (5).

3.6.2 COSTA [18]

In contrast to the mathematical approach of ConSE, the Co-Occurrence Statistics (COSTA) framework uses the outputs of multiple classifiers to infer class similarity. The underlying concept is that similar classes tend to co-occur in similar contexts. This co-occurrence can be quantified and utilized to determine the weights of a new regression model, which can either be directly used for label prediction or integrated into an existing classifier.

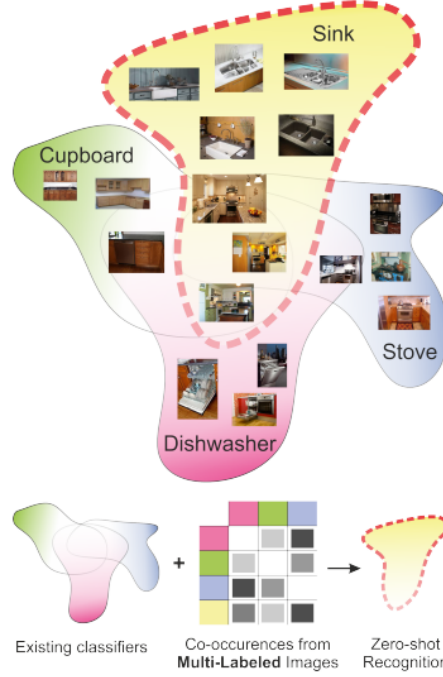


Figure 16: Illustration of COSTA taken from [18]: the classifier for an unseen label is estimated using a weighted combination of existing classifiers and their co-occurrence statistics.

Given k classifiers for the k existing labels, the classifier for the new label y' can be computed as a weighted summation of the existing weights, weighted by the co-similarity metric between y' and each of the known classes weights.

$$w_{y'} = \sum_{y_k \in Y_{seen}} w_{y_k} \cdot s_{y', y_k} \quad (6)$$

There are several proposed ways in [18] to compute the co-occurrence similarities:

- **Normalized co-occurrences.** $s_{ij}^n = \frac{c_{ij}}{c_i}$ with c_{ij} being the number of times labels i and j appears together in a context, which in the case of this thesis is the number of attributes in the semantic space that are relevant for both the labels, and c_i is the total number of occurrences for label i , hence the total number of attributes for which label i is relevant

- **Binarized Co-occurrences.** Motivated by the binarized class-to-attributes mappings used in attribute-based zero-shot classification, this approach is defined as: $s_{ij}^b = \mathbb{I}[c_{ij} \geq t]$ where t is a global threshold set as $t = \frac{1}{m^2} \sum_{i,j} c_{ij}$.
- **Burstiness Corrected Co-occurrences.** Burstiness is the phenomenon where the frequency of an observation is significantly larger than a statistically independent model would predict. The square-root function is commonly used in image retrieval and classification to correct for the burstiness of visual features. Similarly, to reduce the burstiness in label co-occurrences, this metric uses: $s_{ij}^s = \sqrt{c_{ij}}$
- **The Dice’s Coefficient.** $s_{ij}^d = \frac{c_{ij}}{c_i + c_j}$ This measure is used in many Natural Language Processing systems to estimate the semantic relatedness between two terms, typically based on hit counts from web search engines.

This framework allows for the computation of weights that can be injected directly into a trained classifier, thereby enhancing its performance in a hypernetwork-like fashion, much like the approach taken by ICIS. The primary limitation of this method, however, is that the weight computation is static. It relies exclusively on co-occurrence similarity and does not integrate information from the feature space of the trained model.

3.6.3 Analysis of I-ZSL Baselines Results

As shown in Figure 15, the methods analyzed in [5] exhibit very poor performance in the image-free scenario. This is primarily due to the detachment of the knowledge transfer method from the feature space generated by the backbone of the trained model. ICIS, on the other hand, takes a different approach by connecting weights to semantics, generating model-aware regressors.

This allows ICIS to be the only method able to partially overcome the problem of the seen labels bias, showing the highest unseen accuracies (u) across all the datasets.

To gain insight into the impact of switching to a ResNet18, ConSE and COSTA were also applied to the origins discussed in Chapter 2. As expected, this resulted in a drop in performance due to the reduction in the original networks’ capabilities, as shown in table 2.

I-ZSL Method	Zero-Shot Accuracy			Generalised Zero-Shot Accuracy								
	CUB	AWA2	aPY	CUB			AWA2			aPY		
	Acc	Acc	Acc	u	s	H	u	s	H	u	s	H
ConSE	30.3	39.3	29.2	1.7	70.4	3.4	0.2	91.9	0.4	0.1	88.2	0.5
COSTA	1.8	14.8	16.5	0.0	72.5	0.0	0.0	92.1	0.0	0.1	88.4	0.2

Table 2: Comparison of different Zero-Shot Learning methods adapted to the image-free constraint tested over thesis datasets, on ResNet18 classifier.

3.7 ICIS [5]

Unlike traditional hypernetwork-based methods, which are based on complex mechanisms to allow weights generation like those seen in section 3.3, ICIS solves the task of I-ZSL much more simply and efficiently by connecting the embedding spaces generated by a joint autoencoder. This autoencoder can then be used in any of the four resulting directions: weight-to-weight; semantic-to-semantic; weight-to-semantic; semantic-to-weight.

As it is the foundational architecture inspiring most of this thesis work, a deeper description of its architecture and the requirements to train the framework will be detailed in this section.

3.7.1 Architecture

The ICIS architecture is composed of two autoencoders, each trained in a self-supervised manner to reconstruct the specific type of information it is assigned. While this approach is functionally capable of producing a latent space for both autoencoders, without any further addition, it would fail to establish a connection between the learned weight representations and the semantic descriptions of the classes.

For the process to operate as intended, it is necessary to establish an effective mechanism that enables the decoding stage to generate an entire set of weights. This generative-like process requires the semantic description of a class to serve as the guiding element for weight generation.

To address this requirement, the authors proposed a joint autoencoder design, in which the latent spaces produced during encoding are also leveraged to reconstruct the complementary representation.

Given a label l and the embedding associated with the semantic description of that label e_l , the semantic autoencoder can be thought of as the combination of two functions:

- **The encoder function** $\phi_s : A \rightarrow Z_A$, where A is the semantic space, made of the embeddings of each available label, and Z_A the latent space associated with the semantics
- **The decoder function** $\varphi_s : Z_S \rightarrow A$ that reverts the encoding

By selecting appropriate loss functions, autoencoders can be employed as self-supervised learning methods for the construction of latent spaces. In the standard formulation, the autoencoder is trained to reconstruct the original input. However, by altering the loss function or the ground truth, it is also possible to train the model to modify the input in specific ways. For instance, one could design a latent space that intentionally degrades the input representation. This flexibility is one of the fundamental reasons why such a simple framework can be adapted for a wide range of downstream tasks.

In the context of the present framework, an autoencoder is also utilized to learn a latent space for the classifier weights. A key limitation for future adaptations arises at this stage: the classifier weights and biases are flattened into a single vector, which is then processed by the weights autoencoder. This autoencoder can be formally defined as the concatenation of:

- **Encoder function** $\phi_w : W \rightarrow Z_w$, where W denotes the weight space of the trained classifier head, and Z_w the learned latent space;
- **Decoder function** $\varphi_w : Z_w \rightarrow W$, which reconstructs the weights from the latent representation.

As previously mentioned, the two autoencoders are also employed to reconstruct information in the opposite direction. To enable this, two additional mapping functions are introduced:

- **Weight-to-Semantic** $\varphi_{w \rightarrow s} : Z_w \rightarrow A$,
- **Semantic-to-Weight** $\varphi_{s \rightarrow w} : Z_s \rightarrow W$.

These two functions are employed during training to align the latent spaces of weights and semantics. In doing so, they influence the output of the weights decoder, thereby enabling the integration of novel class semantics into the existing classification head of the trained model.

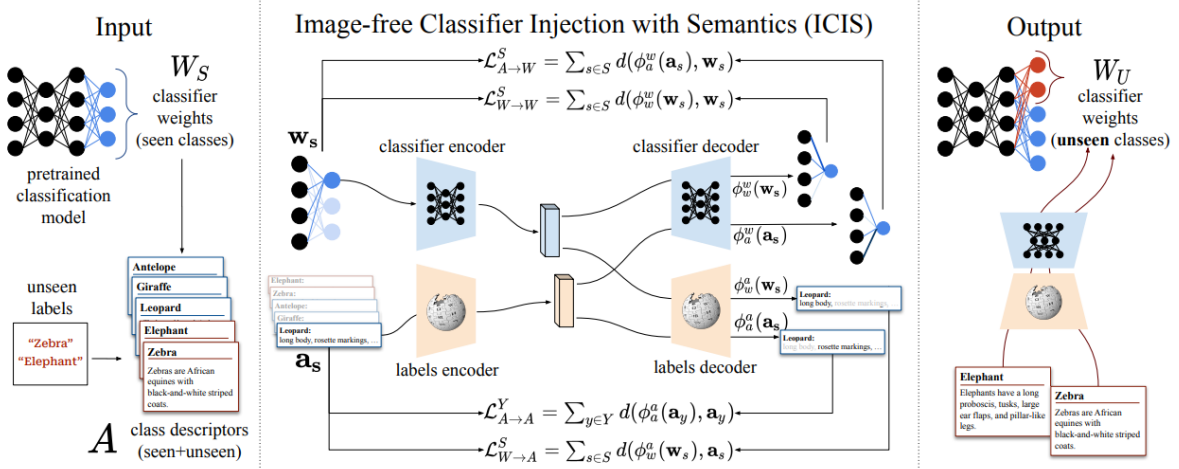


Figure 17: ICIS framework’s architecture, taken from [5]. It is composed of two autoencoders (blue and ocher), with the blue one being the weights autoencoder, representing the $\varphi_w \circ \phi_w$ function, and the ocher one being the semantics autoencoder representing the $\varphi_s \circ \phi_s$ function. Additionally, two additional arches are visible: the one from the weights latent space to the Semantics Output, which represents the $\varphi_{w \rightarrow s}$ mapping, and the one from the semantics latent space to the Weights Output, which represents the $\varphi_{s \rightarrow w}$ mapping.

3.7.2 Data Preparation

As with most hypernetworks, ICIS requires a preprocessing stage for the parameters of the existing network. However, unlike the approaches introduced in section 3.3, which rely on complex data requirements, ICIS depends only on minimal and simple information. This characteristic constitutes both a key strength and a significant limitation of the framework.

In general, Deep Neural Networks for Computer Vision can be conceptually divided into two components:

- **Backbone:** the initial portion of the network, responsible for extracting the core features of an image. Formally, the backbone can be expressed as a function that maps an image to a feature vector $f \in \mathbb{R}^d$, where d denotes the dimensionality of the feature space (512 in the case of ResNet18).
- **Classification Head:** a fully connected linear layer that subdivides the feature space generated by the backbone, acting as a regression function capable of assigning the input to one of multiple categories.

A central advantage of the ICIS framework is that it requires only the weights of the classification head. Consequently, it is possible to construct a dataset in which each class is uniquely associated with both (i) a set of weights (512 for each class in the case of ResNet18) and (ii) a semantic embedding, whose form depends on the method employed to extract embeddings from the class label.

Christensen et al. [5] propose several modalities for extracting context-aware semantic vectors from labels. These include CLIP-based text embeddings [24], BERT embeddings [8], and normalized semantic descriptors as introduced in [30]. Any of these options are viable, as the semantic vectors serve primarily as centroids guiding the generative process of the decoder when producing weights for novel classes.

Once a semantic space A has been constructed using a method of choice, it is possible to collect, for each label $l \in Y_{seen}$, a pair (θ_l, s_l) , where θ_l represents the set of weights in the trained classification head corresponding to label l , and s_l denotes the semantic vector describing that label.

The resulting dataset consists of multiple weight–semantic pairs and can be formally defined as

$$D_{ICIS} := \{(\theta_l, s_l) \mid l \in Y_{seen}\},$$

which is then used during training by the ICIS framework to learn the mapping from semantic descriptions to the weight space of the trained classification head.

3.7.3 Training Process

Once D_{ICIS} has been constructed, training can proceed following a standard approach by dividing the dataset into training, validation, and test subsets to properly assess the generalization capabilities of the method.

As illustrated in fig. 17, the framework introduces four loss functions, each corresponding to a direction in the encoding-decoding process:

- **Weight-to-Weight.** $L_{W \rightarrow W}^{Y_{seen}} = \sum_{l \in Y_{seen}} d(\varphi_{w \rightarrow w}(w_l), w_l)$ models the reconstruction of classifier weights from the classification head of the trained network. This loss measures the distance between the reconstructed weight set and the original weights for each class in the seen set.
- **Semantic-to-Semantic.** $L_{A \rightarrow A}^Y = \sum_{l \in Y} d(\varphi_{s \rightarrow s}(s_l), s_l)$ defines the reconstruction of semantic embeddings. Unlike the weight reconstruction, this loss may be minimized across the entire label set Y if additional labels are available at training time; otherwise, it is restricted to Y_{seen} . The reconstruction quality is again assessed using a generic distance function.
- **Cross-Domain Losses.** $L_{W \rightarrow A}^{Y_{seen}} = \sum_{l \in Y_{seen}} d(\varphi_{w \rightarrow s}(w_l), s_l)$ and $L_{A \rightarrow W}^{Y_{seen}} = \sum_{l \in Y_{seen}} d(\varphi_{s \rightarrow w}(s_l), w_l)$ enforce alignment between the two latent spaces. These terms ensure that semantic descriptions can be effectively mapped to classifier weights, thereby enabling the generation of weights for unseen labels.

It is important to note, however, that cross-domain losses do not guarantee a perfect alignment between the latent spaces. This issue was explicitly investigated by the authors in their ablation study, where different distance metrics were evaluated, including the L2 norm and cosine similarity.

ICIS Ablation	Zero-Shot Accuracy			Generalised Zero-Shot Accuracy								
	CUB	AWA2	SUN	CUB			AWA2			SUN		
	Acc	Acc	Acc	u	s	H	u	s	H	u	s	H
MLP base model	41.4	46.8	49.7	0.0	87.6	0.0	2.0	95.9	4.0	0.0	50.1	0.0
+ Cosine loss	52.7	50.9	48.5	36.6	76.6	49.5	27.3	93.1	42.2	21.9	43.6	29.2
+ Within spaces	58.0	63.5	50.8	39.6	77.6	52.5	33.8	93.5	49.6	38.9	32.5	35.4
+ Across spaces	60.1	64.8	51.7	44.5	75.0	55.9	35.5	93.6	51.4	44.0	27.6	33.9
+ Include A_U	60.6	64.6	51.8	45.8	73.7	56.5	35.6	93.3	51.6	45.2	25.6	32.7

Figure 18: Ablation study from Christensen et al. [5]. The most effective configuration, also reported in fig. 15, consists of using cosine similarity for the cross-domain losses and including unseen labels in the semantic reconstruction.

3.7.4 Key Limitations

Although conceptually simple, the ICIS framework presents several critical limitations, particularly when its application is extended to domains such as Machine Unlearning (MU).

The first limitation is that the framework entirely disregards the backbone of the model when computing weights. While this approach is effective for defining new class partitions within the feature space of a trained model, it cannot be directly applied to more complex tasks such as MU. Restricting modifications to the classification head in this context would merely result in an output suppression mechanism rather than genuine unlearning.

A second limitation concerns scalability. Due to its architectural simplicity, the framework does not support complex computations or adaptations. For ZSL, this simplicity is not problematic, as the classification head depends only on the feature space generated by the backbone and the number of target classes. However, when moving beyond this setting, the lack of scalability becomes a major obstacle.

As will be discussed in the HyCUS adaptation in section 3.10, the method cannot be extended to multiple layers of a classifier network, especially as model complexity increases. The central challenge lies in the vast number of parameters contained in a full network, or even within a single layer of the backbone, which makes direct application infeasible.

To better understand which components are necessary to overcome these challenges, this thesis will also analyze the Machine Unlearning problem, which can be regarded as the symmetric counterpart of Zero-Shot Learning.

This analysis begins with a review of baseline approaches, in order to understand the problem setting and how it is addressed by state-of-the-art methods. Building on this, a possible adaptation of ICIS to MU will be explored to assess the limits of the framework and identify techniques that could extend its applicability.

3.8 Machine Unlearning (MU)

Machine Unlearning (MU) is a subfield of Machine Learning that aims to remove specific knowledge acquired by a model during training. This knowledge may consist of individual samples, concepts, or entire categories, such as classes in classification tasks. The goal is to produce a model that behaves as if it had never seen the removed data. Retraining from scratch achieves this effect and is commonly referred to as the **golden approach**, serving as a benchmark for evaluating unlearning methods. MU has gained increasing attention due to both practical needs and regulatory requirements.

Legally, frameworks such as the General Data Protection Regulation (GDPR) enforce the *"right to be forgotten"*, allowing individuals to request the deletion of their personal data. MU provides a mechanism to ensure that models no longer retain or exploit information derived from such data, but is also relevant to remove bad data such as noisy, mislabeled, biased, or otherwise harmful. Removing such information can reduce confusion, mitigate overfitting, and improve the robustness and fairness of the model.

Although retraining from scratch is the most reliable form of unlearning, it is often computationally infeasible for large-scale datasets. Consequently, alternative strategies have been developed, primarily divided into: (i) fine-tuning-based methods, and (ii) student-teacher frameworks.

Unlearning can occur at different levels of granularity:

- **Class Unlearning:** removing knowledge of one or more labels from a classifier.
- **Sample Unlearning:** eliminating the influence of specific samples from training.
- **Concept/Feature Unlearning:** removing entire sets of features or concepts (e.g., nudity, violence) so the model can no longer recognize them.

Each scenario presents distinct challenges regarding computational cost, retention of useful knowledge, and verification of successful unlearning. Sample unlearning, for example, is particularly relevant for privacy-critical applications, whereas class-level unlearning addresses evolving taxonomies or systematic labeling errors.

It is necessary to mention that the work of this thesis will focus on *class unlearning*, where entire categories are removed from the model's representation, with concepts that could be applied also to other kinds of unlearning. This decision was mainly driven by the amount of work necessary to understand and analyze each of the single parts of the unlearning task, thereby focusing on what is considered the most simple one to gather a first, thorough analysis of the proposed methods.

The objective of this section is to investigate baseline MU strategies, assess their effectiveness in removing targeted information, and evaluate their impact on both retained and removed classes and will serve as a base to better understand the purposes of the various introduced elements.

3.8.1 Problem Statement

To formalize Machine Unlearning for classification tasks, consider the dataset on which the origins, introduced in section 2, were trained. Let $D_s := \{(x, y) \in D \mid y \in Y_{seen}\} \subset D$ denote the subset of data containing only the labels observed during training.

This dataset can be partitioned into a *forget-set* D_f and a *retain-set* D_r , such that $D_f \cap D_r = \emptyset$. The forget-set contains the elements intended to be removed from the model’s knowledge (typically one or more labels), while the retain-set contains the points that should be preserved.

A successful unlearning method must ensure that the final model can no longer recognize the removed labels, without degrading performance on the retain set. A key challenge is distinguishing between true unlearning and mere output suppression, where the model’s ability to recognize a label may remain internally, but the final output is masked.

Output suppression, in a classification context, could be implemented by zeroing the weights corresponding to the labels to be forgotten in the classification head. However, this approach does not remove the underlying feature representations learned for the forgotten labels as the unlearned network would still extract and process information about these labels in a way that could be exploited to still recognize the unlabeled data as part of the training set.

Therefore, a label can be considered truly removed if the model behaves as if the label was never part of its training set, both in the backbone feature representations and in the classification head. For this reason, the definition of class unlearning that will be leading the analysis can be the following:

A label l is considered forgotten if, for all samples x labeled as l , the model treats them as unrelated with the model’s training-set, across both the backbone and the classification head, and l is excluded from the set of recognized labels.

The following section will therefore detail the metrics used to evaluate unlearning methods under this perspective, having care that a subset of those metrics can constitute a valid, retraining-free set of metrics, thereby providing a meaningful contribution to existing research in Machine Unlearning.

3.8.2 Evaluation Baseline

Evaluating Machine Unlearning (MU) methods requires metrics that measure both forgetting and retention, while distinguishing true unlearning from mere output suppression. To this end, several complementary metrics are adopted, each motivated by a specific requirement and defined precisely to enable reproducibility. In what follows, each metric will be presented together with the reasons leading to their choice.

Retain Accuracy. Evaluates whether useful knowledge is preserved after unlearning. This metric is useful as a method can be considered successful only if forgetting does not come at the expense of overall predictive performance. The opposite phenomenon is known in literature as **catastrophic unlearning** and would compromise the model usability.

$$\text{Acc}_r = \frac{1}{|D_r|} \sum_{(x,y) \in D_r} \mathbb{1}[\hat{y}(x) = y]. \quad (7)$$

where $\hat{y}(x)$ denotes the predicted label. A high Acc_r indicates that the model maintains competence on the retained classes.

Forget Accuracy. Quantifies the extent to which the model has lost the ability to recognize samples from the forget-set D_f . It helps identifying whether or not the label is still considered as part of the model’s knowledge, and in which scale.

It is defined analogously to Retain Accuracy as:

$$\text{Acc}_f = \frac{1}{|D_f|} \sum_{(x,y) \in D_f} \mathbb{1}[\hat{y}(x) = y]. \quad (8)$$

The value indicating effective forgetting must be as low as possible, with a tolerance that depends on the amount of labels in the dataset. In this thesis analysis two decimals of the accuracy percentage will be considered as relevant.

Adaptive Unlearning Score (AUS). Captures a trade-off by comparing the unlearned model to the origin model, by offering a balance-assessing metric between Retain and Forget accuracies.

Denoting Acc_r^* and Acc_f^* as the retain and forget accuracies of the origin model, AUS is defined as:

$$\text{AUS} = 1 - \frac{1}{2} \left(\frac{|\text{Acc}_r - \text{Acc}_r^*|}{\text{Acc}_r^*} + \frac{|\text{Acc}_f - \text{Acc}_f^*|}{\text{Acc}_f^*} \right). \quad (9)$$

A higher AUS indicates that the unlearned model is successfully retaining information in a substantial way with what it was able to do before applying the unlearning method, while reducing the forget accuracy.

Jensen–Shannon Divergence (JSD) from the Retrained Model. After forgetting, a model should behave as if it had never encountered the forget-set. While Forget Accuracy measures whether the model can still correctly classify forgotten samples, it does not capture *how* this forgetting occurs. For example, low Forget Accuracy could result from trivial suppression of outputs or from unintended label translation (i.e., mapping forgotten labels to retained ones). In contrast, a model retrained from scratch on the retain-set would still learn meaningful features but would lack the ability to map them to the forgotten classes. The probability distributions of such a model would therefore remain structured but uncertain on forgotten samples. JSD provides a principled way to capture this effect: as a symmetric divergence based on the Kullback–Leibler (KL) divergence, it measures how closely the probability space of the unlearned model aligns with that of the golden retrained baseline, which by construction has never seen the forget-set. For a sample x , let $p(x)$ and $q(x)$ denote the softmax output distributions of the unlearned and retrained models, respectively. The JSD is defined as:

$$\text{JSD}(p \parallel q) = \frac{1}{2} \text{KL}(p \parallel m) + \frac{1}{2} \text{KL}(q \parallel m), \quad (10)$$

where $m = \frac{1}{2}(p + q)$. The Kullback–Leibler divergence between two discrete probability distributions p and q is given by:

$$\text{KL}(p \parallel q) = \sum_i p(i) \log \frac{p(i)}{q(i)}. \quad (11)$$

Averaging $\text{JSD}(p(x) \parallel q(x))$ across all samples quantifies distributional similarity, with smaller values indicating closer alignment between the unlearned model and the golden baseline.

Loss-wise Membership Inference Attack (l-MIA). In the context of Machine Unlearning, an essential requirement is that the model should not retain training-specific information about the forgotten samples. Since we have full knowledge of both the training and testing partitions of the forget-set, we can directly evaluate this by testing whether the model assigns systematically lower losses to training instances than to testing instances. If such a distinction persists, it indicates that the classification head still embeds traces of the forgotten data, revealing incomplete unlearning.

For each sample (x, y) , the loss $\ell(x, y)$ is computed. A regressor or threshold τ is then used to distinguish between forgotten training samples and forgotten testing samples:

$$\text{MIA}_{\text{loss}}(x, y) = \begin{cases} 1, & \ell(x, y) < \tau \quad (\text{classified as train/“member”}) \\ 0, & \ell(x, y) \geq \tau \quad (\text{classified as test/“non-member”}) \end{cases}. \quad (12)$$

If, after unlearning, a regressor can no longer reliably separate train and test elements of the forget-set, it implies that the model no longer retains knowledge of the forgotten samples from the classification head’s perspective. The effectiveness of the attack is typically quantified through metrics such as AUC or accuracy, where values close to random guessing indicate successful unlearning.

Feature-wise Membership Inference Attack (f-MIA). Beyond the classification head, traces of forgotten data may persist in the feature representations learned by the backbone. Even if the output layer no longer predicts forgotten classes, the internal embeddings might still encode discriminative information about them. Since complete knowledge of both the training and testing partitions of the forget-set are known at training-time, we can directly assess whether the backbone continues to separate training and testing instances of forgotten labels. If such separability is present (for example, if a regressor can reliably identify distinctive patterns or stronger feature activations in training samples), this indicates that the forgotten data has not been fully removed from the representation space.

For each sample (x, y) , the backbone representation $f(x)$ is extracted. A regressor or classifier is then trained to differentiate between forgotten training samples and forgotten testing samples based solely on these embeddings:

$$\text{MIA}_{\text{feat}}(x, y) = \begin{cases} 1, & f(x) \text{ classified as train/“member”} \\ 0, & f(x) \text{ classified as test/“non-member”} \end{cases} . \quad (13)$$

If, after unlearning, the regressor can no longer reliably distinguish training from testing elements of the forget-set, this implies that the backbone no longer retains meaningful information about the forgotten data. As with the loss-wise attack, the effectiveness of the feature-wise MIA is measured using AUC or accuracy, where performance close to random guessing indicates successful unlearning at the representation level.

Taken together, these metrics establish a complete and multi-layered framework for evaluating Machine Unlearning in classification tasks, also offering a subset of metrics that allow for retraining-free evaluation of such methods.

JSD, in fact, only serve as a comparative method to confirm that the unlearning is effectively happening, but apart from the academic perspective, it does not offer any additional insight about unlearning with respect to the other metrics. It in fact measures how distant the golden-standard probability space distances from the unlearning method, but it is important to notice that the retraining doesn’t always converge to the same configuration. Hereby, metrics that compare a method to the golden-standard, while useful for academic purposes, do not provide a complete picture of the unlearning quality.

Finally, Membership Inference Attacks (MIAs) serve as adversarial probes: the loss-wise variant tests whether traces of forgotten data remain in the classification head, while the feature-wise variant examines the backbone embeddings to detect residual patterns that could still reveal membership.

By jointly considering output-level, and representation-level behaviors, this collection of metrics enables a rigorous and fine-grained assessment of unlearning methods in computer vision. It ensures that evaluated approaches are not only effective in suppressing unwanted predictions but also thorough in removing the underlying knowledge, thus aligning with both practical requirements and theoretical necessities of Machine Unlearning.

3.9 Literature Methods

While Zero-Shot Learning (ZSL) approaches have been extensively evaluated on the datasets considered in this thesis, most Machine Unlearning (MU) methods have primarily been assessed on CIFAR-like benchmarks. Consequently, this section applies existing MU approaches to the ZSL datasets introduced in the previous chapter. This serves a twofold purpose: (i.) to establish a reliable benchmark for the proposed method, which adapts ICIS to the unlearning setting, and (ii.) to evaluate the robustness of existing approaches when confronted with data distributions that diverge from those typically studied in the literature.

The methods are evaluated on the datasets and experimental setups described in section 2, enabling a comprehensive analysis of their performance across diverse domains. To ensure a sound academic benchmark, the first step consists of retraining ResNet18 exclusively on the retain set, thereby producing a reference (or “golden”) model against which all unlearning methods can be consistently compared.

Before presenting the results, however, several considerations regarding the interpretation of the unlearning metrics must be outlined:

1. The retraining runtime metric ($Time_{run}(\downarrow)$) does not accurately reflect the cost of a full retraining procedure. Since it is based only on fine-tuning from ImageNet1k pre-training, its value is highly dependent on the similarity between the target dataset and the pre-training distribution.
2. Given the relatively small size of the datasets used in this thesis, it is important to note that unlearning procedures consist of two types of iterations: those applied solely on the forget set (which is generally much smaller) and those aimed at correcting knowledge degradation on the retain set (which is comparatively larger).
3. All methods were trained under an innovative and effective early-stopping criterion designed to capture the dynamics of unlearning. Specifically, both retain and forget accuracies are monitored during training, yielding a multi-objective scenario where the goal is to maximize retain accuracy while minimizing forget accuracy. To this end, the early-stopping monitor evaluates the aggregated metric $Acc_r - Acc_f$, which simultaneously reflects the widening gap between the two accuracies and the specialization of a method on the retain set. Unfortunately, this requires evaluating the methods on both D_f and D_r , which is a significant drawback for methods that would otherwise necessitate only one of the two subsets.

3.9.1 Retraining

Retraining serves as the most reliable, yet computationally expensive, baseline for evaluating the effectiveness of unlearning methods. In this setup, the model is retrained from scratch using only the retain set, ensuring that all information related to the forget set is completely removed. To maximize performance and provide a fair reference point, the retraining procedure is carried out by fine-tuning a ResNet18 model pretrained on ImageNet-1k, following standard transfer learning practices.

For each dataset considered in this thesis, three versions of the retraining baseline are generated, corresponding to the low-forget, average-forget, and high-forget scenarios described in section 2. This ensures that the baseline is consistently available across all levels of forgetting difficulty and across the different Zero-Shot Learning (ZSL) benchmarks under analysis.

The results indicate that, as the proportion of labels to be forgotten increases, the retrained models tend to slightly specialize in the classes belonging to the retain set, as shown also in the AUS metric. This effect highlights the trade-off between removing knowledge of the forget classes and improving discriminability among the retained ones, and it provides valuable insight into the behavior of unlearning in settings with varying degrees of class removal.

Table 3: Baseline results on the **CUB** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), l-MIA, f-MIA, and runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
Origin Training	72.4	-	-	83.2	72.3	35m 12s
Low-Forget	74.4	0.00	1.03	54.8	44.2	32m 46s
Avg-Forget	75.9	0.00	1.03	50.6	49.8	25m 29s
High-Forget	74.6	0.00	1.02	50.9	53.9	41m 28s

Table 4: Baseline results on the **AwA2** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), l-MIA, f-MIA, and runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
Origin Training	92.6	-	-	50.4	61.2	46m 00s
Low-Forget	93.1	0.00	1.00	46.7	51.2	37m 06s
Avg-Forget	94.1	0.00	1.02	50.0	52.3	25m 38s
High-Forget	93.9	0.00	1.02	49.4	51.6	24m 25s

Table 5: Baseline results on the **aPY** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), l-MIA, f-MIA, and runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
Origin Training	89.9	-	-	55.5	68.3	12m 02s
Low-Forget	93.1	0.00	1.03	44.5	49.4	8m 42s
Avg-Forget	92.3	0.00	1.02	50.8	52.1	11m 40s
High-Forget	91.3	0.00	1.02	46.4	49.1	9m 29s

It should be emphasized that, unlike other unlearning methods, the Jensen–Shannon Divergence (JSD) is not reported for the retraining baseline, since it will be the ground truth to assess it. Furthermore, the results clearly show that retraining consistently eliminates both types of vulnerabilities, thereby providing the most secure benchmark. Lastly, the AUS metric offers an immediate indication of how strongly the model specializes on the retained set, making it a particularly informative complement to the other evaluation measures.

3.9.2 Fine-tuning

Fine-tuning represents a less computationally expensive alternative to retraining and is often considered a practical baseline for unlearning tasks. Instead of training a model from scratch, the original model is adapted by fine-tuning it exclusively on the retain set, allowing it to forget information related to the removed classes gradually. This approach leverages the already trained weights of the original model, thereby requiring significantly fewer resources compared to full retraining. It is also crucial to note that this method only requires iterating over the retain set, whereas all the other methods will need iterations on both the retain and forget sets.

For consistency with the retraining baseline, three versions of the fine-tuning baseline are generated for each dataset, corresponding to the low-forget, average-forget, and high-forget scenarios with the addition of the JSD metric, which will measure how divergent the produced output is, compared to the golden-standard method. This enables a direct comparison across levels of forgetting difficulty.

Table 6: Fine-tuning baseline results on the **CUB** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	73.9	0.00	1.02	0.12	64.6	78.8	9m 16s
A-Forget	75.0	0.00	1.02	0.13	67.5	82.4	6m 50s
H-Forget	77.4	0.00	1.05	0.12	54.7	77.1	13m 45s

Table 7: Fine-tuning baseline results on the **AwA2** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	89.8	0.00	0.97	0.05	46.3	53.0	16m 20s
A-Forget	91.7	0.00	0.99	0.06	50.3	54.5	17m 34s
H-Forget	91.7	0.00	0.99	0.07	51.2	55.7	13m 50s

Table 8: Fine-tuning baseline results on the **aPY** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	91.4	0.003	1.01	0.07	56.1	57.9	5m 15s
A-Forget	91.4	0.000	1.01	0.07	51.1	59.2	6m 04s
H-Forget	90.4	0.000	1.01	0.08	51.1	50.9	7m 14s

From the results presented in the tables, it is evident that fine-tuning can be an extremely fast approach when properly configured for the unlearning task. However, it exhibits critical limitations that make it unsuitable in several scenarios:

1. As shown in table 8, a subtle yet important indication of poor unlearning quality is observed in the low-forget scenario, where the forget accuracy (Acc_f) is not fully reduced to zero. While this may appear to be a minor issue, it is important to note that the datasets used are relatively small and not fully representative of real-world conditions. In larger or more complex datasets, this trend could become much more pronounced, potentially leading to catastrophic failures in the unlearning process.
2. Although fine-tuning only iterates over the retain set, it does not directly address the removal of information from the model. This highlights the necessity, in Machine Unlearning methods, of also performing iterations over the forget set. Conversely, restricting updates solely to the forget set neglects the information that must be preserved, often resulting in a degradation of retain accuracy.
3. table 6 illustrates that, while information may appear to be removed from the perspective of the loss-MIA metric, the f-MIA metric reveals that a more invasive modification of the backbone is necessary to fully eliminate memorization. This example highlights a key limitation of current unlearning evaluation metrics: they do not fully capture the extent or quality of forgetting.

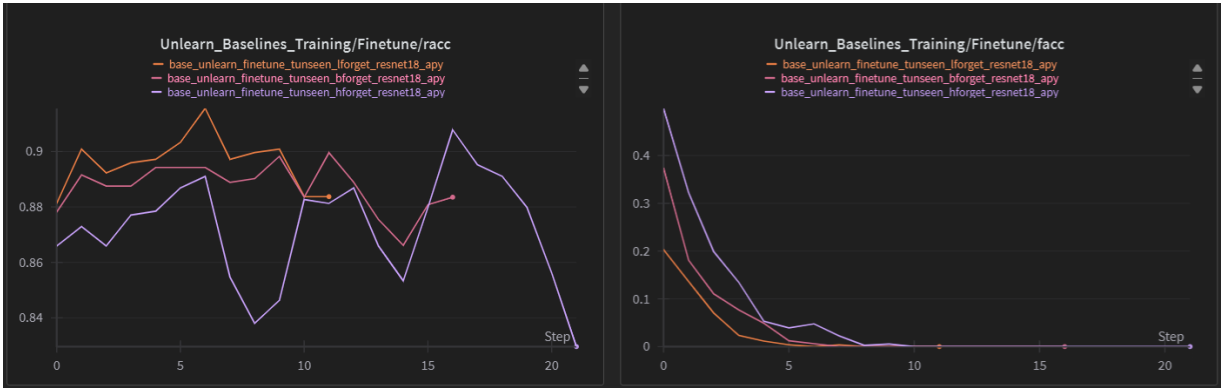
For a more thorough evaluation of the method, also charts of the retain and forget accuracies are here reported, showing a trend to slower forgetting speed as the dataset complexity increases, having its lowest peak in the aPY dataset, where the data distribution is the most varied.



(a) CUB charts of the Retain Accuracy (left) and the Forget Accuracy (right) during the unlearning process. As expected, due to its fine-grained nature, it is the dataset that required the most iterations, and that presents much higher oscillations.



(b) AwA2 charts of the Retain Accuracy (left) and the Forget Accuracy (right) during the unlearning process. It required the fewest number of epochs to be completed, showing a complete zeroed forget accuracy after only one epoch.



(c) aPY charts of the Retain Accuracy (left) and the Forget Accuracy (right) during the unlearning process. As the most complex dataset, in terms of data distribution, a similar trend to CUB is shown: high oscillations and difficulty in zeroing the forget accuracy, causing the method to need multiple iterations over the retain set.

Figure 19: Finetuning trends in Retain and Forget Accuracies.

3.9.3 NegGrad+ [17]

NegGrad+ is an advanced unlearning baseline that explicitly targets the removal of information associated with the forget set by applying negative gradients, while concurrently preserving knowledge on the retain set through standard fine-tuning. Unlike naive fine-tuning, which only adapts the model to the retain set, NegGrad+ performs a dual operation: it “reverses” the learning acquired on the forget set by backpropagating negative gradients computed on the examples to be forgotten, and simultaneously updates the model on the retain set to maintain accuracy. This combined procedure allows the model to unlearn selectively, rather than relying solely on passive forgetting through omission.

This dual application of gradients aims at correcting the NegGrad [13, 1, 28] trend to degrade retain accuracy, which inevitably happens because it only iterates on forget-set, applying disruptive gradients that backpropagate to the shared backbone. This correction, when paired with Finetuning and NegGrad negative behavior, shows that for a data-based unlearning method to happen, it is necessary to iterate on both D_f and D_r .

An important advantage of NegGrad+ over simple fine-tuning is its ability to directly decrease the influence of the forget set on the model’s parameters. This targeted reversal makes it more effective at reducing both retainable memorization and feature-based leakage, which are typically highlighted by f-MIA and l-MIA metrics. At the same time, the method maintains high retain accuracy (Acc_r) by continuing to fine-tune on the retain set, balancing forgetting and preservation. The Jensen–Shannon Divergence (JSD) metric is also meaningful here, providing a measure of how the output distribution departs from the original model while selectively removing specific information.

Table 9: NegGrad+ baseline results on the **CUB** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	71.2	0.00	0.99	0.13	66.1	61.5	10m 25s
A-Forget	72.9	0.00	1.00	0.13	62.9	65.8	7m 36s
H-Forget	74.6	0.00	1.02	0.13	56.7	60.9	7m 32s

Table 10: NegGrad+ baseline results on the **AwA2** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	90.2	0.00	0.97	0.05	49.0	52.8	21m 46s
A-Forget	91.7	0.00	0.99	0.05	48.9	50.8	19m 55s
H-Forget	93.1	0.00	1.01	0.08	49.6	50.0	35m 07s

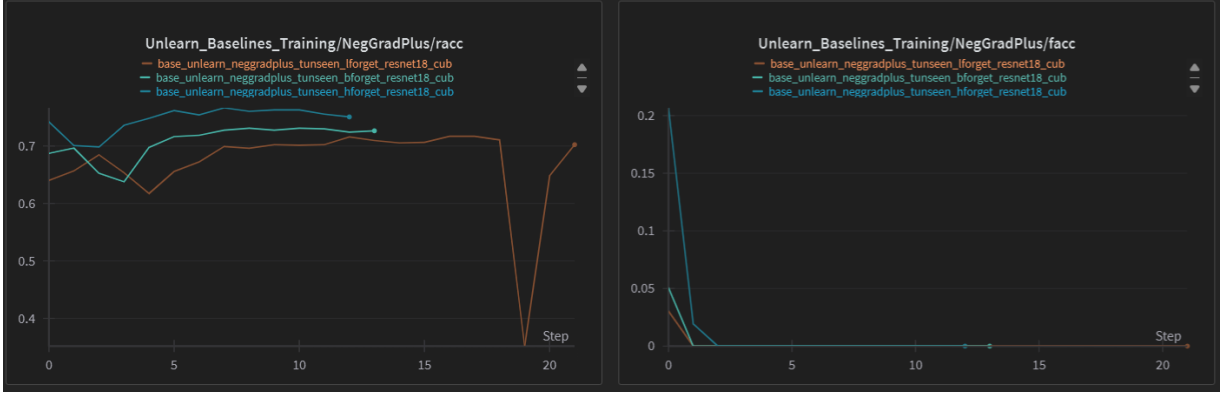
Table 11: NegGrad+ baseline results on the **aPY** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	92.6	0.00	1.02	0.05	53.5	59.7	5m 23s
A-Forget	91.7	0.00	1.01	0.07	53.4	51.9	9m 10s
H-Forget	90.2	0.00	1.01	0.08	52.3	46.5	6m 55s

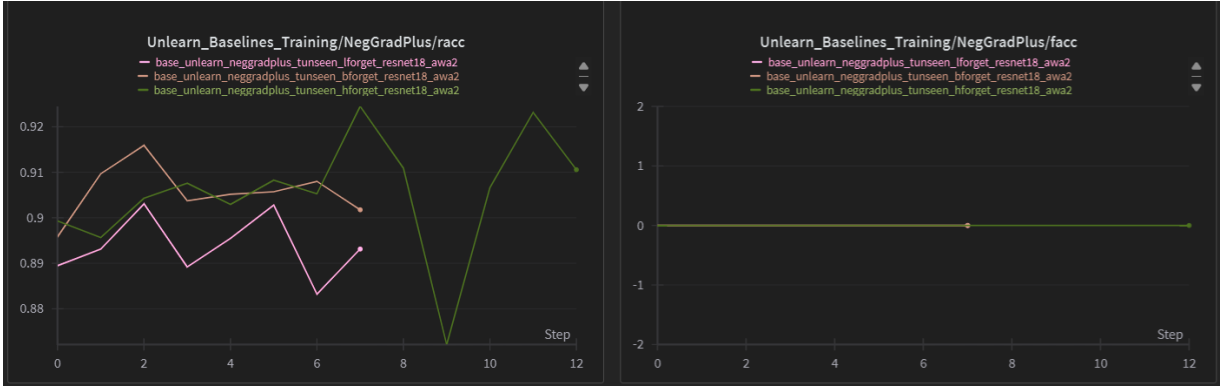
From the results in tables 9 to 11, NegGrad+ demonstrates effective and controlled forgetting while preserving high retain accuracy. Key observations include:

1. Forget accuracy (Acc_f) is consistently reduced to zero across all datasets and scenarios, highlighting the effectiveness of negative-gradient updates in removing targeted knowledge.
2. Compared to fine-tuning, NegGrad+ generally achieves lower l-MIA and f-MIA values, indicating a more comprehensive elimination of memorized information from the forget set, while maintaining AUS close to or slightly above 1.0.
3. Runtime is higher than plain fine-tuning due to the dual operations on both retain and forget sets, yet it remains substantially faster than full retraining, providing a strong trade-off between computational efficiency and unlearning quality.

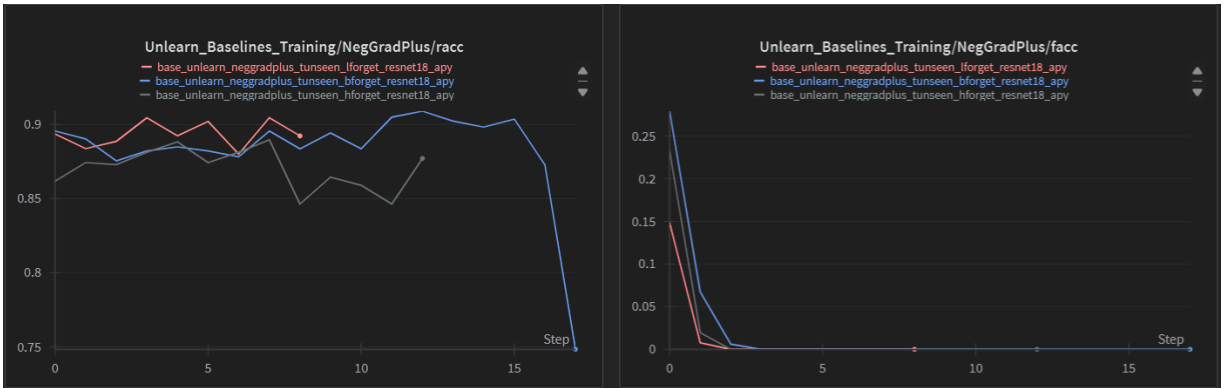
An additional notable trend is observed in the retain and forget accuracy charts (fig. 20), which demonstrate a markedly faster forgetting process. However, in the later stages of training, a tendency toward catastrophic unlearning becomes apparent, likely due to the application of negative gradients across the entire model backbone.



(a) CUB charts of Retain Accuracy (left) and Forget Accuracy (right) during the NegGrad+ unlearning process. A much more stable and quick unlearning is shown, with a huge drop as epochs advance due to the catastrophic unlearning effect.



(b) AwA2 charts of Retain Accuracy and Forget Accuracy during NegGrad+ unlearning. The same trend of the CUB dataset is clearly shown, with faster forgetting and stable unlearning with the same drop as the number of epochs increases.



(c) aPY charts of Retain Accuracy and Forget Accuracy during NegGrad+ unlearning. Trends show similar results to other datasets, demonstrating very solid and data-independent behavior.

Figure 20: NegGrad+ trends in Retain and Forget Accuracies.

3.9.4 Bad Teacher [6]

This framework employs a *student-teacher* paradigm to construct a new model, denoted as the *student*, which effectively forgets the knowledge contained in the forget set D_f while preserving the knowledge associated with the retain set D_r . This is achieved by selectively distilling information from two teachers: a *bad teacher* and a *smart teacher*. For clarity, the student model is denoted as θ_s , while the smart and bad teachers are denoted as θ_t and θ_d , respectively.

The method proceeds by initializing θ_t with the weights of the original pre-trained model, which is the model subject to unlearning. In contrast, θ_d is initialized with completely random weights, corresponding to a network with the same architecture as the original but without any acquired knowledge.

Through the use of a selection mask, θ_s is trained to match the probability distribution produced by θ_t when processing a sample $x \in D_r$, and to match the probability distribution produced by θ_d when processing a sample $x \in D_f$.

The optimization objective is based on the Kullback–Leibler (KL) divergence, which measures the dissimilarity between probability distributions as a difference of the log-softmax between the two spaces, that, by exploiting logarithmic properties, leads to the underlying definition:

$$\mathcal{L}_{\text{KL}} = \mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{i=1}^C P_T(y = i | x) \log \frac{P_T(y = i | x)}{P_S(y = i | x)} \right], \quad (14)$$

where P_T and P_S represent the teacher and student output distributions, respectively, and $\mathcal{D}$ is the dataset used (either retain or forget, depending on the loss component).

This process exhibits both a significant strength and a notable limitation, as will be demonstrated in the results table. Its main advantage lies in its ability to operate on the entire retain set as well as on subsets of it. This property arises because the KL divergence, unlike the standard Cross-Entropy Loss, provides a more general and data-independent target. In fact, as shown in [6], the authors employ only 30% of the retain set, effectively making their approach a pseudo-forget-only method.

While this characteristic offers considerable computational efficiency, it also introduces an important drawback: the partial reconstruction of knowledge from the forget set. Specifically, when θ_s attempts to converge toward the probability space of θ_t , undesired recovery of information from D_f may occur. This phenomenon will become evident in the Forget and Retain Accuracy plots, which display sudden increases even after the forget accuracy has been reduced to zero.

To ensure consistency with the baseline methods, the proposed approach is evaluated on the three benchmark datasets, under the experimental scenarios described in section 2.

Table 12: Bad Teacher baseline results on the **CUB** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	70.8	0.000	0.99	0.13	81.1	76.1	3m 52s
A-Forget	70.7	0.004	0.97	0.14	80.3	70.6	4m 23s
H-Forget	69.9	0.003	0.97	0.19	77.2	71.5	4m 39s

Table 13: Bad Teacher baseline results on the **AwA2** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	92.1	0.003	0.99	0.08	53.6	52.5	16m 15s
A-Forget	92.4	0.012	0.99	0.10	54.4	54.9	11m 39s
H-Forget	91.2	0.006	0.98	0.18	53.4	51.0	14m 29s

Table 14: Bad Teacher baseline results on the **aPY** dataset. Metrics reported: Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	89.6	0.00	0.99	0.11	65.1	61.5	3m 59s
A-Forget	87.9	0.00	0.97	0.19	60.9	57.4	5m 42s
H-Forget	88.8	0.00	0.99	0.17	54.9	49.0	9m 52s

As mentioned before, what emerges from the results table 12 to 14 is a tendency of the method to reconstruct knowledge on the forget-set while performing the retaining epochs. This issue also affects the ability of the method to effectively forget the data from an MIA point of view for both losses and features, especially in the fine-grained dataset CUB, where the origin’s vulnerabilities are further exposed.

An even better insight to that reconstruction issue, is given by the charts monitoring the retain and forget accuracies, as shown in fig. 21



(a) Application of the Bad Teacher method on the CUB dataset, illustrating a progressive degradation of retain accuracy across training epochs, with a slight increase in forget accuracy observed between epochs 6 and 7.



(b) Application of the Bad Teacher method on the AWA2 dataset, exhibiting slight oscillatory behavior across epochs and reaching an optimal gap between retain and forget performance after only three epochs.



(c) Application of the Bad Teacher method on the aPY dataset, demonstrating a consistent decline in retain accuracy over time, accompanied by multiple instances of partial reconstruction of knowledge on the forget set.

Figure 21: Bad Teacher trends in Retain and Forget Accuracies.

3.9.5 SCRUB [17]

This method is another student-teacher approach that builds on the Bad-Teacher framework, addressing some of the core limitations discussed in the previous section:

1. Unlike the Bad-Teacher method, which initializes a secondary “bad teacher” θ_d , this approach exclusively leverages the fully trained teacher model θ_t . The student model θ_s then selectively aligns with the teacher for samples $x \in D_r$ and diverges for samples $x \in D_f$. This eliminates the need for an additional model, reducing computational and memory overhead.
2. During the retain epochs, both the KL divergence term introduced in the previous section and the standard cross-entropy loss from fine-tuning are applied. The KL loss is weighted by a hyperparameter to mitigate the reconstruction issue:

$$\frac{\alpha}{N_r} \sum_{x_r \in D_r} d(x_r; \theta_s) + \frac{\gamma}{N_r} \sum_{(x_r, y_r) \in D_r} l(f(x_r; \theta_s), y_r) - \frac{1}{N_f} \sum_{x_f \in D_f} d(x_f; \theta_s), \quad (15)$$

where d measures the distance between the teacher model and the current student probabilities, produced by a sample x .

3. To handle the min-max contrast, a technique commonly used in adversarial networks is adopted: minimization epochs on the retain set are alternated with maximization epochs on the forget set, and maximization is halted once no further adjustments are possible.
4. After the forget accuracy reaches zero, additional fine-tuning is performed exclusively on the retain set. This “healing” step improves performance on retained data while ensuring that information from the forgotten set is effectively removed from the student model.

Although slightly slower than some alternative methods, SCRUB demonstrates consistent performance across all datasets and robust metrics across various scenarios. Furthermore, it exhibits stable behavior across different hyperparameter configurations. The only minor limitation is a slight degradation in performance under the low-forget scenario on the CUB dataset, highlighting the inherent challenge that fine-grained datasets pose for unlearning methods.

Table 15: SCRUB baseline results on the **CUB** dataset. Metrics include Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	69.9	0.00	0.98	0.15	50.8	58.0	17m 10s
A-Forget	75.5	0.00	1.03	0.12	54.4	55.6	23m 15s
H-Forget	76.2	0.00	1.03	0.13	55.0	56.4	21m 15s

Table 16: SCRUB baseline results on the **AwA2** dataset. Metrics include Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	91.7	0.00	0.99	0.04	47.0	53.6	21m 33s
A-Forget	92.5	0.00	1.00	0.04	49.7	51.6	22m 01s
H-Forget	91.9	0.00	0.99	0.06	48.2	50.3	18m 55s

Table 17: SCRUB baseline results on the **aPY** dataset. Metrics include Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	92.5	0.00	1.02	0.05	55.8	53.9	8m 22s
A-Forget	92.3	0.00	1.02	0.06	49.3	51.9	5m 42s
H-Forget	88.8	0.00	0.99	0.07	50.1	56.4	6m 43s

As reported in the tables above, SCRUB demonstrates a high degree of consistency throughout the unlearning process. Minor underperformances relative to the reference (golden) method are observed, primarily in scenarios involving fine-grained information. Overall, SCRUB achieves performance comparable to NegGrad+, which attains slightly higher retain accuracy but exhibits reduced effectiveness in mitigating privacy risks, as measured by membership inference attacks (MIAs).

SCRUB also exhibits unfavourable computational efficiency, however, as previously noted, runtime should be interpreted in the context of the dataset and experimental setup. A direct example of that can be noticed in how SCRUB requires only a single epoch to fully unlearn targeted information, consistently reducing forget accuracy to zero without oscillation. This strong impact of the unlearning, though causes higher oscillations in the retain accuracy, which led to the underperformance over CUB dataset. This behavior is clearly illustrated by the trends presented in fig. 22.



(a) SCRUB applied to the CUB dataset. Retain accuracy decreases more noticeably when unlearning larger amounts of fine-grained classes, with average-forget and high-forget scenarios highlighted in orange and pink, respectively. Forget Accuracy stays steady at 0



(b) SCRUB applied to the AwA2 dataset. Forget accuracy consistently reaches zero, while retain accuracy exhibits minor oscillations across epochs.



(c) SCRUB applied to the aPY dataset. The observed trend closely mirrors that of the AwA2 dataset, demonstrating consistent unlearning behavior across different data distributions.

Figure 22: SCRUB trends in Retain and Forget Accuracies.

3.9.6 Analysis of MU Baselines Results

The evaluation of MU baselines across the CUB, AwA2, and aPY datasets, commonly used for Zero-Shot Learning (ZSL) benchmarking, revealed several consistent patterns and notable challenges:

1. Across all methods, there was a shared difficulty in effectively unlearning the fine-grained CUB dataset, highlighting its intrinsic complexity compared to AwA2 and aPY. Despite this challenge, most methods exhibited relatively stable behaviors across the proposed forget scenarios.
2. Among the baselines, NegGrad+ and SCRUB demonstrated the most consistent and robust performance, effectively mitigating the effects of forgetting while maintaining stable behaviors as measured by the f-MIA and l-MIA metrics. These metrics are crucial for evaluating whether differential behaviors can be detected between training and non-training data after unlearning. In contrast, Finetuning and Bad Teacher showed highly variable and unreliable outcomes, often failing to correctly retain or forget information depending on the scenario.
3. A critical observation across all methods is the inherent dependency on iteratively accessing both retain and forget datasets. Excluding either dataset almost inevitably leads to unstable or unintended behaviors, particularly when evaluation involves Membership Inference Attacks (MIAs) targeting both the backbone and the classification head. This dependency underscores the fragility of existing MU baselines and their sensitivity to dataset-specific dynamics.

From this analysis, it is evident that a data-independent approach, which directly addresses unlearning at the level of network weights in both the backbone and classification head, offers several key advantages:

1. Mitigates the heavy data-dependence of current methods, which becomes particularly challenging when applied to real-world datasets containing fine-grained samples and large volumes of data.
2. Improves explainability by providing a direct interpretation of why specific parameters are modified, offering a more conceptual understanding of the roles individual neurons play in the network.
3. Ensures a more consistent and scalable approach that can be applied across diverse datasets with minimal performance variation, as most computations in deep neural networks for computer vision, both in classification and other tasks, are shared in the backbone, with only a few dataset-specific computations occurring to help differentiation in the classification head, as will be detailed in section 4

Building on the MU baseline evaluation, the ICIS method will be applied to the same datasets. The established evaluation framework, including f-MIA and l-MIA metrics and defined forget scenarios will be used to ensure a fair comparison. This will highlight the strengths and weaknesses of ICIS and guide the development of the proposed unified unlearning framework that will be discussed in chapter 4.

3.10 HyCUS: ICIS Adaptation to the Unlearning Task

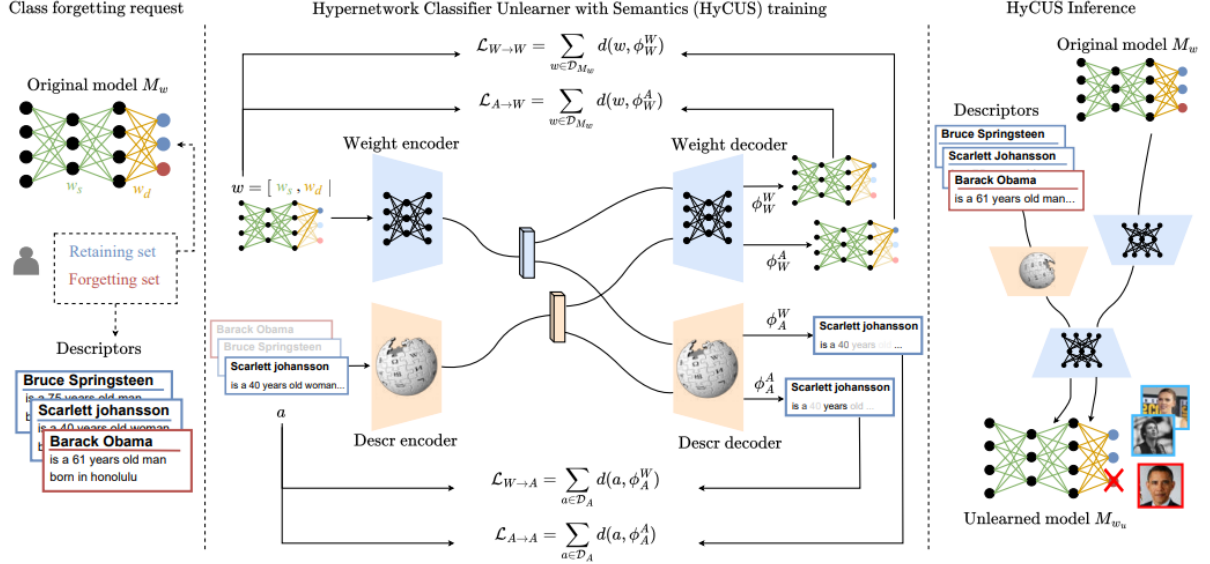


Figure 23: Architecture of HyCUS. The framework is derived from ICIS, with the main differences being the input to the joint autoencoder and the reconstruction process of the trained model. The trained model is divided into a shared component (green) and a distinct component (yellow), which are aligned to generate a w vector for the autoencoder.

The limitations of existing MU baselines, particularly their reliance on both retain and forget datasets, and their inconsistent performance across scenarios, highlight the need for a data-independent, parameter-level approach.

As discussed in earlier sections, such approaches are generally referred to as hypernetworks in the literature, with examples including the recently introduced DiHyFo [26]. However, a major limitation of most hypernetwork-based frameworks is their dependence on highly restrictive requirements, such as access to multiple training epochs of the original model, which is often unrealistic. The strength of ICIS, and consequently of HyCUS, lies in its post-hoc applicability, making it more practical in scenarios where such information is unavailable.

Building upon the efficiency and simplicity of the ICIS framework, HyCUS adopts the same architecture, based on a dual autoencoder with cross-domain mapping functions to align latent spaces (see fig. 23), while modifying the training strategy for a more task-specific and more intrusive approach to happen.

This approach, while extremely simple, lacks in adaptability as the backbone encoding requires a vectorial form that can be applied directly through linear layers. This brings to a core limitation as it has been almost mandatory to select weights of the most basic layers in the last computational block of the backbone, as using more complex ones would have led to sudden reductions, negatively impacting the learning of the latent space.

3.10.1 Applied Adaptations

While the overall architecture of the joint autoencoder has been maintained basically the same, for a more invasive approach to happen and allow a direct intervention to the backbone as well as to the classification head, HyCUS framework operates various substantial variations in the training pipeline:

(I). Dataset Preparation Instead of constructing the dataset solely from classification head weights paired with labels, HyCUS divides the network into two components: the shared part θ_s , corresponding to the backbone and common to all labels, and the distinct part θ_d^l , specific to each label $l \in Y_{seen}$. The resulting dataset is therefore defined as $D_{HyCUS} := \{(\theta_s, \theta_d^l, s_l), \forall l \in Y_{seen}\}$, given that the original model is trained on $D_s := \{(x, l) \mid l \in Y_{seen}\}$. The semantic embeddings s_l of a label are still acquired using similar methodologies to ICIS. In the case of this thesis work, they are derived from BERT’s application to the standard name of the label according to Wikipedia to ensure that naming conventions couldn’t alter the produced embedding, and that it is derived from an enriched descriptive context.

(II). Training Phase. During each training step, a label is extracted from the dataset, together with the relative triplet $(\theta_s, \theta_d^l, s_l)$. The weights latent space is then obtained by the application of the encoder to either:

- a version of θ_s perturbed with gaussian noise, concatenated with the $\theta_d^{l_r}$ of a random label selected among the labels $l_r \in Y_{retain}$, if $l \in Y_{forget}$.
- the concatenation of θ_s and θ_d^l as they are, if $l \in Y_{retain}$.

Finally, the weight decoding process, influenced by the semantic autoencoder in an identical way as it happened in ICIS, is obtained and compared to the altered triplet to obtain the L2-loss.

(III). Injection Phase. At the end of the training, the framework produces one version of the network for each label. While this is suitable for modifying classification heads, the shared part must remain unique. To achieve this, all generated shared weight sets are aggregated through an element-wise mean, as the ablation study of previous works on this framework clearly showed better performances using this aggregation type. The reconstructed weights are then injected directly into the original model, which is subsequently evaluated under the same conditions as other baselines in order to assess unlearning quality, strengths, and limitations.

3.10.2 Evaluations and Core Considerations

For the experiments evaluation to be consistent and fair with the baselines previously analyzed, the same evaluation metrics are directly applied to the origin, injected with the HyCUS-generated weights, while the runtime indicates the complessive time required for both training and injection to happen. Similarly to other methods, the results are reported in three distinct tables: 18 to 20

Table 18: HyCUS results on the **CUB** dataset. Metrics include Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	72.5	0.00	1.01	0.13	66.2	83.8	28m 51s
A-Forget	71.5	0.00	0.99	0.15	59.4	80.9	48m 39s
H-Forget	73.3	0.00	1.01	0.18	64.7	80.3	47m 39s

Table 19: HyCUS results on the **AwA2** dataset. Metrics include Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	93.0	0.00	1.00	0.03	50.8	54.1	17m 37s
A-Forget	93.9	0.00	1.01	0.04	55.5	55.2	14m 08s
H-Forget	93.8	0.00	1.02	0.09	52.6	52.9	21m 13s

Table 20: HyCUS results on the **aPY** dataset. Metrics include Retain Accuracy, Forget Accuracy, Adaptive Unlearning Score (AUS), JSD, l-MIA, f-MIA, and Runtime.

Scenario	$Acc_r(\uparrow)$	$Acc_f(\downarrow)$	AUS ($\uparrow$)	JSD ($\downarrow$)	l-MIA (50%)	f-MIA (50%)	$Time_{run}(\downarrow)$
L-Forget	93.4	0.00	1.03	0.04	59.3	66.2	28m 19s
A-Forget	93.9	0.00	1.03	0.07	60.7	62.6	41m 44s
H-Forget	91.1	0.00	1.02	0.09	58.5	59.5	16m 16s

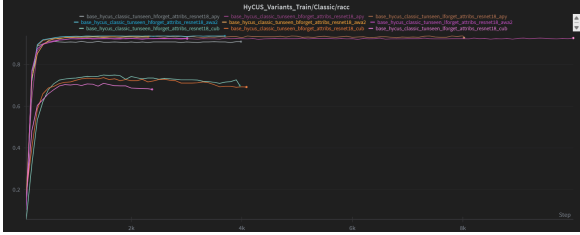
A preliminary analysis of the results reveals both strengths and weaknesses of the current framework:

Strengths:

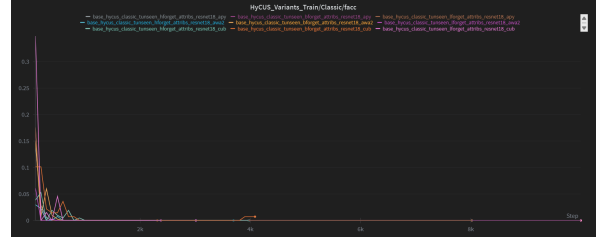
1. The framework demonstrates highly consistent and robust performance across all datasets and scenarios, underscoring the advantages of its data-independent nature.
2. By integrating the charts in fig. 24, HyCUS exhibits a much higher step efficiency as the time metric refers to more than 6k epochs.
3. Membership Inference Attack (MIA) vulnerability is consistently reduced when monitoring loss values, in line with the JSD metric. For instance, on the CUB dataset, l-MIA decreases from 72.3 to a maximum of 66.2, approaching the performance of the strongest baseline.

Weaknesses:

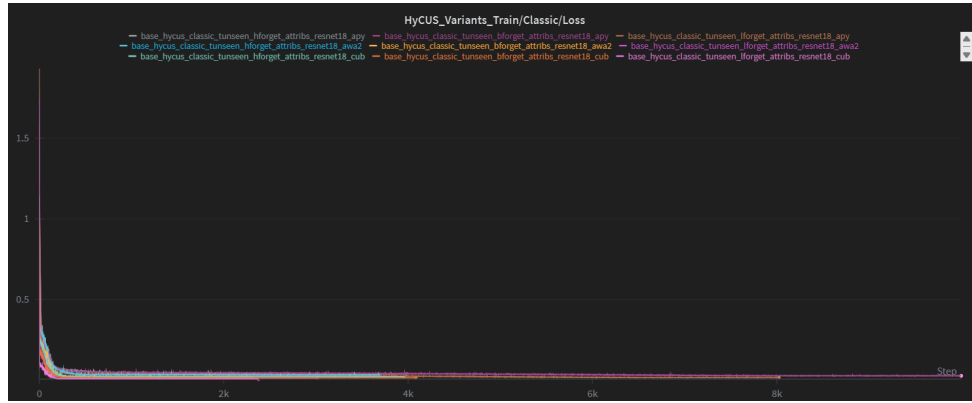
1. While effective in addressing unlearning in regressors, the method exhibits a tendency to increase vulnerability to f-MIAs (e.g., in CUB $83.2 \rightarrow 83.8$) and provides limited improvements in aPY ($68.3 \rightarrow 66.2$), revealing a nature to function largely as an output suppression strategy rather than a proper unlearning method.
2. The framework suffers from poor scalability, as the weight encoder requires $N_\theta \times L$ parameters, where N_θ is the total number of shared and distinct weights and L the latent space dimensionality, making it impractical for modifying deeper backbone layers.
3. The need to overfit on the forget set necessitates a large amount of training steps and limits adaptability to new forget sets, requiring retraining whenever new labels must be forgotten.
4. The absence of reliable early-stopping criteria hinders training efficiency, as forgettance cannot be directly inferred from $Acc_r - Acc_f$ as in other methods.



(a) Retain accuracy across datasets and scenarios.



(b) Forget accuracy across datasets and scenarios.



(c) Reconstruction loss of the weights under the proposed training strategy.

Figure 24: Training dynamics of HyCUS across benchmark datasets and scenarios presented in section 2.

3.10.3 Need For A Deeper Analysis

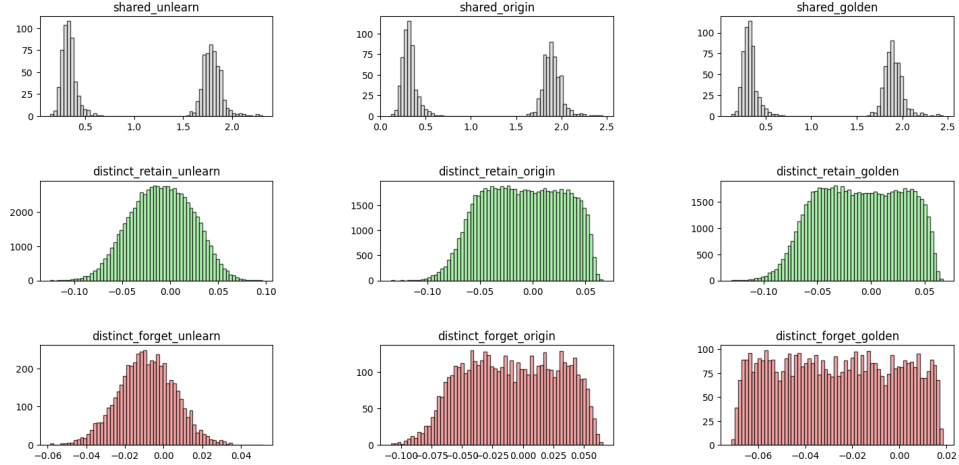
The results indicate that, although the proposed unlearning strategy functions consistently across different setups, its influence remains too localized to completely eliminate the effect of forgotten samples. In practice, HyCUS only provides a partial suppression of outputs rather than a true removal of learned information.

This limitation becomes evident when comparing the weights generated by HyCUS with those of both the original model and the golden approach. Taking CUB as a reference case, where HyCUS demonstrated the greatest difficulty in removing learned information, it is clear that restricting the method to a single and simple layer (namely, the batch normalization layer preceding average pooling in ResNet18) is insufficient. Furthermore, when comparing the weight distribution of the golden model against that of the origin, the two appear almost identical, suggesting that this layer exerts limited influence in the unlearning process (fig. 25). The results also reveal a tendency of HyCUS to randomize distinct weights not only for the forget labels but also for the retain labels, thereby reducing its effectiveness and its ability to preserve useful representations.

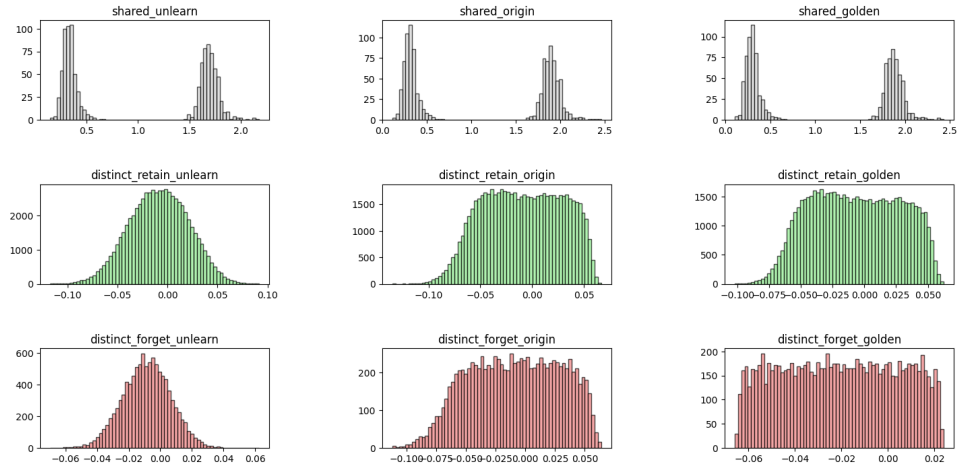
A deeper reflection on these weaknesses highlights a more fundamental obstacle: the black-box nature of standard deep learning models. Once trained, the backbone parameters encode information in a highly intertwined manner, making it nearly impossible to isolate which subsets of weights are directly responsible for the differential treatment of forget versus retain samples. As a consequence, even if HyCUS were extended to operate on deeper or alternative layers, it would be impossible to identify which parameters in the backbone are dataset-specific rather than generic computation common for all integrated samples.

These limitations severely impair HyCUS ability to be extended beyond a simple suppression method and thereby poses a necessity for a more advanced and scalable framework.

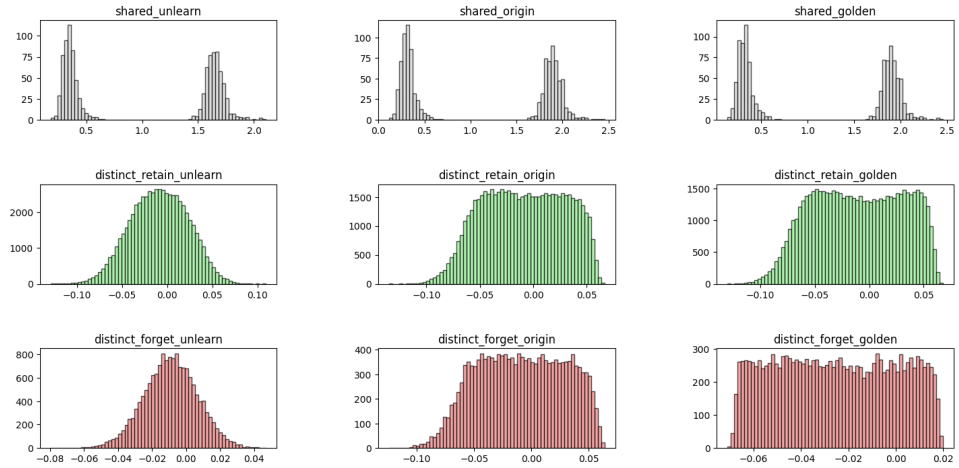
This realization will serve as one of the central motivations for the next stage of this thesis: instead of seeking solutions directly in the parameter space, the following chapter will introduce a new framework that operates within a higher-order latent space, defined by the authors as *hyper-representation space*. This space is explicitly structured to simplify a network to a set of connected embeddings, where multiple instances of a model, or even multiple different architectures, can coexist. As will be shown, such a representation makes it possible to identify the components directly responsible for dataset-specific computations as well as task specific ones, effectively shifting from a parameter-level manipulation to a representation-level control.



(a) low-forget scenario analysis.



(b) average-forget scenario analysis.



(c) high-forget scenario analysis.

Figure 25: HyCUS-generated weights (left) compared to the origin (center) and the golden approach (right) on CUB dataset.

3.10.4 Conclusive Analysis

In summary, while HyCUS demonstrates the feasibility of post-hoc unlearning through a dual-autoencoder strategy, its reliance on modifications confined to a single layer, together with the tendency to randomize weights across both forget and retain classes, considerably limits its overall effectiveness. These findings suggest that unlearning cannot be reliably achieved through localized interventions alone; rather, it necessitates architectural adaptations capable of propagating influence more broadly and consistently throughout the network backbone. Accordingly, these limitations provide a clear motivation for exploring a more flexible and scalable framework.

Building on this motivation, the following section introduces a more advanced autoencoder-based architecture: SANE [27], which can simultaneously learn multiple versions of the original network and allow the creation of an hyper-representation space, also giving valuable insights in why HyCUS failed in unlearning. This section examines the method in detail and considers how the architecture can be adapted into a general generative framework, capable of constructing an effective neuron-centric latent representation of the network.

In addition, a theoretical discussion of the weight-as-data paradigm is presented, highlighting how these principles can enhance latent space learning and enable applications that extend beyond the original scope of HyCUS’s joint autoencoder.

This perspective provides a deeper understanding of the weight space itself, offering a conceptual framework for reinterpreting the training of deep neural networks. By viewing network weights as structured, analyzable data rather than merely as parameters of a black-box function, this approach facilitates more interpretable, explainable, and theoretically grounded insights into the organization and behavior of the network as a whole.

Finally, a comprehensive overview of both the strengths and limitations of the established framework will be provided, establishing a baseline for the subsequent analysis of the thesis’s contributions and for identifying directions for future research.

4 The Weight-as-Data Framework

Across the domains considered in this thesis, a common element is the use of a pre-trained network that performs effectively on its designated task. The following experiments explore how the information encoded in the learned weights can be repurposed by interpreting the parameters as data, thereby enabling a deep learning model to undertake a generic weight-space learning task.

4.1 Introduction to the Weight Space

The weight space of a neural network is the high-dimensional space defined by all possible values of its trainable parameters. Each point in this space corresponds to a different version of the model, which determines how inputs are processed and how the network performs on a given task. Since modern architectures contain millions of parameters, this space is extremely large, therefore, understanding its structure is essential to explain how models learn, generalize, and react to perturbations.

4.1.1 Weight Space Properties and Symmetrical Redundancies

A key feature of the weight space is the presence of symmetries and redundancies. These properties mean that several different parameter configurations can represent models with the same behavior. Common examples include:

- **Neuron permutation:** neurons within a hidden layer can be swapped, together with their incoming and outgoing connections, without changing the function of the network.
- **Weight permutation:** the order of the weights connected to a neuron can be rearranged, as long as the corresponding inputs are permuted in the same way, leaving the neuron’s output unchanged.
- **Weight rescaling:** multiplying the weights of one layer by a constant and dividing the weights of the next layer by the same constant gives the same computation.
- **Layer decomposition or compression:** a single layer can be split into multiple equivalent layers, or several layers can be merged into one, while still representing the same function.

These properties imply that weight space contains many equivalent solutions, which can be exploited to enable different weight-space data augmentation techniques: by sampling from or moving between equivalent configurations, it is possible to generate multiple models that solve the same task, which can improve generalization, but can also be useful to learn meta-information about the weight-space. For example, neuron permutation can help with model compression and parameter sharing; layer decomposition can enable low-rank approximations; and weight rescaling can improve initialization and normalization.

4.1.2 Geometry of the Loss Landscape

The geometry of the loss landscape in weight space strongly influences optimization and generalization. Several studies show that the regions of the loss landscape where solutions are found affect how well models generalize. For example, Keskar et al. [16] demonstrate that training with large batch sizes can lead to *flat minima* (regions where small parameter changes cause only small variations in loss). These solutions tend to generalize better than those in *sharp minima* as in those areas small perturbations would lead to higher variations in the loss function. In addition, different minima discovered in independent training runs are often connected by low-loss paths, a property known as *mode connectivity* [11].

These observations suggest that weight space is not made of isolated solutions but of large connected regions of good minima. This idea will be used in later experiments, where augmentation in weight space will help create several versions of the trained *origins* (introduced in Chapter 2) that perform similarly.

4.1.3 Standard Deep Learning in Weight Space

Training in deep learning can be seen as a guided search through weight space. Gradients computed from a loss function show directions that reduce error, and optimization algorithms such as stochastic gradient descent (SGD) update the parameters along these directions. Through this process, the model gradually moves across the weight space until it reaches a configuration that approximates the ground truth as closely as possible.

This optimization process is effective but can also lead to overfitting: since gradients are calculated to reduce error on the training set, the model may adapt too closely to the training samples, losing its ability to generalize. This over-memorization of the model can lead to privacy vulnerabilities that can be exploited with aimed attacks like the MIA ones discussed in the previous chapter.

To prevent overfitting and improve generalization, several regularization techniques have been developed. Classical examples include dropout and weight decay, while more advanced methods include Sharpness-Aware Minimization (SAM) [10] and Stochastic Weight Averaging (SWA) [15]. These methods make direct use of the geometry of weight space, steering optimization away from sharp minima.

Other approaches, instead, directly modify the gradients: in the example given in Mori et al.'s work [20], higher stochastic noise is added to standard SGD updates, leading to better generalization during model training. This acts similarly to injecting small Gaussian perturbations into the parameters of a trained model, which corresponds to explore the immediate neighborhood in the weight space and exploit mode connectivity to obtain more robust solutions with less overfitting.

4.2 SANE Framework [27]

Building upon the previous considerations, the **Sequential Autoencoder for Neural Embeddings (SANE)** will be analyzed, as a framework designed to compute hyper-representations of neural networks by encoding them as sequences of tokens rather than relying on their full weight vectors. This formulation enables SANE to: (i.) reduce the parameters space to an higher level representation of it; (ii.) introduce the computation of multiple different architectures within the same space; (iii.) allowing the computation of different hyper-representations adaptable to the task that has to be solved, through the usage of transformer blocks.

By decoupling training from both model size and architecture, SANE provides the flexibility to integrate multiple versions of different models. This capability facilitates the learning of both discriminative and generative downstream tasks without requiring access to image data.

Through the use of a combination of transformers and autoencoders, the framework can, in fact, be trained to learn different task-aware hyper-representations z of the various models, and further exploit them to compute projection heads adaptable to different discriminative tasks. This design allows efficient adaptation to downstream tasks while maintaining independence from the hyper-representation space, thereby enhancing SANE’s scalability and versatility in weight-based learning.

Furthermore, its autoencoder-based design is compatible with the joint autoencoder proposed in [5]. This compatibility enables extensions toward conditioned generative approaches, supporting the simultaneous learning of multiple downstream tasks such as zero-shot learning (ZSL) and machine unlearning (MU). The semantic autoencoder of ICIS can in fact be interpreted as a way to affect the generative process, which guides the learning of the latent space. When transposed to SANE, this allows to affect the attention system within the transformer blocks, effectively changing the learned hyper-representation space and allowing it to encode different aspects of the analyzed networks.

4.2.1 Architecture

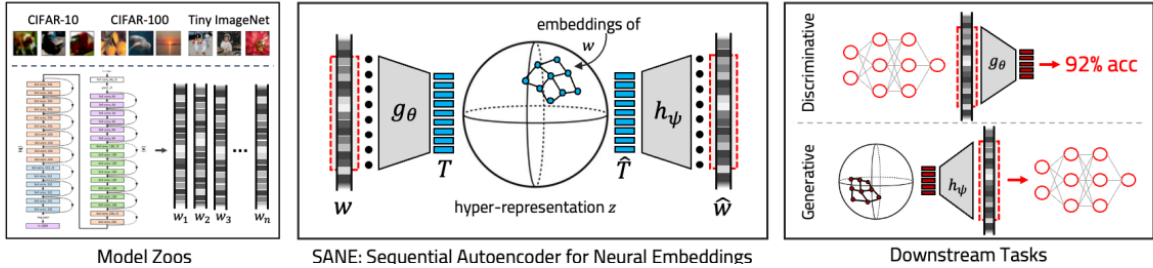


Figure 26: SANE architecture. Model zoos are transformed into weight vectors, which are then decomposed in tokens of fixed dimensions and parsed through a transformer encoder g_θ giving the hyper-representation z of a neural network as a sequence of embeddings. The hyper-representation is then used to learn either discriminative or generative tasks through the usage of a further autoencoder that produces the projection heads.

The architecture of SANE is centered around a sequential autoencoder that operates on tokenized representations of neural network weights. Instead of treating an entire weight vector as a monolithic input, the model decomposes network parameters into sequences of tokens, enabling the application of sequence-based learning methods analogous to those used in natural language processing, with some minor changes relevant for the neural networks context.

- **Encoder.** The encoder processes fixed-size windows of n tokens, transforming them into a compact latent representation, referred to as the hyper-representation z . In this reduced-dimensional space, each window corresponds to a network embedding, designed to capture both structural and functional properties of the original model. Importantly, this formulation remains independent of the base architecture or model size, provided that the network can be expressed as a sequence of n tokens.

The encoder block is implemented as a GPT-2 style transformer, inspired by Karpathy’s `nanoGPT` implementation². This transformer is followed by a linear layer that further reduces the dimensionality of the hyper-representation space. Finally, an additional dimensionality-reduction layer projects the z vector into an even lower-dimensional space, producing the projection head ϕ_z that serves as the input for downstream tasks

- **Decoder.** The decoder reconstructs the original weight sequence either from the projection head ϕ_z or directly from the hyper-representation z . Its architecture mirrors the encoder in reverse: a series of linear layers first expand the latent representation into a dimension compatible with the transformer blocks, after which a final linear layer restores each embedding to the original $window_size \times token_size$ vector.
- **Positional Embedding.** As discussed in section 3.5.1, positional embeddings are required to make transformer models sensitive to the ordering of tokens, enabling both position-aware and context-aware representations. Within the SANE framework, positions are represented as 3D vectors: the first dimension uniquely indexes each token; the second dimension identifies the corresponding layer, which is also used for the model reconstruction; and the third dimension specifies the token’s position within a layer. The final positional embedding is obtained by summing the embeddings of the first two dimensions and concatenating this result with the embedding of the third dimension:

$$pe_1 = posemb(p[:, :, 0]), \quad pe_2 = posemb(p[:, :, 1]), \quad pe_3 = posemb(p[:, :, 2]),$$

$$pe = [pe_1 + pe_2, pe_3].$$

²<https://github.com/karpathy/nanoGPT/blob/master/model.py>

4.2.2 Data Preprocessing

To enable the framework to be trained across multiple downstream tasks, the process is divided into two phases: a *pre-training* stage, where multiple architectures are trained for several epochs on a given task, and a *downstream learning* stage, which leverages meta-information about each model (e.g., accuracy scores) to support performance prediction and task adaptation.

The potential set of architectures that could be incorporated into this framework, referred by the authors as *models zoos*, is extremely large, and may include checkpoints from various convolutional neural networks (CNNs) trained for different numbers of epochs on classification tasks such as CIFAR-10, CIFAR-100, and Tiny ImageNet, as well as more complex network zoos like ResNet-based architectures.

However, directly combining such a large collection of models introduces several challenges. In particular, checkpoints may differ in layer naming conventions, parameter scales and permutations, and other structural aspects, which complicates their joint utilization. To address these issues, the model matching and merging techniques of Ainsworth et al. [2], based on rebasin theory, are applied. These techniques align all models to a common reference structure and subsequently normalize their weights, thereby providing a consistent representation suitable for training within the framework.

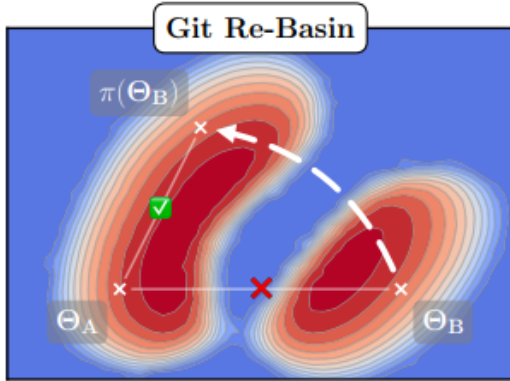


Figure 27: Example of valid VS invalid operations to transpose a model’s basin to a reference one, following [2] work

The method proposes algorithms to find the optimal permutation that aligns the weights of two models. This is done by minimizing the distance between the permuted weights of one model and the weights of the other. The algorithms essentially "teleport" each model’s solution into the same basin as the reference by exploiting the mode connectivity property of models trained on the same dataset.

Once all the versions of each model in the zoo are standardized and conducted to a base reference, each model is tokenized either entirely or by selectively choosing which layers to exclude (for example excluding batch normalization layers rather than downsample layers). This process is necessary to match different epochs of the same architecture to be tokenized with same positional encodings, so that a token can be stored in different evolutions during the training process.

Tokenization process is then achieved by iteratively aligning each layer into a single vector, which is further splitted it into $\frac{layer_dim}{token_size}$ tokens, zero-padding the incomplete ones and

constructing a mask to recognize which positions correspond to padding. A dataset is hence constructed with those preprocessed tokens, with specific functions to randomly extract any number of windows from each architecture and version computed.

4.2.3 Training Process

The training of SANE is organized in two stages: (i) a pre-training phase, during which both the hyper-representation space and the projection space are learned, and (ii) a secondary downstream learning phase, where task-specific objectives, either discriminative or generative, are optimized.

During pre-training, the optimization is primarily driven by two complementary loss functions:

Masked Reconstruction Loss. This objective leverages a padding mask to compute L2 reconstruction errors only over the non-padded elements of the input, ensuring that the autoencoder faithfully reconstructs the original weight sequences.

Normalized Temperature-scaled Cross-Entropy (NT-Xent [31]). This contrastive loss relies on two augmented views derived from the same original representation. Its objective is to align corresponding features across the views while pushing apart non-matching ones. Concretely, cosine similarity is used as a feature-level similarity metric, which is then combined with the ground-truth distribution to compute a cross-entropy loss. Ideally, the cosine similarity between corresponding features should approach 1, while all other similarities should approach 0. Self-similarities are excluded from both alignment and separation terms. The loss is therefore defined as:

$$\begin{aligned}\mathcal{L}^{\text{NT-Xent}} &= -\frac{1}{N} \sum_{i,j \in \mathcal{MB}} \left(\frac{\text{sim}(z_i, z_j)}{\tau} - \log \sum_{k=1}^{2N} \mathbb{1}_{[k \neq i,j]} \exp \left(\frac{\text{sim}(z_i, z_k)}{\tau} \right) \right), \\ &= \underbrace{-\frac{1}{N} \sum_{i,j \in \mathcal{MB}} \frac{\text{sim}(z_i, z_j)}{\tau}}_{\mathcal{L}_{\text{alignment}}} + \underbrace{\frac{1}{N} \sum_i \log \left(\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i,j]} \exp \left(\frac{\text{sim}(z_i, z_k)}{\tau} \right) \right)}_{\mathcal{L}_{\text{distribution}}},\end{aligned}$$

where $\text{sim}(z_i, z_j)$ denotes cosine similarity between two views of a projected hyper-representation ϕ_z , τ is a temperature parameter that controls the softness of the probability distribution, and $\mathcal{MB} = \{\tilde{x}_{1i}, \tilde{x}_{1j}, \dots, \tilde{x}_{Ni}, \tilde{x}_{Nj}\}$ with $|\mathcal{MB}| = 2N$ defines the set of paired features.

At each training step, two indices are used to traverse the dataset of tokenized networks: index i iterates over different architectures in the model zoo, while index j iterates over multiple versions (epochs) of the same architecture. For each i , several windows are randomly sampled across the different j versions, and each window w is processed through the following steps:

- compute the hyper-representation z of w ;
- compute the projection ϕ_z from z ;

- derive two noise-augmented views ϕ_{z_1} , ϕ_{z_2} from ϕ_z ;
- reconstruct the window $\hat{w}$ from z ;
- evaluate the reconstruction and contrastive losses described above.

Finally, in the downstream learning phase, the learned projection space can be combined with meta-information about model versions to solve specific discriminative or generative tasks, with an example of a discriminative task being the prediction of a model’s instance performances.

In [27], some results on discriminative tasks are provided both for small generic CNNs and ResNet architectures, showing the vast applicability of the method discussed. Those tables are here reported for completeness of informative purposes:

	MNIST			SVHN			CIFAR-10 (CNN)		
	W	s(W)	SANE	W	s(W)	SANE	W	s(W)	SANE
Acc.	0.965	0.987	0.978	0.910	0.985	0.991	-7.580	0.965	0.885
Ep.	0.953	0.974	0.958	0.833	0.953	0.930	0.636	0.923	0.771
GGAP	0.246	0.393	0.402	0.479	0.711	0.760	0.324	0.909	0.772

Table 21: Results of property prediction on populations of small Convolutional Neural Networks (CNNs). A regression task was performed to predict: (i) test set prediction accuracies (*Acc*); (ii) training epoch (*Ep.*); and (iii) generalization gap (*Ggap*). Three distinct methodologies were employed for this prediction: (i) raw model weights (*W*); (ii) statistical properties of model weights (*s(W)*); and (iii) SANE embeddings.

	CIFAR-10		CIFAR-100		TINY-IMAGENET	
	s(W)	SANE	s(W)	SANE	s(W)	SANE
Acc.	0.880	0.879	0.923	0.922	0.802	0.795
Ep.	0.999	0.999	0.999	0.992	0.999	0.980
GGAP	0.490	0.512	0.882	0.879	0.704	0.699

Table 22: Results of property prediction on populations of ResNet-like architectures. A regression task was performed to predict: (i) test set prediction accuracies (*Acc*); (ii) training epoch (*Ep.*); and (iii) generalization gap (*Ggap*). The prediction methodologies employed were: (i) statistical properties of model weights (*s(W)*); and (ii) SANE embeddings.

4.3 Adaptation to Generative Tasks

In line with the objectives of this thesis, the original SANE autoencoder architecture was simplified and adapted to support a more general weight-learning task across multiple datasets. Conventionally, the SANE framework is applied to models trained on the same dataset, where multiple epochs are exploited to learn robust weight hyper-representations in a very similar way of [23] work.

While this approach is well suited for the original scope of such frameworks, to simplify the scope of the thesis, a different goal is pursued: the development of post-hoc, image-free generative autoencoder that can be used as a support for the pre-trained model, and that can be adapted to multiple generative downstream tasks, such as MU and ZSL.

To this end, the SANE training process has been reformulated as a generic weight-learning pre-training stage, where projection heads and contrastive objectives were unnecessary and were thereby excluded from the network’s architecture. Furthermore, given the post-hoc setting, alternative augmentation strategies were required to enhance pre-training, as the unavailability of multiple epochs is essential for realistically applying that hypothesis.

4.3.1 Preprocessing and Training Process

A key modification derives from the assumption that only pretrained model origins (as discussed in Chapter 2) are available. To accommodate this, both the tokenization and reconstruction processes were adapted by introducing a moving-window mechanism, enabling the framework to process entire models consistently.

This design introduces an additional constraint: origins often differ in size, leading to windows of varying lengths. To standardize processing across models and datasets, incomplete windows are padded with zero-filled tokens, and to prevent these artificial elements from influencing reconstruction, they are assigned a synthetic layer index in the positional embedding. This ensures that the tokenizer excludes them from both influencing the produced embeddings and affecting the reconstruction process.

This tokenization process produces a dataset of triplets (t, p, m) , where t denotes the token sequence, p represents the 3D positional information defined in the SANE framework, and m is the corresponding padding mask. By construction, t and m share the same token dimension, while p encodes the positional embedding for each token. For efficient integration with PyTorch’s `Dataset` and `DataLoader` interfaces, windows are initialized as sets of possible starting indices, where the final index is computed as `min(window_size, last_dataset_index)`, and incomplete windows are padded accordingly to the strategy described earlier.

This setup enables a first form of data augmentation for the transformer-based autoencoder: by varying the window stride, it is possible to alter the token combinations included in each extracted window. This allows the model to both revisit the same tokens across multiple training steps, consequently increasing the number of iterations per token, and compute self-attention over different token groupings, thus, enriching contextual interactions.

4.3.2 Single Origin Experiments

Treating the stride as a tunable hyperparameter enables a series of experiments designed to investigate its influence on learning dynamics, as well as the effect of architectural simplifications introduced to mitigate the computational constraints of the infrastructure used in this thesis. Using the AwA2 origin as the experimental baseline, the results obtained are summarized in the table below.

Table 23: Experimental results across different stride (sws=4 means $stride = \frac{window_size}{4}$), number of transformer blocks (b), embedding size (e), and number of attention heads (h) configurations. Origin accuracy and F1 score remain constant, while reconstructed values vary per setup.

sws	b	e	h	Origin Acc.	Reconst. Acc.	Origin F1	Reconst. F1
1	1	256	1	92.60	4.58	90.20	12.16
1	1	256	2	92.60	3.29	90.20	4.53
1	1	512	1	92.60	1.76	90.20	5.11
1	1	512	2	92.60	6.28	90.20	12.56
1	2	256	1	92.60	2.40	90.20	4.01
1	2	256	2	92.60	2.99	90.20	7.89
1	2	512	1	92.60	15.15	90.20	23.64
1	2	512	2	92.60	18.13	90.20	26.68
2	1	256	1	92.60	28.45	90.20	36.91
2	1	256	2	92.60	65.35	90.20	66.61
2	1	512	1	92.60	75.15	90.20	76.69
2	1	512	2	92.60	84.44	90.20	82.64
2	2	256	1	92.60	71.89	90.20	72.11
2	2	256	2	92.60	75.77	90.20	74.29
2	2	512	1	92.60	2.33	90.20	4.73
2	2	512	2	92.60	63.85	90.20	75.83
4	1	256	1	92.60	88.60	90.20	86.53
4	1	256	2	92.60	87.18	90.20	84.32
4	1	512	1	92.60	92.14	90.20	89.75
4	1	512	2	92.60	91.76	90.20	89.60
4	2	256	1	92.60	89.97	90.20	87.63
4	2	256	2	92.60	89.86	90.20	87.66
4	2	512	1	92.60	87.75	90.20	85.37
4	2	512	2	92.60	91.96	90.20	89.70

From table 23, a consistent trend emerges: increasing the number of steps leads to improved performance. Furthermore, a higher number of windows, obtained by reducing the window stride, facilitates learning considerably. By contrast, the number of transformer blocks and attention heads appears to have little influence on the reconstruction process.

To better understand which factors most strongly affect the training process, two complementary analyses were conducted. Specifically, the distribution of the Mean Squared Error (MSE) between the reconstructed model weights and the original weights was evaluated with respect to both the type of layer and the layer dimensionality:

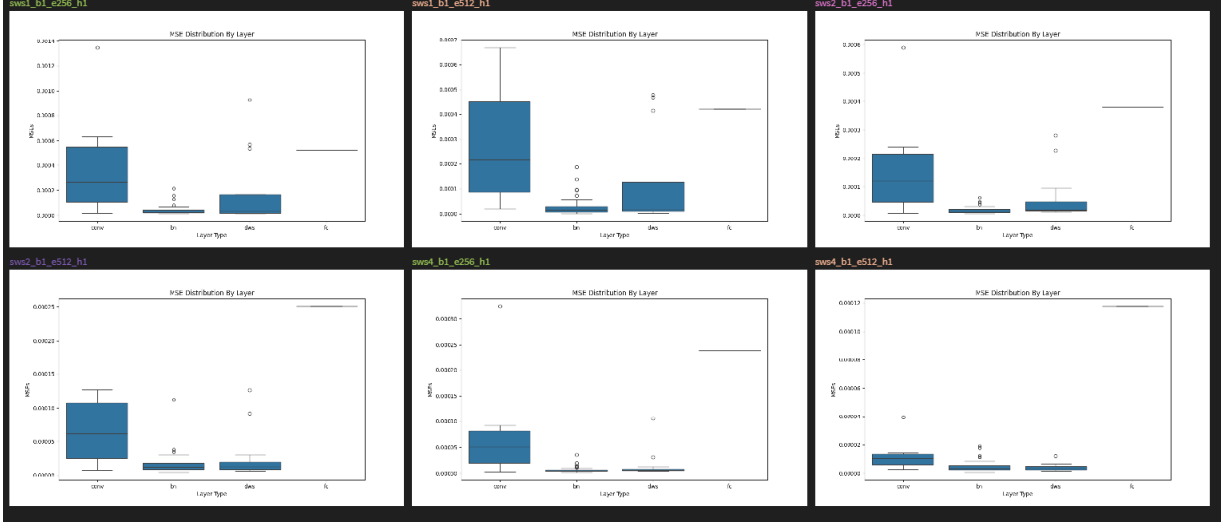


Figure 28: Distribution of MSE values across different layer types in the ResNet18 architecture. Results are reported only for configurations with $b = 1$ and $h = 1$, as transformer block count and attention heads were found to be negligible factors in table 23.

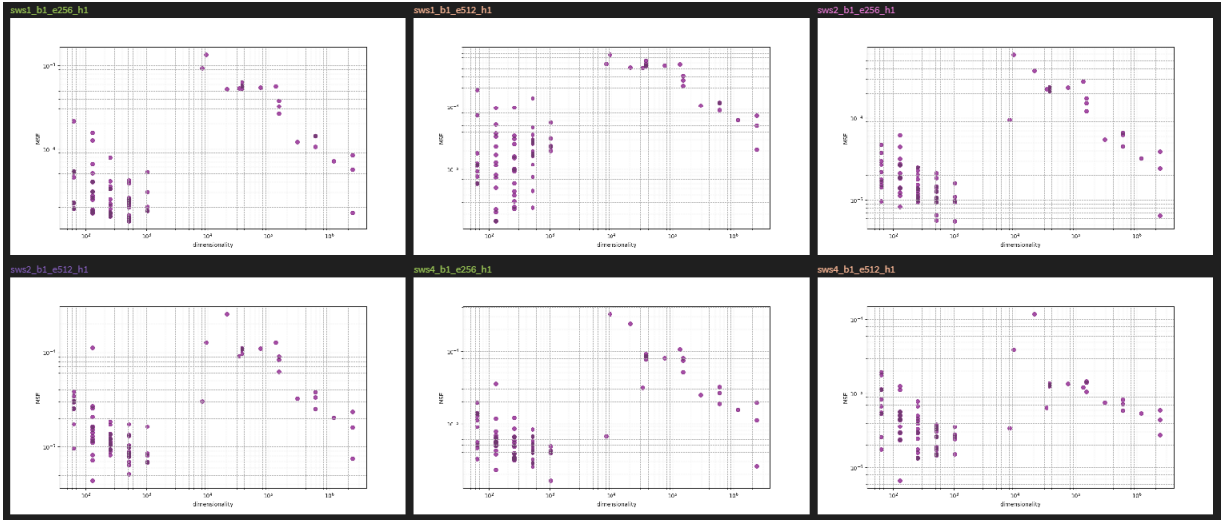


Figure 29: Scatterplot of MSE values (y-axis) as a function of layer dimensionality (x-axis) in the ResNet18 architecture. Results are reported only for configurations with $b = 1$ and $h = 1$, as transformer block count and attention heads were found to be negligible factors in table 23.

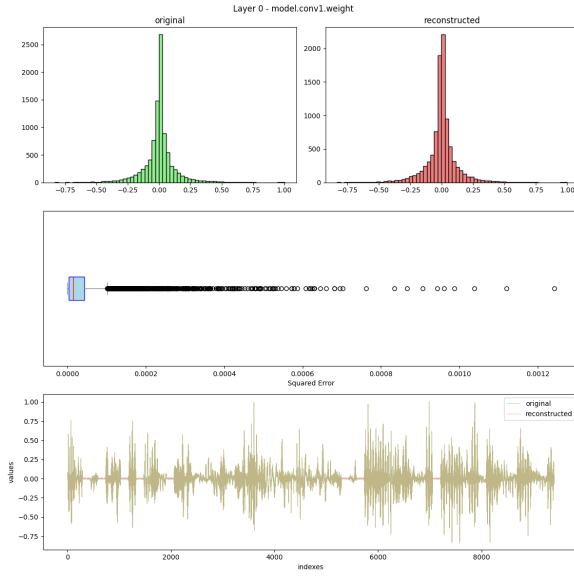
From fig. 29, it can be observed that lower-dimensional layers are generally easier to reconstruct, although no strict correlation with dimensionality is evident. In contrast, fig. 28 reveals a clearer pattern: convolutional layers and the fully connected (FC) classification head consistently yield higher MSE values compared to batch normalization and down-sampling layers.

This indicates a link between layer complexity and reconstruction difficulty. However, under this guideline, the FC layer would be expected to be easier to reconstruct than convolutional layers. Instead, in the most challenging and best-performing scenario (with $sws = 4$, $e = 512$), convolutional layers exhibit substantially lower MSEs than the FC layer.

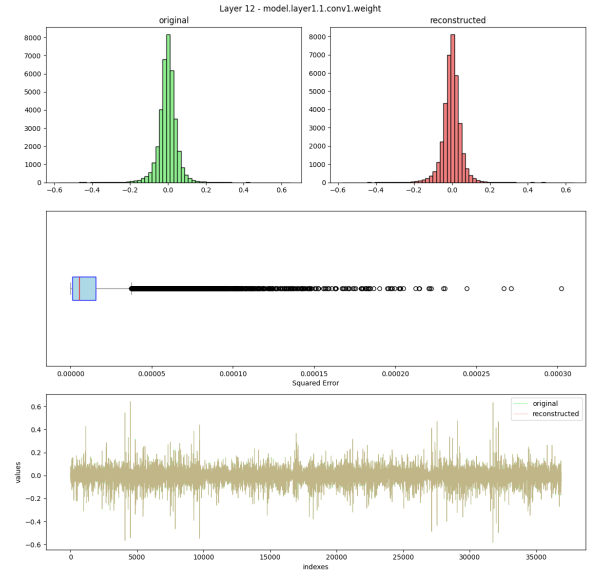
An additional anomaly arises in table 23, where the configuration $sws = 2$, $b = 2$, $e = 512$, $h = 1$ shows a pronounced performance drop, behaving as a clear outlier. Further inspection revealed that in this case, the model tends to produce very small but negative values for the running variance parameters of downsampling and batch normalization layers. This leads to invalid square-root operations and the emergence of a "ghost layer" (a layer that consistently outputs empty activations), resulting in fixed, data-independent predictions.

To mitigate this issue, a lower bound of zero was imposed on reconstructed running variance values. To further investigate the divergence observed in the FC layer, instead, a more detailed examination of the reconstruction process was carried out.

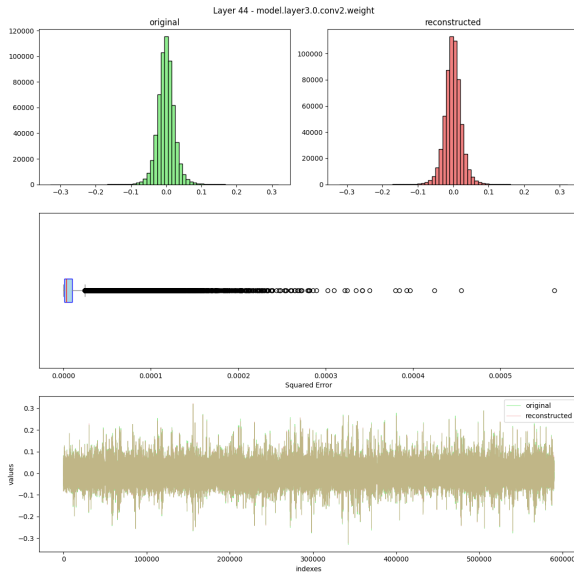
The results, summarized in fig. 30, demonstrate that for most convolutional layers, both in early and late stages, the reconstructed weight distributions are nearly identical to the original ones. This is reflected in the low MSE values and strong value-wise alignment (bottom plots in each group). In contrast, the FC classification head (fig. 31) displays a distribution that, although improving, still resembles a random Gaussian. This suggests that the number of tokens associated with the FC head is considerably lower than that of convolutional layers, implying that the framework implicitly prioritizes convolutional tokens. Consequently, the dataset is imbalanced, with an overrepresentation of convolutional tokens relative to classification tokens.



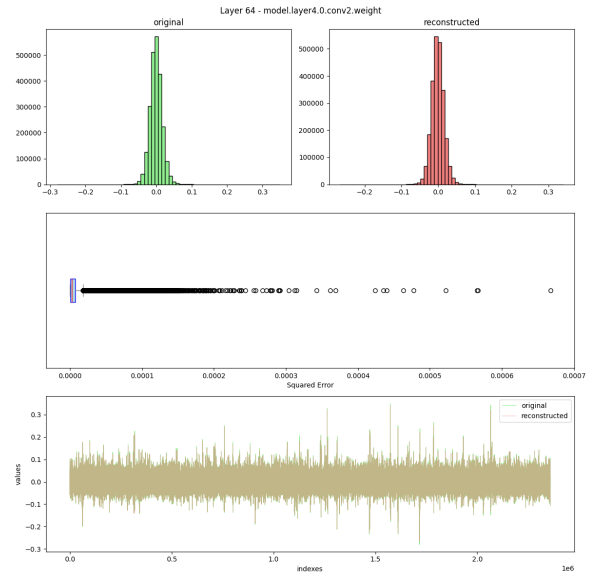
(a) Input convolutional layer.



(b) First convolution in Layer 1.



(c) First convolution in Layer 3.



(d) First convolution in Layer 4.

Figure 30: Comparison of reconstructed and original weight distributions for convolutional layers across different stages of ResNet18.

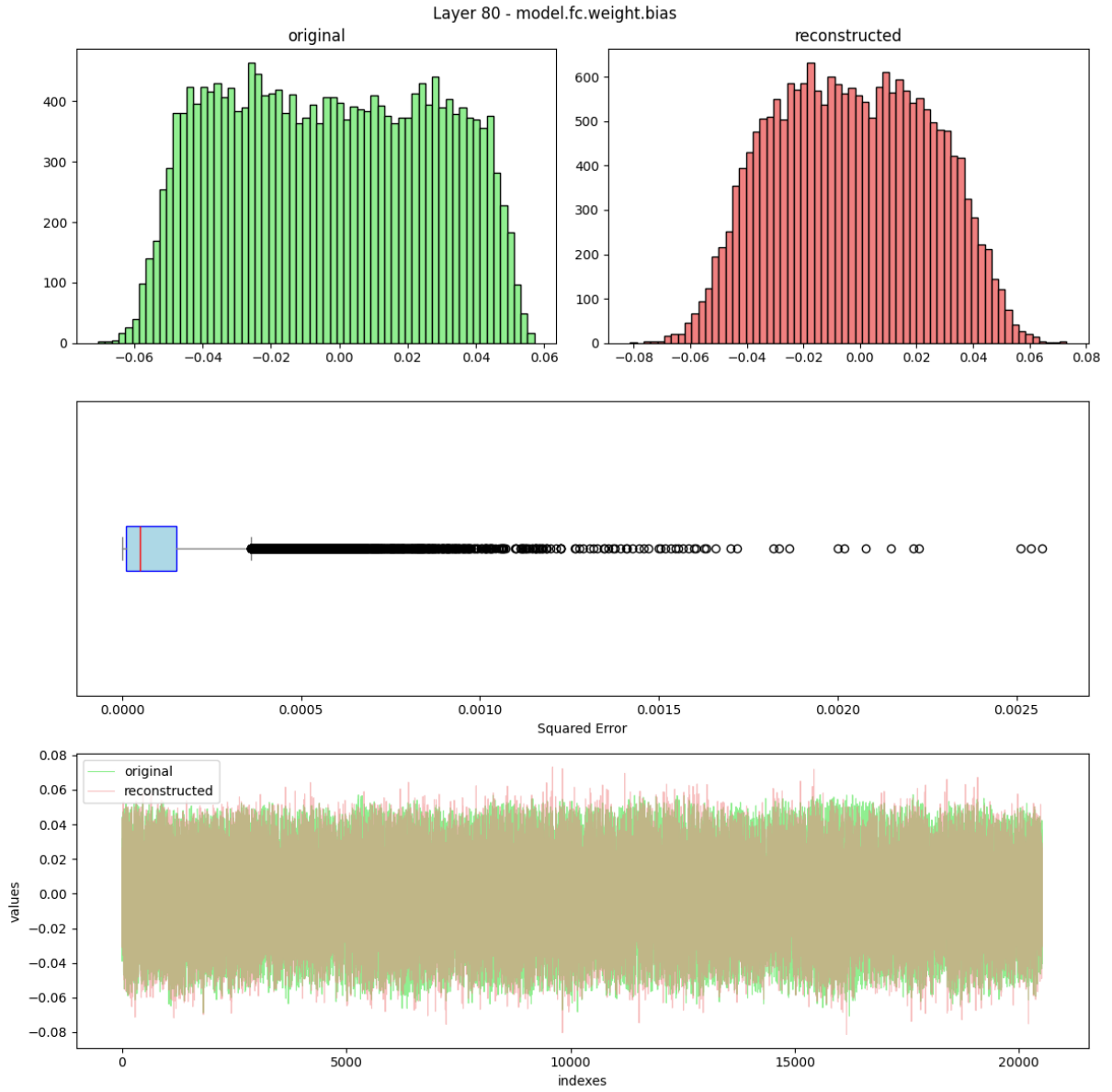


Figure 31: Reconstruction analysis of the fully connected classification head. The top panel shows the original (green) and reconstructed (red) weight distributions; the central panel reports the distribution of MSE values; the bottom panel compares index-wise weight alignments.

4.3.3 Augmented Single Origin Experiments

Given the analysis in the previous section, two core issues arise when addressing the weight learning with the framework:

1. An augmentation technique is required to allow the model to better generalize also on the classification head
2. Given the very small values in the weight space, a learning rate scheduling is necessary to escape the huge amount of local minima.

To address the first issue, the mode connectivity [11] property is exploited to extract, using a small random noise injection, 2 different versions of the AwA2 origin, with the requirement of having a maximum performance loss tolerance of 10^{-1} .

In this perspective, the algorithm implemented to produce such augmentations is a simple random search with decreasing noise injection to extract closer neighbors, considered valid only if the performance loss requirement is met:

Algorithm 1 Checkpoint Randomization and Evaluation Procedure

```

1: Initialize scale  $\leftarrow 9 \times 10^{-1}$ 
2: while  $n_{\text{generated}} < n_{\text{random}}$  do
3:    $R \leftarrow \text{RandomizeCheckpoint}(\text{checkpoint}, \text{scale})$ 
4:    $\text{LoadWeights}(\text{model}, R)$ 
5:    $M \leftarrow \text{Evaluate}(\text{model}, \text{testset}, \text{ID} = n_{\text{generated}})$ 
6:   if  $\text{comparison\_metric} - M[\text{accuracy}] \geq \text{performance\_loss\_tolerance}$  then
7:      $\text{LogFailure}(\text{scale}, M[\text{accuracy}], \text{comparison\_metric})$ 
8:      $\text{scale} \leftarrow \text{scale} \times \text{step}$ 
9:     continue
10:  end if
11:   $\text{LogSuccess}(\text{scale}, M[\text{accuracy}], \text{comparison\_metric})$ 
12:   $\text{SaveCheckpoint}(R, M, n_{\text{generated}})$ 
13:   $n_{\text{generated}} \leftarrow n_{\text{generated}} + 1$ 
14:   $(T, K, P) \leftarrow \text{TokenizeWeights}(R)$ 
15:   $\text{Append}(\text{tokens}, T); \quad \text{Append}(\text{masks}, K); \quad \text{Append}(\text{positions}, P)$ 
16:   $\text{Append}(\text{properties}, M)$ 
17: end while

```

Using algorithm 1, the dataset is constructed to contain n_{random} models arranged sequentially, immediately following the origin’s tokenization. This sequential organization preserves the indexing format, ensuring compatibility with PyTorch’s `Dataset` and `DataLoader` classes. As a result, each training step can access the dataset without modifying its final structure, since data is only retrieved via the starting indices of the respective windows. Moreover, it is possible to either keep or exclude the origin from the dataset, to use it as a validation tool and effectively early-stop the training process.

The configurations corresponding to a single transformer block ($b = 1$) and a single attention head ($h = 1$) were rerun under the augmented setup. The results, including the runtime of each configuration, are reported in table 24:

Table 24: Experimental results for the $b = 1$, $h = 1$ configurations under the augmented setup. Stride (sws) is defined as $stride = \frac{window_size}{sws}$. Runtime is reported in hh:mm:ss format.

sws	e	Origin Acc.	Recon. Acc.	Origin F1	Recon. F1	Runtime
1	256	92.60	88.73	90.20	86.47	16m 42s
1	512	92.60	92.14	90.20	89.77	29m 23s
2	256	92.60	91.87	90.20	89.40	28m 18s
2	512	92.60	92.57	90.20	90.35	57m 20s
4	256	92.60	92.31	90.20	90.10	53m 58s
4	512	92.60	92.50	90.20	90.23	1h 47m 24s

The augmented setup leads to noticeable improvements across all evaluated configurations, with both Accuracy and F1-Score showing enhancement. In several cases, more complex configurations even surpass the original model’s performance.

Runtime measurements indicate that, while the method is generally efficient, the $sws = 4$, $e = 512$ configuration requires substantially more time due to the increased computational cost of learning three distinct model versions. In contrast, the simplest scenario ($sws = 1$, $e = 256$) remains highly efficient, with only minor reductions in performance. These observations suggest that reducing the stride has a greater impact on computational time than on the quality of the reconstructed models. Furthermore, the embedding dimension consistently influences the final reconstructed metrics, with higher dimensions leading to more accurate reconstructions.

Performance gains are also influenced by the adoption of the OneCycleLR scheduler, which periodically resets the learning rate, creating peaks in the loss that facilitate escaping local minima. An example of this behavior is shown in the loss curve for the $sws = 2$, $e = 512$ configuration in fig. 32:

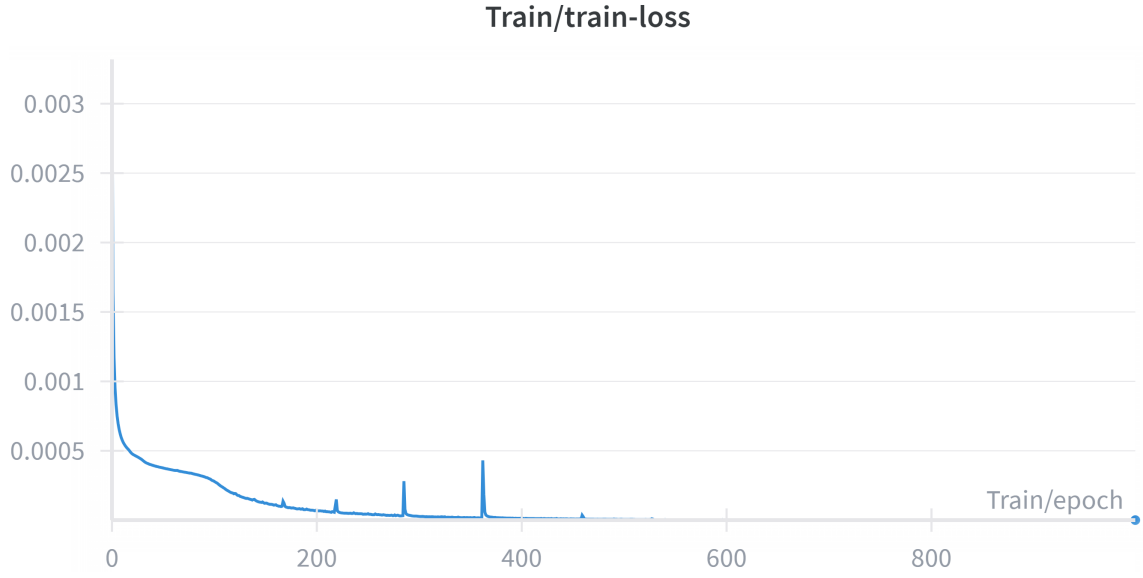


Figure 32: Training loss over 1000 epochs for the $sws = 2$, $e = 512$ configuration, showing multiple learning rate cycle resets that contribute to peak performance.

Another notable observation is the direct correlation between window stride and the model’s ability to learn the fully connected (FC) layer. While errors are significantly reduced in all configurations under the augmented setup (maximum error on the order of 10^{-5}), the boxplots in fig. 33 highlights that reducing the stride further minimizes the gap between convolutional layer errors and FC layer errors:

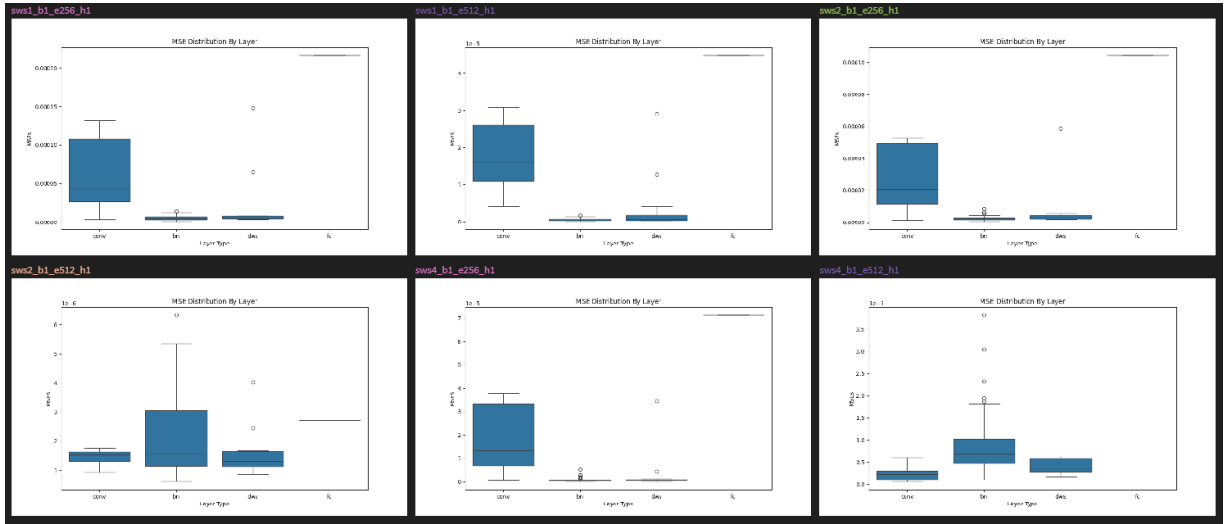


Figure 33: Mean Squared Error distributions grouped by layer type under the augmented setup, illustrating reduced discrepancies between convolutional and fully connected layers.

The improved performance is further confirmed by the weight distribution plots in fig. 34 in which the reconstructed model closely follows the distribution of the original network when using $e = 512$:



Figure 34: Weight distributions of the original network (green) versus the reconstructed network (red). Configurations shown (top-left to bottom-right): $sws = 1, e = 256$; $sws = 1, e = 512$; $sws = 2, e = 256$; $sws = 2, e = 512$; $sws = 4, e = 256$; $sws = 4, e = 512$.

Perfect reconstruction is achieved already in the $sws = 2, e = 512$ configuration, as illustrated in fig. 35. Based on these results, three core configurations were selected for further experiments:

1. **Light Configuration:** $sws = 1, e = 256, b = 1, h = 1$ — prioritizing efficiency.
2. **Average Configuration:** $sws = 2, e = 512, b = 1, h = 1$ — balanced performance, top performer in previous experiments.
3. **Heavy Configuration:** $sws = 4, e = 512, b = 2, h = 2$ — highest quality version, suitable for multi-dataset scenarios despite slightly increased complexity.

Having established strong performance in the single-dataset scenario, the next step has been to extend the framework to multi-dataset settings using the origins trained on ZSL benchmark datasets. This allows an evaluation of the model’s capability to simultaneously learn multiple configurations.

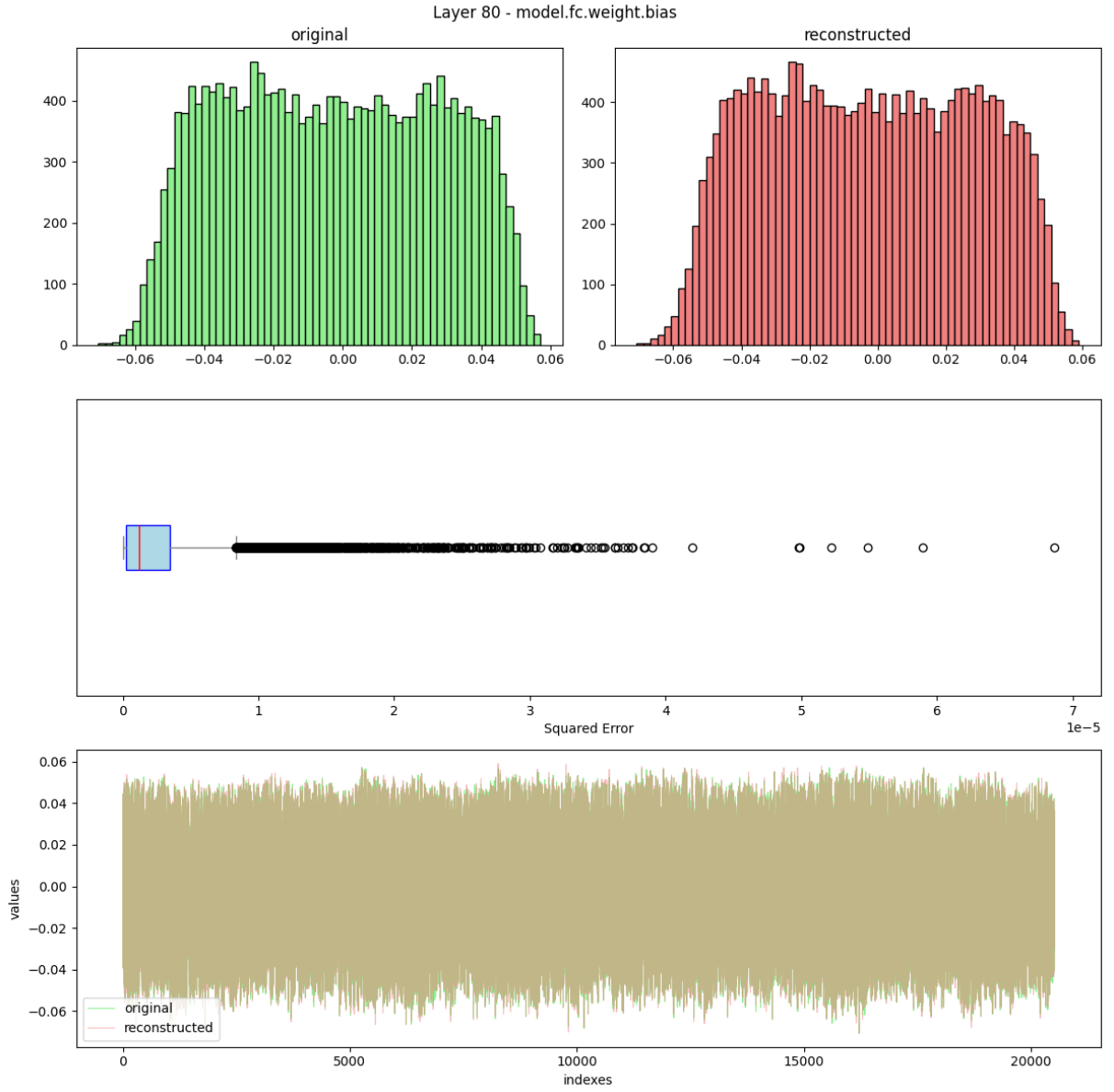


Figure 35: Reconstruction versus original weights distribution for the $sws = 2$, $e = 512$, $b = 1$, $h = 1$ configuration. The figure occupies a full page for clarity.

4.3.4 Multi-Origin Experiments

The objective of this set of experiments is to evaluate the framework’s ability to jointly learn multiple hyper-representation spaces derived from distinct pretrained model origins. This setting extends the single-origin experiments by introducing additional variability and complexity, thereby probing the generalization capacity of the proposed approach.

To ensure comparability, the three benchmark configurations identified in the previous section were retained:

1. **Light Configuration.** Stride equal to the window size ($sws = 1$); one transformer block with a single attention head ($b = 1, h = 1$); embedding dimensionality $e = 256$.
2. **Balanced Configuration.** Stride equal to half the window size ($sws = 2$); one transformer block with a single attention head ($b = 1, h = 1$); embedding dimensionality $e = 512$.
3. **Heavy Configuration.** Stride equal to one quarter of the window size ($sws = 4$); two transformer blocks, each with two attention heads ($b = 2, h = 2$); embedding dimensionality $e = 512$.

To adapt the dataset construction to this new setup, each origin (CUB, AwA2, aPY) was tokenized independently. Window selection was carried out using the set of valid starting indices, consistent with the procedure employed in the single-origin experiments. This enabled the use of the `Dataset` and `DataLoader` classes in PyTorch, with shuffling activated to enhance training diversity. The resulting datasets were then merged using the `ConcatDataset` class. This modular design avoided the need for additional code dedicated to multi-origin data management.

Training was conducted for 1000 epochs under the same protocol as in the single-origin setting. In particular, the OneCycleLR scheduler was retained due to its effectiveness in promoting convergence and preventing entrapment in local minima.

To further assess the robustness of the framework, an additional line of experiments was introduced in which the number of randomized models per origin was increased. This augmentation yielded a pool of 18 models in total (three origins, each with six versions: the original plus five randomized variants). Both the augmented and non-augmented multi-origin configurations were evaluated in order to quantify the impact of model randomization on performance.

To facilitate interpretation, results are reported in three separate tables, each corresponding to one of the benchmark configurations. Within each table, both the augmented and non-augmented multi-origin setups are compared, while baseline metrics remain directly visible in the *Origin Acc.* and *Origin F1* columns.

Table 25: Multi-origin experimental results for the Light configuration ($sws = 1$, $e = 256$, $b = 1$, $h = 1$). Both augmented and non-augmented setups are reported. Runtime is only reported once as the training involves all the origins at once.

Origin	Setup	Origin Acc.	Recon. Acc.	Origin F1	Recon. F1	Runtime
CUB	No-Augment	72.40	58.51	73.40	62.67	23m 04s
	Augmented	72.40	72.13	73.40	73.47	1h 23m 28s
AwA2	No-Augment	92.60	87.29	90.20	84.66	–
	Augmented	92.60	92.23	90.20	90.01	–
aPY	No-Augment	89.90	81.36	90.60	83.78	–
	Augmented	89.90	88.28	90.60	89.41	–

Table 26: Multi-origin experimental results for the Balanced configuration ($sws = 2$, $e = 512$, $b = 1$, $h = 1$). Both augmented and non-augmented setups are reported.

Origin	Setup	Origin Acc.	Recon. Acc.	Origin F1	Recon. F1	Runtime
CUB	No-Augment	72.40	72.13	73.40	73.23	1h 03m 39s
	Augmented	72.40	72.70	73.40	73.77	5h 36m 42s
AwA2	No-Augment	92.60	91.64	90.20	89.10	–
	Augmented	92.60	92.55	90.20	90.30	–
aPY	No-Augment	89.90	87.28	90.60	88.88	–
	Augmented	89.90	89.68	90.60	90.36	–

Table 27: Multi-origin experimental results for the Heavy configuration ($sws = 4$, $e = 512$, $b = 2$, $h = 2$). Both augmented and non-augmented setups are reported.

Origin	Setup	Origin Acc.	Recon. Acc.	Origin F1	Recon. F1	Runtime
CUB	No-Augment	72.40	73.23	73.40	74.24	1h 52m 36s
	Augmented	72.40	65.81	73.40	68.40	6h 46m 30s
AwA2	No-Augment	92.60	91.49	90.20	88.92	–
	Augmented	92.60	90.52	90.20	88.51	–
aPY	No-Augment	89.90	87.62	90.60	89.47	–
	Augmented	89.90	84.55	90.60	86.68	–

The results reported in Tables 25 to 27 highlight several consistent trends across the three benchmark configurations.

First, the introduction of augmentation generally improves reconstruction performance in the Light and Balanced settings. In particular, for the Light configuration, augmentation substantially boosts the reconstruction metrics across all datasets, with improvements exceeding +10 points in both accuracy and F1 for CUB and aPY. The Balanced configuration confirms this trend, showing that augmentation stabilizes performance and reduces the gap between origin-level accuracy and reconstruction, particularly for AWA2 and aPY. These findings indicate that augmentation is most beneficial in low-capacity settings, where it counteracts underfitting and enhances cross-origin generalization.

By contrast, the Heavy configuration presents a different outcome. Here, augmentation does not consistently improve results and in some cases degrades performance, particularly for CUB, where reconstruction accuracy drops by nearly 8 points compared to the non-augmented variant. A similar, though less pronounced decline, is observed for AWA2 and aPY. This suggests that in high-capacity settings, the variability introduced by augmentation may impair convergence, potentially by over-regularizing the model and preventing it from fully leveraging its representational power. Conversely, the non-augmented variant achieves stronger results, indicating that when multiple transformer blocks and attention heads are available, a direct training strategy is more effective and naturally promotes generalization without the need for additional augmentation.

Runtime analysis reinforces these observations. Augmentation significantly increases training times across all settings, with multiplicative overheads ranging from $3.6\times$ (Light) to more than $5\times$ (Balanced). The Heavy configuration is the most expensive, requiring almost 7 hours in the augmented case compared to less than 2 hours without augmentation. Thus, while augmentation can be crucial in lighter configurations to achieve competitive reconstruction quality, its cost and diminishing returns in heavier models must be carefully considered.

In summary, the experiments demonstrate that augmentation is a powerful tool for improving generalization in low- and mid-capacity settings, but may be detrimental in high-capacity scenarios where the model already possesses sufficient expressive power. Future work could explore adaptive augmentation strategies that balance diversity and stability depending on the capacity of the underlying architecture.

4.4 Visualization of the Hyper-Representation Space

To gain a deeper understanding of the **hyper-representation space** learned by the autoencoder, a series of visualizations was conducted. The primary objective was to observe how the framework organizes and represents models trained on different origins, particularly in a multi-origin setting. This approach provides a qualitative complement to the quantitative metrics presented in the previous tables, offering intuitive insights into the model’s learned embeddings.

The visualization of high-dimensional spaces is a challenging task due to several inherent problems. The most prominent is the *curse of dimensionality*: a phenomenon where the volume of a space increases exponentially with its dimensionality, making it increasingly difficult to find representative data points, as distances between them become less meaningful. A common strategy to address this is dimensionality reduction, which projects the data into a lower-dimensional, visually interpretable space (e.g., 2D or 3D). However, such projections risk losing critical information and distorting the underlying structure.

For this purpose, the **t-Distributed Stochastic Neighbor Embedding (t-SNE)** algorithm was chosen. t-SNE is a non-linear dimensionality reduction technique well-suited for embedding high-dimensional data into a lower-dimensional space while preserving the local structures. This means that models that are ”close” in the original high-dimensional hyper-representation space will remain close in the 2D visualization, allowing for visual inspection of clustering and relationships.

This visualization serves several key purposes:

1. **Confirmation of Learning.** The most fundamental motive is to visually confirm that the autoencoder is indeed learning a **semantically meaningful** representation: a successful model should not only have good reconstruction metrics but also organize the models in its latent space based on their intrinsic properties, such as their pretraining origin.
2. **Qualitative Analysis of Separation.** The visualization allows for a direct assessment of how well the different model origins are separated. If the autoencoder has successfully captured the unique characteristics of each dataset (CUB, AwA2, aPY), an expect outcome is to see distinct, well-defined clusters, with each corresponding to a specific origin. This provides a clear, qualitative measure of the framework’s ability to discriminate between different training histories.
3. **Insight into Intra-Class and Inter-Class Variability.** Beyond simple separation, the plots can reveal subtle dynamics: the density and spread of each cluster can indicate the variability within models from the same origin, and, similarly, the distance and overlap between clusters can suggest the degree of similarity or difference between the origins themselves from the perspective of the learned model representations.

4.4.1 Experimental Methodology

The visualization was performed on the set of models from the three original training origins (CUB, AwA2, aPY), which were not augmented with randomized variants. The experimental steps have been conducted as follows:

1. **Model Sampling.** For each of the three origins, the pre-trained ResNet18 model was tokenized, and the resulting windows were passed through the chosen configurations of the autoencoder. For each window, a hyper-representation of the weights within it was obtained.
2. **Dimensionality Reduction.** The collection of high-dimensional hyper-representations for all windows across the three origins was then passed through the t-SNE algorithm. The output was a 2D coordinate for each hyper-representation, enabling plotting on a standard scatter plot.
3. **Plotting.** Each point on the scatter plot encodes a single hyper-representation (or window) from a ResNet18 model. The points were color-coded according to their pretraining origin (CUB, AwA2, or aPY) to visually highlight the clustering patterns (a legend is included to map colors to their respective origins).
4. **Configuration Analysis.** This procedure was applied to the models from each of the three benchmark configurations (Light, Balanced, and Heavy) that showed the best results to observe if the learned space and clustering behavior varied with the capacity of the autoencoder. It is worth mentioning that, while the *Light configuration* and the *Balanced configuration* both included randomized variants in the training, the *Heavy configuration* did not, making it a valid example to assess the impact of the augmentation on the hyper-representation space.

The results of this visualization are presented in the subsequent figures and can be a valuable insight for understanding how the framework succeeded in distilling the complex relationships between diverse DNN models within a unified space. To also improve the analysis a first test has been conducted over the untrained architecture to produce the embeddings from the same starting origins (fig. 36)

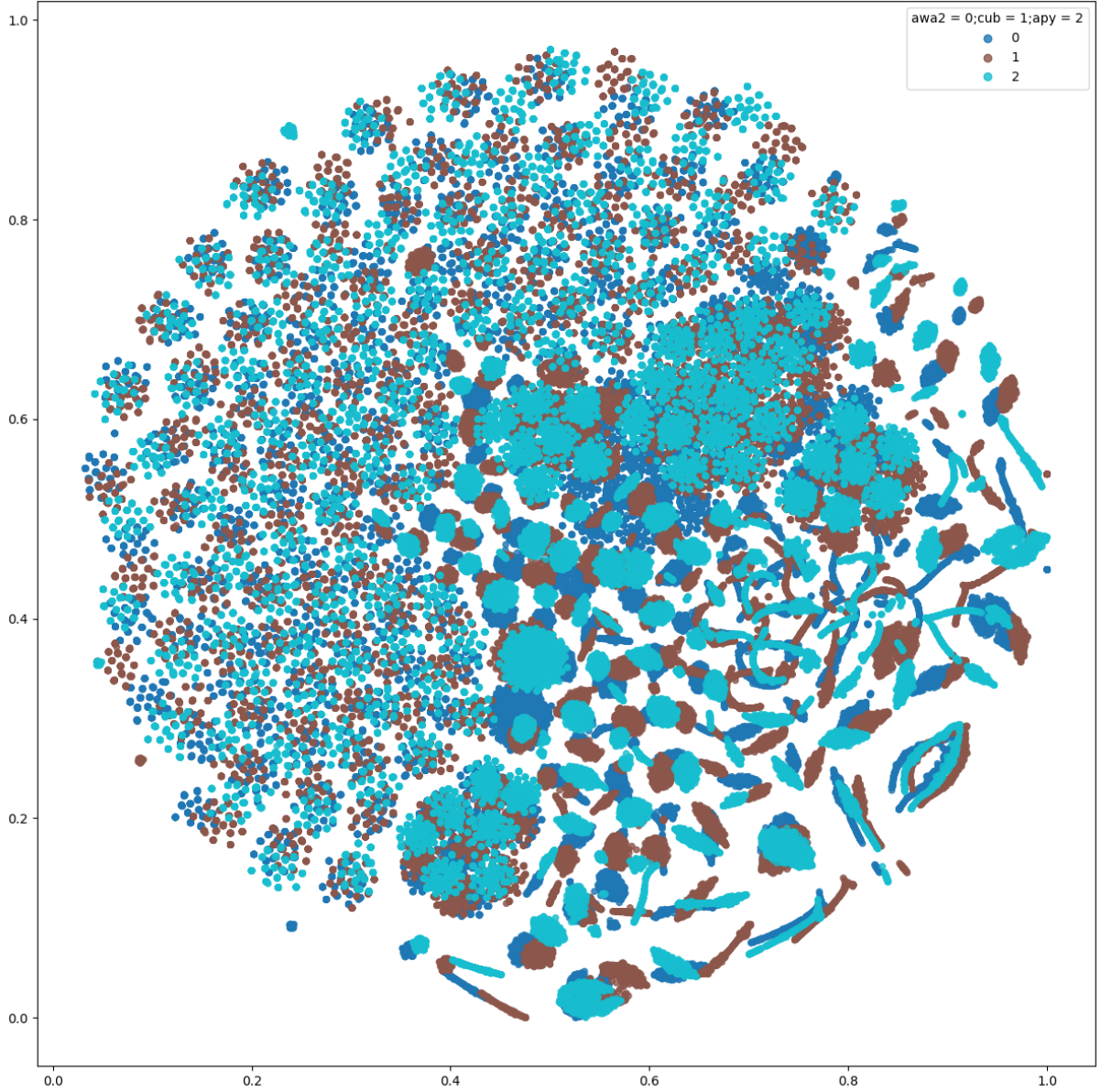
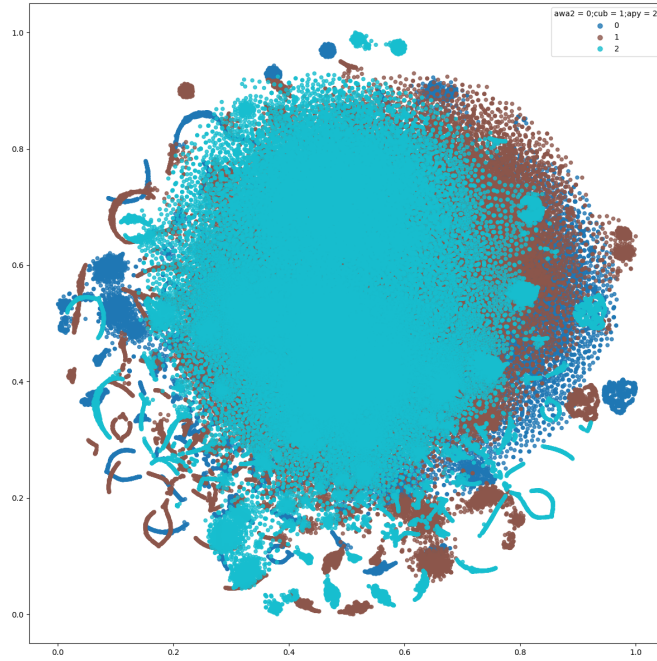
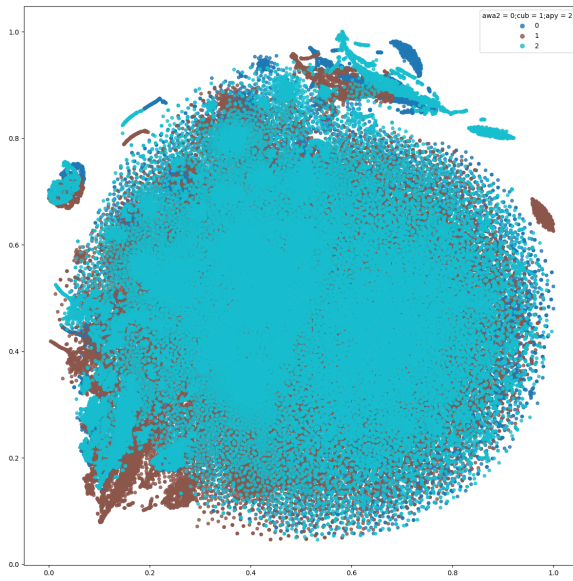


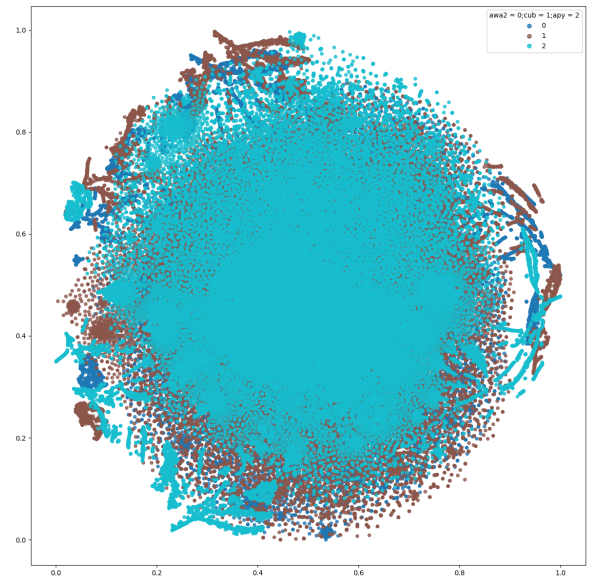
Figure 36: t-SNE visualization of the hyper-representation space generated from the untrained version of the simplified SANE framework. Each point represents a hyper-representation of a model weight window, color-coded by the model’s pretraining origin from which was obtained: CUB (Brown), Awa2 (Blue), aPY (Light Blue).



(a) Heavy Configuration



(b) Light Configuration



(c) Balanced Configuration

Figure 37: t-SNE visualization of the hyper-representation space for the three benchmark configurations: Heavy (a), Light (b), and Balanced (c). Each point represents a hyper-representation of a model weight window, color-coded by the model's pretraining origin from which was obtained: CUB (Brown), AwA2 (Blue), aPY (Light Blue).

4.4.2 Key Observations

The t-SNE visualizations provide compelling qualitative insights into the learned hyper-representation space. The plots, produced for each of the three autoencoder configurations, reveal how the framework organizes the embeddings of a model’s weight windows, highlighting key aspects of the learned structure and the effects of our training methodology.

Untrained Visualization

The first aspect that emerges from the visualization of the untrained hyper-representation space is the natural tendency of it to be clustered in multiple different and sparse blocks, where the different origins are often overlapped and sometimes shifted by a small amount.

While this first result could be misleading, a possible interpretation of it could be that since the hyper-representation space is the result of the encoding of structured neural networks with distinctly purposed layers such as convolutional layers and batch normalization layers, the hyper-representation space itself has a tendency to be structured as well, even in an untrained form.

Here, in fact, while a structure is present, it is not possible to distinguish which role would such blocks have in the task performed by the origin networks. It seems, instead, that the various embeddings produced are randomly taking a position because of the structural role more than a task-specific embedding: they are grouping out of the network structure instead of for the purpose they have in the network.

Trained Visualizations

Here is where the previous statement finds confirmation: in the trained visualizations, the embeddings are mostly reorganized into a central mass that is mostly overlapping for all the methods. Out of this mass, a bunch of minor clusters, with very specific structures that are sometimes copied and sometimes mirrored in other origins emerge across the peripheries. It can be noticed that in all the configurations there are recurring shapes, with aPY and AwA2 being the most overlapping ones at a point where blue color (representing AwA2 origin) is barely visible.

Another very interesting aspect of this visualization emerges from the symmetries: while most of the embeddings reside in the central mass, there are several blocks that are placed in the periphery and that show common shapes. By looking for instance to the Heavy Configuration in figure 37, there are several groups that are very similar in both structures and values:

- $x = [0.2, 0.4]$; $y = [0.0, 0.2]$ area showing a two blocks shape with a denser block above a smaller one almost attached to it in aPY origin’s representation (light blue), perfectly mirrored by AwA2’s one (blue) in the area $x = [0.0, 0.2]$; $y = [0.4, 0.6]$
- $x = [0.8, 0.1]$; $y = [0.2, 0.6]$ area showing three very distinct blocks that are almost identical in shape, again almost mirrored for AwA2 and aPY origins, and slightly transposed for CUB.

Analysis of the Central Mass

A primary observation across all three configurations is the presence of a **dense central mass of embeddings**. This mass is composed of hyper-representations from all three origins (CUB, AwA2, and aPY), suggesting that a significant volume of the models’ parameters are likely dedicated to generic computations like downsampling, batch normalization, and fundamental convolutions.

These shared, low-level operations result in embeddings that are inherently close in the hyper-representation space, thus forming the dense central cores, although a closer inspection reveals that the center of those masses is not perfectly uniform: it shows a slight bias in its composition per origin. This subtle bias indicates that even these generic computational layers exhibit minor, dataset-specific differences, which the autoencoder is sensitive enough to detect in the most performant scenarios.

The Formation of Distinct Clusters

Beyond the central mass, the visualizations show distinct, **small clusters** forming along the periphery of the plot. These clusters are a direct result of the autoencoder’s ability to discern dataset-specific embeddings. The presence of these separate, well-defined clusters, with occasional symmetric arrangements, demonstrates that the autoencoder has effectively learned to isolate the core characteristics unique to each dataset.

These embeddings likely correspond to the higher-level convolutional layers responsible for learning more specialized features, such as bird parts for CUB or specific animal features for AwA2. The separation and structure of these clusters provide compelling visual proof that the framework is successful in its objective of learning a semantically meaningful representation of model properties.

The Impact of Augmentation

Another fundamental observation comes from comparing the visualizations of the augmented and non-augmented configurations. The Heavy configuration, which was trained without augmentation, shows a significantly clearer separation, not only of the peripheral clusters but also of the central masses.

This finding strongly suggests that the augmentation process affects the training by condensing the central masses. The introduction of randomized versions of the same model forces the autoencoder to find commonalities across all variants as it leads to a representation where the shared, generic computations are more tightly grouped, consequentially having a more relevant presence within the dataset used for training them.

Conversely, training on a single, non-augmented set of models allows the autoencoder to learn a more distinct embedding for each model’s generic layers, resulting in a less condensed, more separable central mass. This provides valuable insight into how the framework learns, that could lead to core simplifications.

4.4.3 Differential Analysis of the Residual Blocks

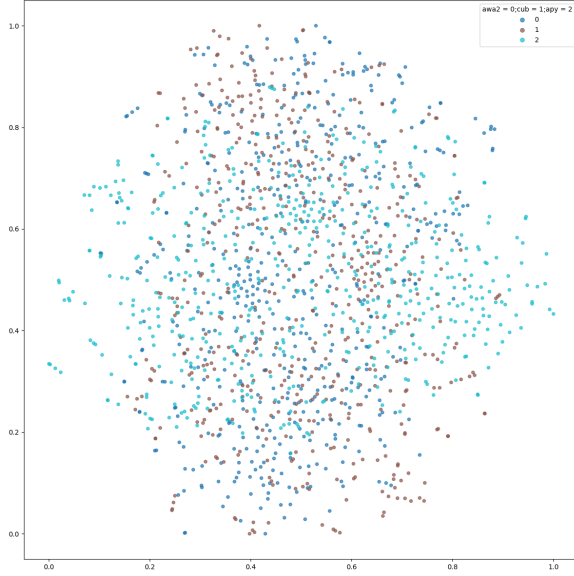
To further investigate how the framework achieves a clear separation of different origins within the hyper-representation space, and to explore its potential contribution to the field of Explainable AI (XAI), this section presents an additional set of analyses.

The *Heavy Configuration* was selected for this study, as it demonstrated the strongest separation in the hyper-representation space and exhibited reduced dependence on data augmentation. In order to refine the visualization analysis and to better characterize how the different origins perform their classification tasks, each residual block of the three origins was independently tokenized and processed using the same methodology described in the previous analysis.

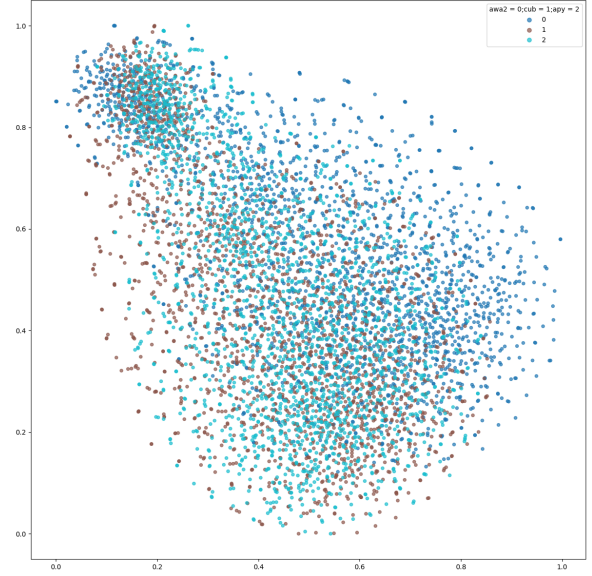
This setup, however, revealed a core limitation of the framework: to ensure consistency between tokenization and positional embeddings, the configuration had to be retrained with the updated positional encodings (both for layers and tokens), as a lack of specific indexes altered the reconstruction in the pretrained configurations. While this limitation can be overcome with a deeper investigation of the role that each positional dimension has in the final encoding, to keep this analysis simple, the retraining has been preferred, allowing further investigations for future works.

With this adjustment, the analysis was restricted to the convolutional layers of the selected residual stage, which were tokenized and incorporated into the hyper-representation space learning process. For consistency with the previous section, the resulting two-dimensional features were again visualized using the same color-coding strategy.

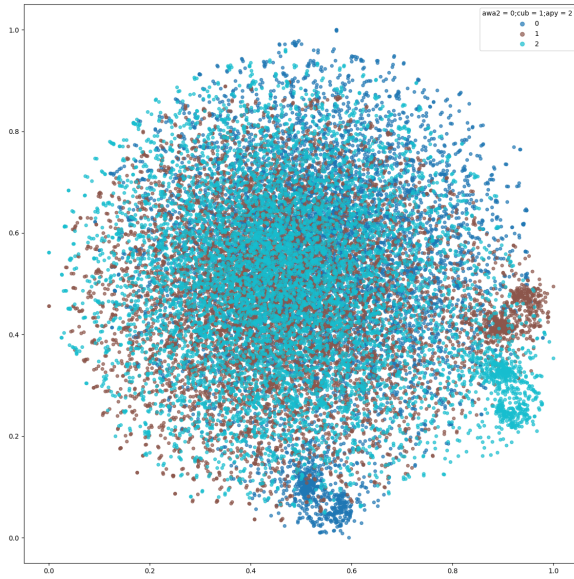
This procedure produced four distinct visualizations (Fig. 38), corresponding to the four residual blocks of the ResNet18 architecture. These results provide an informative perspective on the contribution of each block to the separation of dataset-specific feature spaces, thereby offering deeper insights into the internal mechanisms underlying the classification process.



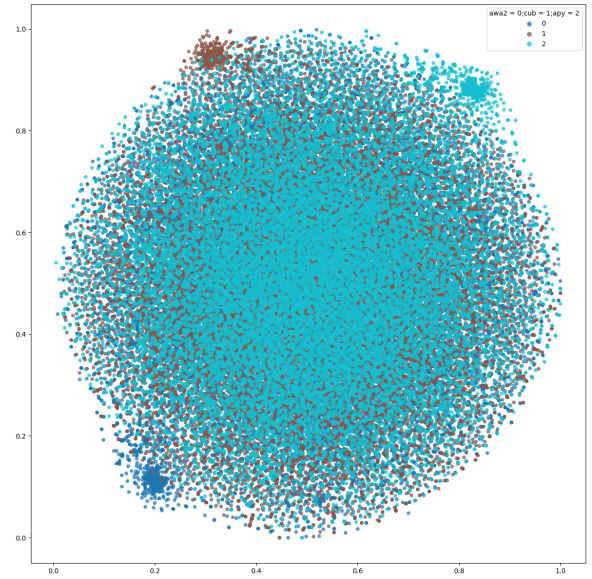
(a) Hyper-representation of the first residual block in the ResNet18 origins.



(b) Hyper-representation of the second residual block in the ResNet18 origins.



(c) Hyper-representation of the third residual block in the ResNet18 origins.



(d) Hyper-representation of the fourth (final) residual block in the ResNet18 origins.

Figure 38: Differential visualization of the hyper-representation space across the four residual blocks of ResNet18, illustrating the representations of the three origins.

4.4.4 Patterns of Separation Across Residual Blocks

The visualizations in fig. 38 reveal several noteworthy patterns: the earlier residual blocks (Figures 38a and 38b) display a high degree of overlap between the origins, together with a noticeably sparser structure that directly reflects the reduced number of parameters in these blocks. Their representations appear distributed almost randomly across the hyper-representation space, suggesting that lower-level convolutional features remain largely shared across datasets and primarily serve as general-purpose computations for common feature extraction tasks.

By contrast, the later residual blocks (Figures 38c and 38d), while still retaining a central mass of generic computations, also exhibit more clearly defined clusters associated with each origin. This progression indicates that dataset-specific and discriminative features are increasingly encoded in the deeper layers, with separable regions emerging in the hyper-representation space.

These observations are consistent with the established understanding of convolutional architectures: early layers capture generic low-level features such as edges and textures, while deeper layers encode more abstract, task-specific representations. By projecting the parameters of each residual block into the hyper-representation space, the framework makes these hierarchical transitions directly observable.

The ability to localize where separation emerges highlights the potential of the proposed approach as a tool for Explainable AI, providing an interpretable view of how individual residual blocks contribute to the overall classification process. Moreover, with the application of additional techniques, this analysis could be extended to extract valuable insights for more complex tasks beyond standard classification problems.

The visualizations also emphasize how residual blocks cooperate sequentially to simplify the task for subsequent layers: each block progressively reduces the complexity of the feature space, ultimately producing representations that are close to linearly separable.

This finding adds a new interpretability perspective: although deep neural networks are often regarded as black boxes, the hyper-representation space exposes a more transparent view of how knowledge is structured and transferred across layers. In particular, the results suggest that transferring an origin’s training to a different dataset could, in principle, be achieved by substituting the clustered regions corresponding to one origin with those of another across subsequent residual blocks.

Such a mechanism, which can be visually interpreted as swapping only the clustered regions in the hyper-representation space across layers, while keeping the central mass intact, would effectively enable the transfer of model knowledge between datasets, thereby opening new directions for efficient adaptation and unlearning strategies.

4.5 Framework Simplifications

Building upon the valuable insights gained from the framework’s ability to differentiate various versions of the same network, a new set of simplifications was introduced to evaluate the core potential of the method under a minimalist setup.

A key assumption for this approach is the observed property in most computer vision tasks, where the initial stages of deep neural networks are dedicated to common computations such as edge detection. As in fact confirmed by the t-SNE visualizations (fig. 37), only the latter stages of the network are the ones primarily influencing the feature space used for dataset-specific classification.

Based on this principle, the first simplification has been to streamline the tokenization process by omitting generic layers like batch normalization and downsampling, retaining only the convolutional layers, as done in the differential analysis, while also keeping the classification head. This led to the design of a new experiment to assess the method’s potential and scalability, not only from an architectural point of view, but also in terms of which parameters are essential to solve a specific tasks.

Proving this key aspect of this novel framework, would directly address what was the main limitation of HyCUS: not only it has been possible, via the differential analysis in the previous section, to identify which parameters are essential to solve a task, but it is also possible to further exclude them from the tokenization scaling the method to only address what is strictly necessary for a specific task.

To efficiently address this experimentation, a single multi-dataset training run of the Light configuration was configured, where only the final fourth block of the ResNet18 model and its classification head were tokenized for each pretraining origin and version. To rigorously test the framework, the augmentation included 100 randomized versions for each origin, where this time, the origin were excluded from the training, and their tokenization has been used as validation to allow better generalization.

Table 28: Comparison of Autoencoder Reconstruction Performance vs. Origin Model Performance

Dataset	Metric	Recon. Model	Origin Model
CUB	Accuracy	72.76	72.40
	F1-Score	74.04	73.40
AwA2	Accuracy	92.48	92.60
	F1-Score	90.17	90.20
aPY	Accuracy	89.55	89.90
	F1-Score	90.40	90.60
Runtime: 8h 41m 45s			

The results presented in table 28 confirm that the autoencoder is capable of producing reconstructions with performance metrics highly consistent with the original models, accordingly with previous results. For both the AwA2 and aPY datasets, the reconstructed models achieve accuracy and F1-scores that are almost identical to their origin counterparts, demonstrating that the tokenization and reconstruction process effectively preserves the critical information encoded within the model weights.

A noteworthy observation is the slight improvement in both Accuracy and F1-Score for the reconstructed model on the CUB dataset, obtained despite the experiment being conducted using the less complex Light configuration. This suggests that the autoencoder’s learning process may function as a form of cross-dataset regularization: by condensing the CUB-related embeddings and integrating them within the hyper-representation space alongside other learned origins, the framework appears to enhance the predictive performance of the reconstructed model, pulling each origins towards a common centroid and resulting in a natural tendency to affect the generative process to produce hybrid embeddings.

4.6 Results Summary and Considerations

In conclusion, the various experiments conducted in this section posed a solid foundation for the weight-as-data paradigm, introducing a powerful weight-learning tool that demonstrated notable strengths, particularly in terms of:

- **Flexibility**, as it has been shown to be applicable across multiple tasks, not only discriminative and generative, but also as an XAI tool capable of transparently highlighting which information is essential for a specific task by leveraging attention-aware embeddings.
- **Scalability**, both in terms of architectural scalability and in terms of what can be included in the hyper-representation space computation, enabling a variety of configurations even for relatively simple tasks.
- **Computational capabilities**, as it was successfully integrated with purpose-designed augmentation techniques to improve efficiency, although the limited number of epochs available still acted as a bottleneck for tasks requiring frequent re-iterations of the method.

On the other hand, the framework also exhibited several core limitations:

- **Tokenization flexibility**, since the need to maintain consistency with positional embeddings required continuous retrainings whenever the tokenization strategy was modified, significantly impacting usability and efficiency.
- **Augmentation overhead**, as simpler configurations demanded a substantial amount of augmentation to ensure stable learning. While this reinforced the architectural scalability of the framework, it simultaneously introduced a considerable memory overhead, particularly in lower-performance setups, thereby reducing accessibility in resource-constrained environments.

5 Conclusions and Future Work

Throughout the course of this thesis, the Weight-as-Data paradigm has emerged as a promising and innovative approach for addressing diverse learning tasks. In particular, hypernetworks have been explored, highlighting core limitations arising from their reliance on additional information about a model’s training process. This dependency can represent a major drawback in research environments where such training details are proprietary and thus unavailable, reducing reproducibility.

While these limitations often discourage the use of this learning paradigm, favoring instead data-driven approaches could significant challenges such as:

- the need for large amounts of data to effectively train networks, often raising serious privacy concerns that are difficult to resolve,
- reliance on computationally and memory-intensive processes, frequently repeated due to dataset errors, privacy restrictions, or shifts in the neural network’s operating environment,
- dependence on training datasets (e.g., image sets for computer vision tasks), which may themselves be considered additional training information and are not always publicly accessible.

Despite these challenges, recent methods such as Zero-Shot Learning with ICIS [5] and Machine Unlearning with HyCUS have demonstrated that such limitations can be mitigated. Both approaches exemplify how trained model weights alone can serve as the sole data source for learning new tasks, enforcing an image-free constraint in two distinct problem domains.

These methods further showed that, through the use of an autoencoder, it is possible to learn a sub-representation of a model’s weight space, enabling a type of reconstruction that can be guided by auxiliary processes, actively influencing the decoding phase, to generate task-specific variations.

Although interesting for its simplicity, a crucial limitation of these approaches has also been identified: while in ZSL it was sufficient to act only on the classification head, this was not the case in Machine Unlearning, where the problem is inherently more complex, and minimal modifications to the backbone proved insufficient. With this limitation being explicitly analyzed in MU, it is also reasonable to expect it to extend to other downstream tasks as well.

Motivated by these findings, and by the need for a more scalable and unified framework: the SANE approach [27] was investigated and subsequently adapted to a more general generative setting, potentially enabling the framework to address both tasks, even when tackled simultaneously.

In this context, the SANE autoencoder has been shown to learn an advanced form of latent representation: the hyper-representation space. Here, the model is not encoded into a single point derived from the full weight vector (which is infeasible in practice). Instead, it is represented as a set of task-aware embeddings obtained through a flexible number of transformer blocks and attention heads.

For the purposes of this thesis, this framework was simplified, as discriminative problems were outside its scope. The resulting simplified version was tested under multiple configurations, demonstrating its adaptability to various scenarios through the use of augmentation techniques. Another core characteristic of this extended space, is the ability to be adapted, modifying its structure dependently on the training process and loss functions applied.

To enforce this characteristic, by using a simple masked reconstruction loss it has been shown how classification-specific separation can be visualized, extending the usability of the method to different areas of modern Artificial Intelligence, including the problematic Explainable AI. In this context, further analyses could be conducted on diverse tasks, and observe how successful this type of approaches can be.

In particular, during the various experiments, it has been possible to directly visualize the co-working of different residual blocks in the classification task, leading to valuable observations on how relevant specific residual blocks were to the classification of diverse datasets.

This observation led to important simplifications, enabling the tokenization process to focus solely on the feature-generating and classification layers, showing the immense adaptability of the method. As a result, the framework achieved highly efficient operation across more than 300 ResNet18 versions, with training time constrained only by the storage and retrieval of the extremely high amount of tokens.

Building on this experimental foundation, this section introduces key applications of the proposed framework, highlighting both its strengths and limitations. These insights open clear avenues for future research and have the potential to be applied across multiple areas of Artificial Intelligence.

5.1 Core Contributions and Applications

The analyses conducted in this thesis produced several contributions, particularly in the domains of Machine Unlearning and Weight Learning. These contributions include novel ideas, critical concerns, and valuable insights that extend existing research.

First, the concept of image-free weight learning, initially introduced by Christensen and his collaborators for Zero-Shot Learning in computer vision classification, was extended to the more complex task of Machine Unlearning. This extension revealed important challenges, showing that even small differences between tasks can require substantially different representations within weight space.

Then, a thorough analysis of major Machine Unlearning methods was conducted across a wider and more diverse set of data distributions, confirming the robustness of some established methods in the literature, while demoting others in more specific scenario like the one of fine-grained classification.

Moreover, the introduction of feature-related vulnerabilities was introduced, allowing the distinction between the respective roles of the backbone and classification head in unlearning. It was shown that modifying only one of these two core components results in either incomplete or catastrophic unlearning. Specifically, restricting modifications to the classification head led to mere output suppression (as in Finetuning and HyCUS), whereas overly intrusive modifications to the backbone necessitated subsequent finetuning of the classification head (as in NegGrad [13, 1, 28] and Bad Teacher [6]). However, by introducing finetuning steps (as in NegGrad+ and SCRUB [17]), it was possible to achieve near-optimal unlearning performances.

Given the analyzed methods, a new set of evaluation metrics was proposed, enabling a comprehensive assessment of Machine Unlearning methods without requiring retraining. By combining Retain and Forget Accuracies with both loss-MIA and feature-MIA, it became possible to evaluate unlearning quality across both the backbone and classification head, demonstrating that, since membership inference attacks can expose separable behaviors of trained models on training versus test data, effective unlearning must ensure thorough removal from both feature space and classification probability distributions.

Finally, a unified perspective on the weight-learning paradigm was introduced, presenting it as a practical means for rapidly adapting pre-trained models to new tasks. The resulting framework embodies a flexible neuron-centric generative approach with potential applications and contributions across diverse domains of Artificial Intelligence. In this perspective, new directions and applications can be explored in future researches in many sub-fields of modern Artificial Intelligence like the ones illustrated below.

5.1.1 Cross-Domain Learning

The first potential application, directly arising from the research conducted in this thesis, lies in supporting the adaptation of pretrained networks to tasks for which they were not originally designed. Importantly, this can be achieved without additional training or finetuning, even when the new task belongs to a different domain from the network’s initial purpose.

As demonstrated in this work, in fact, a weight-learning approach such as ICIS can be adapted, through suitable adjustments, to address a new task like Machine Unlearning. By extension, this adaptation process could also be applied to cross-domain scenarios. For instance, a Deep Neural Network, trained for the specific domain of classification, could be repurposed for an object recognition task, despite the difference in scope.

This kind of adaptation can be interpreted as a broader and more complex generalization of Zero-Shot Learning which focuses on adapting a pretrained model to new labels within the same domain (e.g., classification). Cross-domain learning, instead, requires adapting the model’s output to a different yet related task. Similarities can also be observed within the literature methods in this area often resembling those used in ZSL, with frameworks such as Zero-Shot and Few-Shot Learning commonly appearing in cross-domain research (for example, in the work of Paedeh et al. [22]).

The plausibility of this application is further supported by the results presented in Chapter 4. Specifically, even when trained on the specific task of classification with the same architecture, the framework exhibited consistent behavioral patterns when multiple datasets were integrated in the same training session. Notably, models trained on the CUB dataset, which constitutes the weakest performing case, showed relative improvements, whereas models trained on stronger datasets (aPY and AwA2) displayed slight underperformance, suggesting that the training on multiple datasets simultaneously acted as a soft form of influence during the reconstruction process.

Evidence for this effect is provided in table 27, where an increased number of transformer blocks, trained without augmentation, tended to produce slightly weaker reconstructions for the strongest-performing origins (aPY and AwA2), while generating stronger reconstructions for the weakest-performing origin (CUB). Since this configuration involved no augmentation, the only aspect that could affect the reconstruction is the coexistence of different dataset origins within the same hyper-representation space, which could have introduced a natural influence across reconstructions, effectively pulling them toward a shared centroid.

This inherent tendency of the framework could therefore be leveraged by explicitly incorporating cross-domain architectures. By carefully guiding the decoding process in a generative and task-specific manner, it may be possible to adapt pretrained networks across domains more effectively and with minimal computational overhead.

5.1.2 Explainable AI - XAI

A further promising application of the proposed framework derives directly from the structure of the hyper-representation space and its ability to substantially reduce the dimensionality of weight space through a transformer-based attention-aware encoding. In this setting, entire parameter sets can be treated as sequences of tokens, processed through a small number of transformer blocks in much the same way as words in a sentence.

This design enables the generation of meaningful visualizations of weight structures, offering new opportunities in the field of Explainable Artificial Intelligence (XAI). In fact, a central challenge of XAI has always been to understand how Deep Neural Networks internalize, structure, and utilize knowledge across tasks. Traditional approaches typically analyze activations or saliency maps in the input space, providing only task-dependent and often limited perspectives. In contrast, the framework explored in this thesis constructed a representation space where the roles of parameters emerged in a structured way, immediately aiding in the identification and resolution of problems that arose during experimentation.

In particular, as shown in Chapter 4, the hyper-representation space, when trained to reconstruct origins trained on different datasets, showed clear distinctions of generic convolutional layers in the first stages of the network, and more specific divisions in deeper ones. These insights offered an immediate debugging showing both why HyCUS failed in effectively removing data from the backbone, and also directly showed the information that was necessary to solve the issue. This decomposition provides a novel entry point for interpretability, revealing structured patterns that remain otherwise inaccessible when analyzing only inputs and outputs.

Building on these insights, XAI could benefit from the following contributions:

- By isolating embeddings associated with specific tasks, it becomes possible to trace how task-relevant computations emerge and evolve in the hyper-representation space.
- The tokenization process, by structuring weights into interpretable units, offers a neuron-centric perspective that can expose functional similarities and differences across models.
- The generative nature of the framework allows for a manifold of different explorations: embeddings can be perturbed, recombined, or suppressed to directly test their influence on model behavior.

From this perspective, the introduced framework, not only provides tools to solve specific tasks, but also for *explaining* the internal structure of neural networks when dealing with them: as the hyper-representation space is constructed from a loss, derived by attention-aware computations, it is possible to directly link each embedding to the final generative process, resulting in an hyper-space that immediately highlights the parameter-to-task correlation. Extending the applications of this framework toward this direction, it has the potential to contribute to the broader goals of XAI, allowing to restrict the gap between black-box performance and transparent understanding of model behavior.

5.1.3 Federated Learning

Another promising application of the simplified framework lies in the domain of Federated Learning. This setting aims to collaboratively train a global model in a decentralized manner by distributing the training process across multiple *clients*.

Two fundamental challenges, both directly tied to data, can be identified in this context: (i.) **data heterogeneity**, where each client typically has access only to a portion of the dataset, with the risk that some subsets may be entirely absent. (ii.) **the strict data privacy constraint**, which prevents clients from exchanging raw data. As a result, the coordinating server, which is responsible for aggregating client updates into a unified model, must integrate knowledge from diverse sources **without direct access to their training data**.

In this scenario, the framework introduced in the previous chapter appears particularly well suited as its central objective is to learn a hyper-representation space in which multiple versions of the same network architecture are encoded into structured blocks of embeddings. As shown, this space tends to concentrate information into a dense set of generic-purpose embeddings, with only a limited number of task-specific ones, naturally influencing the generated model to have merged embeddings with other versions.

Leveraging this property could enable a task-specific training process in which hyper-representations from different local models are merged. Such an approach would address several challenges simultaneously:

- Local training could proceed independently, without the need to store or transmit any additional information about the local training process.
- Clients would share only the hyper-representations of their models, rather than raw parameters thereby reducing the risk of privacy leakage, as illustrated in Chapter 3 through Membership Inference Attacks.
- The coordinator could securely and efficiently integrate local contributions without requiring retraining or additional data.

From this perspective, the autoencoder could be conceptualized as a form of distributed security mechanism. Specifically, the encoder would reside on local clients, generating hyper-representations of their trained models, while the coordinator, holding the decoder, would integrate these into the evolving global model.

By applying a centroid-based influence during decoding, the coordinator could align the global representation with client contributions in a secure and controlled manner, even in online scenarios by moving the global representation towards the local parts as they are received.

5.1.4 Continuous Learning

Another natural extension of the proposed framework is in the area of Continuous Learning, where the objective is to enable a model to adapt progressively to new tasks or evolving environments without requiring complete retraining. This field is particularly challenging, as it demands not only efficient integration of new knowledge but also the preservation of previously acquired capabilities, avoiding issues such as catastrophic forgetting, already introduced and addressed in section 3.8.

The framework explored in chapter 4 provides a compelling basis for such an approach: the hyper-representation space encodes multiple versions of the same network architecture into structured embeddings, naturally organizing information into generic-purpose and task-specific components. This characteristic lends itself to incremental adaptation, where new knowledge can be incorporated by manipulating embeddings rather than by retraining the entire model.

A promising direction for exploration in this scenario involves the use of a **genetic approach** within the hyper-representation space. Here, embeddings can be treated as genetic strings that may be combined, mutated, and evolved to explore new solutions:

- At a **neuron level**, where individual embeddings could be recombined to create novel embedding units, extending the representational capacity of the network in a classical genetic algorithm fashion. In this sense, the hyper-representation space could work as the initial population and could be extended as new options are explored.
- At a **model level**, where hyper-representations of entire networks, which can be seen as structured matrices of embeddings, could be explored to generate new models by combining the matrices over the column dimension. Here the matrix, representing the trained model in the hyper-representation space, could be seen as a form of networks DNA, with an initial population that could be obtained by exploiting the augmentation proposed in chapter 4

This dual strategy would allow exploration at multiple granularities, from fine-grained modifications of individual embeddings to large-scale structural adaptations across models through a more advanced pretraining of the proposed simplified framework. With the decoding phase acting as the mechanism for translating evolved embeddings back into functional network weights, meaningful solutions could be explored by exploiting the generative nature of the framework.

By adopting this genetic perspective, Continuous Learning could be realized as an ongoing evolutionary process in the hyper-representation space as well as an online process where the trained framework moves the model across the hyper-representation space as new feedbacks are received. In both those scenarios many explorations would be possible with a first static version enabling Continuous Learning, and a second natural evolution that may also adapt and evolve the hyper-representation space itself.

5.2 Future Directions and Final Thoughts

This thesis has critically examined the dominant data-centric paradigm in Deep Learning, highlighting how its reliance on large datasets introduces practical, ethical, and reproducibility challenges. In response, the Weight-as-Data paradigm has been explored as a new direction, where trained weights are treated as natural encodings of the data that produced them.

This alternative view has been investigated in the contexts of Zero-Shot Learning and Machine Unlearning, demonstrating both opportunities and vulnerabilities. In particular, it was shown that evaluation strategies such as membership inference attacks must extend beyond the output layer to the feature space, as otherwise hidden weaknesses may remain undetected.

From this perspective, a refined understanding of Machine Unlearning emerges: it should not merely suppress outputs, but instead ensure that forgotten labels are indistinguishable from labels never seen during training. This opens to a possible future line of investigation, such as designing regressors that discriminate between forgotten and unseen labels, to provide more rigorous guarantees of successful unlearning. It should be in fact impossible for a regressor, in a correctly unlearned model, to clearly distinguish between different guesses of what was forgotten and what was truly unseen by the model.

Beyond unlearning, this work also points toward contributions in Explainable AI. The hyper-representation space made it possible to immediately recognize and resolve issues that arose during experimentation. This suggests that more advanced visualization strategies could turn the framework into a tool for debuggability and problem identification, enabling researchers and practitioners to identify and correct failures much easier, even in complex scenarios.

At the same time, an important limitation must be acknowledged. The current framework captures relationships between weights in a largely non-causal way, which may lead to impoverished representations of interactions across layers. The use of positional encodings that include the layer index partially addresses this issue, but future research could extend this idea toward more precise encodings of inter-layer dependencies, ensuring that structural relationships are preserved as faithfully as possible.

Looking ahead, the insights of this work suggest that weights are not passive parameters, but active carriers of structured knowledge. Future research should therefore continue to develop frameworks that exploit this latent structure while ensuring robustness, privacy, and generalizability.

Ultimately, this thesis highlights that addressing the limitations of current Deep Learning will require not only technical advances, but also conceptual shifts in how we understand what a model knows. By treating weights as data even in a post-hoc manner, this work contributes to that shift, offering both practical tools and a foundation for future exploration.

References

- [1] Amnesiac machine learning. 35:11516–11524.
- [2] S. K. Ainsworth, J. Hayase, and S. Srinivasa. Git re-basin: Merging models modulo permutation symmetries, 2023.
- [3] R. Brochier, A. Guille, and J. Velcin. Global vectors for node representations. In *The World Wide Web Conference*, WWW '19, page 2587–2593. ACM, May 2019.
- [4] S. Chen, W. Hou, S. Khan, and F. S. Khan. Progressive semantic-guided vision transformer for zero-shot learning, 2024.
- [5] A. Christensen, M. Mancini, A. S. Koepke, O. Winther, and Z. Akata. Image-free classifier injection for zero-shot classification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19072–19081, October 2023.
- [6] V. S. Chundawat, A. K. Tarun, M. Mandal, and M. Kankanhalli. Can bad teaching induce forgetting? unlearning in deep networks using an incompetent teacher, May 2023.
- [7] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [8] J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018.
- [9] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [10] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur. Sharpness-aware minimization for efficiently improving generalization, 2021.
- [11] T. Garipov, P. Izmailov, D. Podoprikin, D. Vetrov, and A. G. Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns, 2018.
- [12] S. Gidaris and N. Komodakis. Generating classification weights with gnn denoising autoencoders for few-shot learning, 2019.
- [13] A. Golatkar, A. Achille, and S. Soatto. Eternal sunshine of the spotless net: Selective forgetting in deep networks, 2020.
- [14] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015.
- [15] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson. Averaging weights leads to wider optima and better generalization, 2019.

- [16] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang. On large-batch training for deep learning: Generalization gap and sharp minima, 2017.
- [17] M. Kurmanji, P. Triantafillou, J. Hayes, and E. Triantafillou. Towards unbounded machine unlearning, Oct 2023.
- [18] T. Mensink, E. Gavves, and C. G. Snoek. Costa: Co-occurrence statistics for zero-shot classification. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 2441–2448, 2014.
- [19] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space, 2013.
- [20] T. Mori and M. Ueda. Improved generalization by noise enhancement, 2020.
- [21] M. Norouzi, T. Mikolov, S. Bengio, Y. Singer, J. Shlens, A. Frome, G. S. Corrado, and J. Dean. Zero-shot learning by convex combination of semantic embeddings, 2014.
- [22] N. Paeedeh, M. Pratama, M. A. Ma’sum, W. Mayer, Z. Cao, and R. Kowalczyk. Cross-domain few-shot learning via adaptive transformer networks, 2024.
- [23] W. Peebles and S. Xie. Scalable diffusion models with transformers, 2023.
- [24] Q. Qian and J. Hu. Online zero-shot classification with clip, 2024.
- [25] G. Raghavan and M. Thomson. Solving hybrid machine learning tasks by traversing weight space geodesics, 2021.
- [26] J. M. L. Rangel, S. Schoepf, J. Foster, D. Krueger, and U. Anwar. Learning to forget using hypernetworks, 2024.
- [27] K. Schürholt, M. W. Mahoney, and D. Borth. Towards scalable and versatile weight space learning, 2024.
- [28] A. Thudi, G. Deza, V. Chandrasekaran, and N. Papernot. Unrolling sgd: Understanding factors influencing machine unlearning, 2022.
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023.
- [30] Y. Xian, C. H. Lampert, B. Schiele, and Z. Akata. Zero-shot learning—a comprehensive evaluation of the good, the bad and the ugly. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(9):2251–2265, Sep 2019.
- [31] W. Ågren. The nt-xent loss upper bound, 2022.